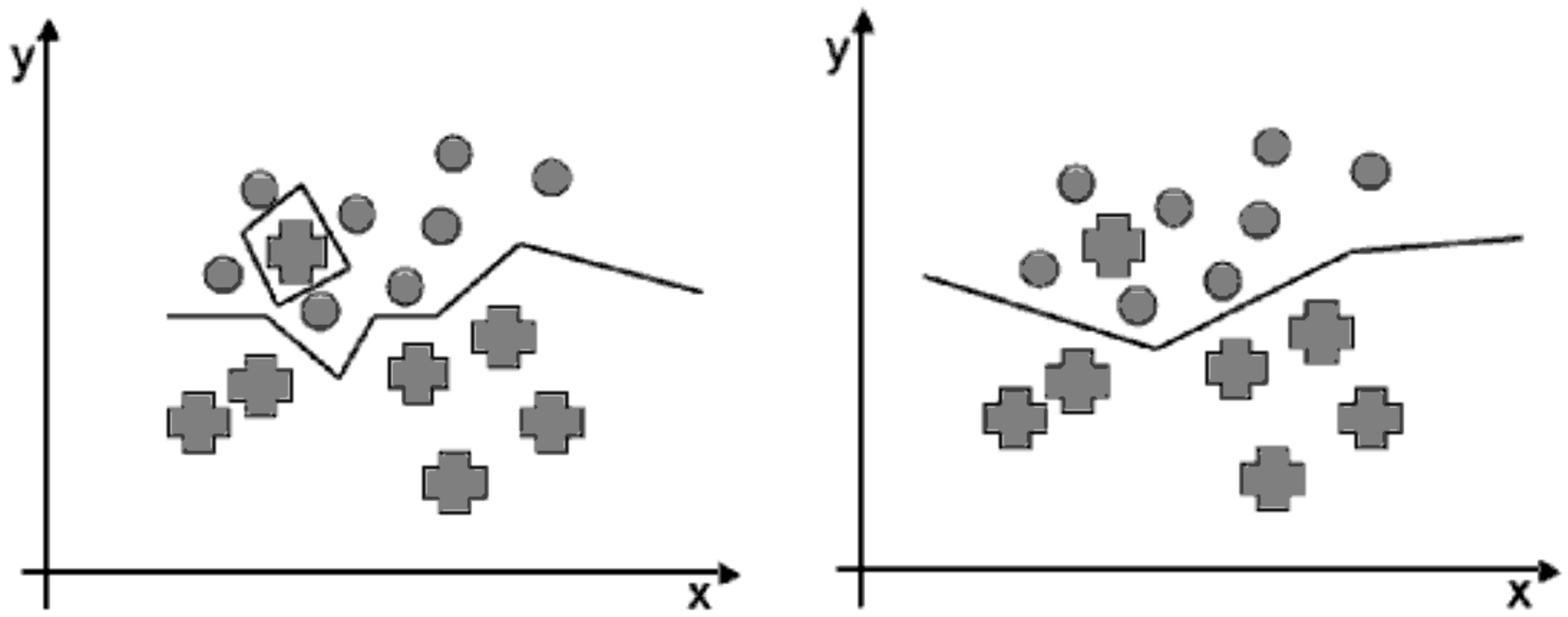


FIGURE 8.9: The nearest neighbours decision boundary with *left:* one neighbour and *right:* two neighbours.

grows. This is not as bad as it might appear at first: there are a set of methods such as KD-Trees (see Section 8.4.2 for more details) that compute this in $\mathcal{O}(N \log N)$ time. However, more importantly, as the number of dimensions increases, so the distance to other datapoints tends to increase. In addition, they can be far away in a variety of different directions—there might be points that are relatively close in some dimensions, but a long way in others. There are methods for dealing with these problems, known as adaptive nearest neighbour methods, and there is a reference to them in the Further Reading section at the end of the chapter.

The only part of this that requires any care during the implementation is what to do when there is more than one class found in the closest points, but even with that the implementation is nice and simple:

```
def knn(k,data,dataClass,inputs):

    nInputs = shape(inputs)[0]
    closest = zeros(nInputs)

    for n in range(nInputs):
        # Compute distances
        distances = sum((data-inputs[n,:])**2,axis=1)

        # Identify the nearest neighbours
        indices = argsort(distances,axis=0)

        classes = unique(dataClass[indices[:k]])
        if len(classes)==1:
            closest[n] = unique(classes)
        else:
            counts = zeros(max(classes)+1)
```

Copyrighted material

```
    for i in range(k):
        counts[dataClass[indices[i]]] += 1
        closest[n] = max(counts)

return closest
```

We are going to look next at how we can use these methods for regression, before we turn to the question of how to perform the distance calculations as efficiently as possible, something that is done simply but inefficiently in the code above. We will then consider briefly whether or not the Euclidean distance is always the most useful way to calculate distances, and what alternatives there are.

8.4.1 Nearest Neighbour Smoothing

Nearest neighbour methods can also be used for regression by returning the average value of the neighbours to a point, or a spline or similar fit as the new value. The most common methods are known as kernel smoothers, and they use a kernel (a weighting function between pairs of points) that decides how much emphasis (weight) to put onto the contribution from each datapoint according to its distance from the input. We will see much more about kernels in a different context in Section 5.2, but here we shall simply use two kernels that are used for smoothing.

Both of these kernels are designed to give more weight to points that are closer to the current input, with the weights decreasing smoothly to zero as they pass out of the range of the current input, with the range specified by a parameter λ. They are the Epanechnikov quadratic kernel:

$$K_{E,\lambda}(x_0, x) = \begin{cases} 0.75\left(1 - (x_0 - x)^2/\lambda^2\right) & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}, \tag{8.25}$$

and the tricube kernel:

$$K_{T,\lambda}(x_0, x) = \begin{cases} \left(1 - \left|\frac{x_0 - x}{\lambda}\right|^3\right)^3 & \text{if } |x - x_0| < \lambda \\ 0 & \text{otherwise} \end{cases}. \tag{8.26}$$

The results of using these kernels are shown in Figure 8.10 on a dataset that consists of the time between eruptions (technically known as the repose) and the duration of the eruptions of Mount Ruapehu, the large volcano in the centre of New Zealand's north island. Values of λ of 2 and 4 were used here. Picking λ requires experimentation. Large values average over more datapoints, and therefore produce lower variance, but at the cost of higher bias.

Copyrighted material

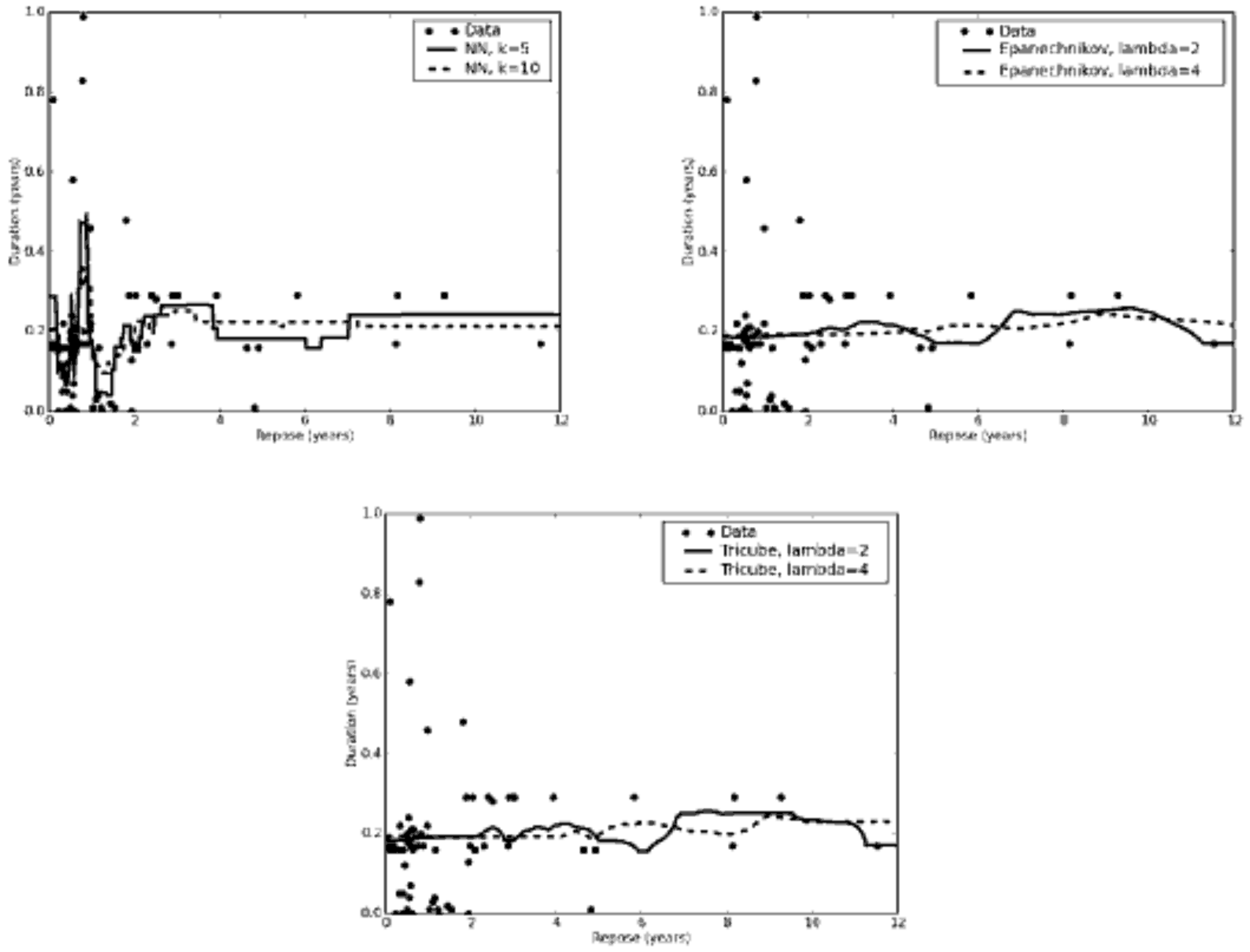


FIGURE 8.10: Output of the nearest neighbour method and two kernel smoothers on the data of duration and repose of eruptions of Mount Ruapehu 1860–2006.

8.4.2 Efficient Distance Computations: the KD-Tree

As was mentioned above, computing the distances between all pairs of points is very computationally expensive. Fortunately, as with many problems in computer science, designing an efficient data structure can reduce the computational overhead a lot. For the problem of finding nearest neighbours the data structure of choice is the KD-Tree. It has been around since the late 1970s, when it was devised by Friedman and Bentley, and it reduces the cost of finding a nearest neighbour to $\mathcal{O}(\log N)$ for $\mathcal{O}(N)$ storage. The construction of the tree is $\mathcal{O}(N \log^2 N)$, with much of the computational cost being in the computation of the median, which with a naïve algorithm requires a sort and is therefore $\mathcal{O}(N \log N)$, or can be computed with a randomised algorithm in $\mathcal{O}(N)$ time.

The idea behind the KD-tree is very simple. You create a binary tree by choosing one dimension at a time to split into two, and placing the line through the median of the point coordinates of that dimension. Not that different to a decision tree (Chapter 6), really. The points themselves end up as leaves of the tree. Making the tree follows pretty much the same steps as usual for constructing a binary tree: we identify a place to split into two choices, left and right, and then carry on down the tree. This makes it natural to write the algorithm recursively. The choice of what to split and where is what makes

Copyrighted material

the KD-tree special. Just one dimension is split in each step, and the position of the split is found by computing the median of the points that are to be split in that one dimension, and putting the line there. In general, the choice of which dimension to split alternates through the different choices, or it can be made randomly. The algorithm below cycles through the possible dimensions based on the depth of the tree so far, so that in two dimensions it alternates horizontal and vertical splits.

The centre of the construction method is simply a recursive function that picks the axis to split on, finds the median value on that axis, and separates the points according to that value, which in Python can be written as:

```
# Pick next axis to split on
whichAxis = mod(depth,shape(points)[1])

# Find the median point
indices = argsort(points[:,whichAxis])
points = points[indices,:]
median = ceil(float(shape(points)[0]-1)/2)

# Separate the remaining points
goLeft = points[:median,:]
goRight = points[median+1:,:]

# Make a new branching node and recurse
newNode = node()
newNode.point = points[median,:]
newNode.left = makeKDtree(goLeft,depth+1)
newNode.right = makeKDtree(goRight,depth+1)
return newNode
```

Suppose that we had seven two-dimensional points to make a tree from: $(5,4),(1,6),(6,1),(7,5),(2,7),(2,2),(5,8)$ (as plotted in Figure 8.11). The algorithm will pick the first coordinate to split on initially, and the median point here is 5, so the split is through $x = 5$. Of those on the left of the line, the median y coordinate is 6, and for those on the right it is 5. At this point we have separated all the points, and so the algorithm terminates with the split shown in Figure 8.12 and the tree shown in Figure 8.13.

Searching the tree is the same as any other binary tree; we are more interested in finding the nearest neighbours of a test point. This is fairly easy: starting at the root of the tree you recurse down through the tree comparing just one dimension at a time until you find a leaf node that is in the region containing the test point. Using the tree shown in Figure 8.13 we introduce the testpoint $(3,5)$, which finds $(2,2)$ as the leaf for the box that $(3,5)$ is in. However, looking at Figure 8.14 we see that this is not the closest point at

Copyrighted material

Hidden page

all, so we need to do some more work.

The first thing we do is label the leaf we have found as a potential nearest neighbour, and compute the distance between the testpoint and this point, since any other point has to be closer. Now we need to check any other boxes that could contain something closer. Looking at Figure 8.14 you can see that point $(3, 7)$ is closer, and that is the label of the leaf for the sibling box to the one that was returned, so the algorithm also needs to check the sibling box. However, suppose that we used $(4.5, 2)$ as the testpoint. In that case the sibling is too far away, but another point $(6, 1)$ is closer. So just checking the sibling is not enough — we also need to check the sibling of the parent node, together with its descendants (the cousins of the first point). A look at the figure again should convince you that the algorithm can then terminate. This leads to the following Python program:

```
def returnNearest(tree,point,depth):
    if tree.left is None:
        # Have reached a leaf
        distance = sum((tree.point-point)**2)
        return tree.point,distance,0
    else:
        # Pick next axis to split on
        whichAxis = mod(depth,shape(point)[0])

        # Recurse down the tree
    if point[whichAxis]<tree.point[whichAxis]:
        bestGuess,distance,height = returnNearest(tree.left,
        point,depth+1)
    else:
        bestGuess,distance,height = returnNearest(tree.right,
        point,depth+1)

    if height<=2:
        # Check the sibling
        if point[whichAxis]<tree.point[whichAxis]:
            bestGuess2,distance2,height2 = returnNearest(tree.
            right,point,depth+1)
        else:
            bestGuess2,distance2,height2 = returnNearest(tree.
            left,point,depth+1)

    # Check this node
    distance3 = sum((tree.point-point)**2)
    if (distance3<distance2):
        distance2 = distance3
        bestGuess2 = tree.point
```

Copyrighted material

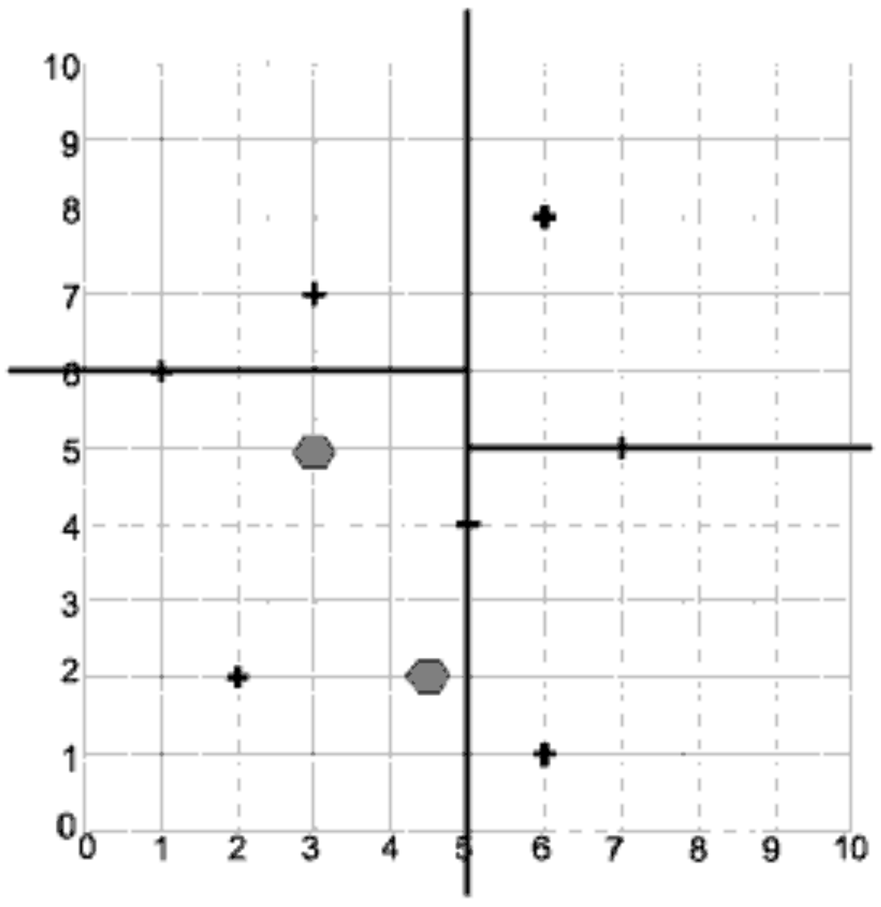


FIGURE 8.14: Two testpoints for the example KD-tree.

```
if (distance2<distance):
    distance = distance2
    bestGuess = bestGuess2
return bestGuess,distance,height+1
```

8.4.3 Distance Measures

We have computed the distance between points as the Euclidean distance, which is something that you learnt about in high school. However, it is not the only option, nor is it necessarily the most useful. In this section we will look at the underlying idea behind distance calculations and possible alternatives.

If I were to ask you to find the distance between my house and the nearest shop, then your first guess might involve taking a map of my town, locating my house and the shop, and using a ruler to measure the distance between them. By careful application of the map scale you can now tell me how far it is. However, when I set out to buy some milk I'm liable to find that I have to walk rather further than you've told me, since the direct line that you measured would involve walking through (or over) several houses, and some serious fence-scaling. Your 'as the crow flies' distance is the shortest possible path, and it is the straight-line, or Euclidean, distance. You can measure it on the map by just using a ruler, but it essentially consists of measuring the distance in one direction (we'll call it north-south) and then the distance in another direction that is perpendicular to the first (let's call it east-west) and then squaring them, adding them together, and then taking the square root of that. Writing that out, the Euclidean distance that we are all used to is:

Copyrighted material

Hidden page

If we put $k = 1$ then we get the city-block distance (Equation (8.28)), and $k = 2$ gives the Euclidean distance (Equation (8.27)). Thus, you might possibly see the Euclidean metric written as the L_2 norm and the city-block distance as the L_1 norm. These norms have another interesting feature. Remember that we can define different averages of a set of numbers. If we define the average as the point that minimises the sum of the distance to every data point, then it turns out that the mean minimises the Euclidean distance (the sum-of-squares distance), and the median minimises the L_1 metric. We met another distance measure earlier in the chapter: the Mahalanobis distance in Section 8.2.2.

There are plenty of other possible metrics to choose, depending upon the data space. We generally assume that the space is flat (if it isn't, then none of these techniques work, and we don't want to worry about that). However, it can still be beneficial to look at other metrics. Suppose that we want our classifier to be able to recognise images, for example of faces. We take a set of digital photos of faces and use the pixel values as features. Then we use the nearest neighbour algorithm that we've just seen to identify each face. Even if we ensure that all of the photos are taken fully face-on, there are still a few things that will get in the way of this method. One is that slight variations in the angle of the head (or the camera) could make a difference, another is that different distances between the face and the camera (scaling) will change the results, and another is that different lighting conditions will make a difference. We can try to fix all of these things in preprocessing, but there is also another alternative: use a different metric that is invariant to these changes, i.e., it does not vary as they do. The idea of invariant metrics is to find measures that ignore changes that you don't want. So if you want to be able to rotate shapes around and still recognise them, you need a metric that is invariant to rotation.

A common invariant metric in use for images is the tangent distance, which is an approximation to the Taylor expansion in first derivatives, and works very well for small rotations and scalings; for example, it was used to halve the final error rate on nearest neighbour classification of a set of handwritten letters. Invariant metrics are an interesting topic for further study, and there is a reference for them in the Further Reading section if you are interested.

Further Reading

For more on nearest neighbour methods, see:

- T. Hastie and R. Tibshirani. Discriminant adaptive nearest neighbor classification and regression. In David S. Touretzky, Michael C. Mozer,

Copyrighted material

Hidden page

Problem 8.3 Use the naïve Bayes classifier on the datasets that you used for the decision tree (this will involve some effort in turning the textual data into probabilities) and compare the results.

Problem 8.4 Extend the Gaussian Mixture Model algorithm to allow for more than two classes in the data. This is not trivial, since it involves modifying the EM algorithm.

Problem 8.5 Modify the KD-tree algorithm so that it works on spheres in the data, rather than rectangles. Since they no longer cover the space you will have to add some cases that fail to return a leaf at all. However, this means that the algorithm will not return points that are far away, which will make the results more accurate. Now modify it so that it does not use the Euclidean distance, but rather the L_1 distance. Compare the results of using these two methods on the `iris` dataset.

Problem 8.6 Use the small figures of numbers that are available on the book website in order to compute the tangent distance. You will have to write code that rotates the numbers by small amounts in order to check that you have written it correctly. What happens when you make large rotations (particularly of a 6 or 9)? Compare using nearest neighbours with Euclidean distance and the tangent distance to verify the results claimed in the chapter. Extend the experiment to the MNIST dataset.

Copyrighted material

Chapter 9

Unsupervised Learning

The learning algorithms that we have seen to date have made use of a training set that consists of a collection of labelled **target** data. This is obviously useful, since it enables us to show the algorithm the correct answer to sample data, but in many circumstances it is difficult to obtain—it could, for instance, involve somebody labelling each instance by hand. In addition, it doesn't seem to be very biologically plausible: most of the time when we are learning, we don't get told exactly what the right answer should be. In this chapter we will consider exactly the opposite case, where there is no information about the correct outputs available at all, and the algorithm is left to spot some similarity between different inputs for itself. We will see some more methods of doing this in Chapter 10, while in Chapter 13 we will consider the intermediate possibility, where the system receives information about whether or not it is correct, but is not told how to correct errors.

Unsupervised learning is a conceptually different problem. Obviously, we can't hope to perform regression—we don't know the outputs for any points, so we can't guess what the function is. Can we hope to do classification then? The aim of classification is to identify similarities between inputs that belong to the same class. There isn't any information about the correct classes, but if the algorithm can exploit similarities between inputs in order to **cluster** inputs that are similar together, this might perform classification automatically. So the aim of unsupervised learning is to find clusters of similar inputs in the data without being explicitly told that these datapoints belong to one class and those to a different class. Instead, the algorithm has to discover the similarities for itself.

The supervised learning algorithms that we have discussed so far have aimed to minimise some external error criterion—mostly the sum-of-squares error—based on the difference between the targets and the outputs. Calculating and minimising this error was possible because we had target data to calculate it from, which is not true for unsupervised learning. This means that we need to find something else to drive the learning. The problem is more general than sum-of-squares error: we can't use any error criterion that relies on targets or other outside information (an **external** error criterion), we need to find something internal to the algorithm. This means that the measure has to be independent of the task, because we can't keep on changing the whole algorithm every time a new task is introduced. In supervised learning the

Copyrighted material

error criterion was task-specific, because it was based on the target data that we provided.

To see how to work out a general error criterion that we can use, we need to go back to some of the important concepts that were discussed in Section 4.1.1: input space and weight space. If two inputs are close together then it means that their vectors are similar, and so the distance between them is small (distance measures were discussed in Section 8.4.3, but here we will stick to Euclidean distance). Then inputs that are close together are identified as being similar, so that they can be clustered, while inputs that are far apart are not clustered together. We can extend this to the nodes of a network by aligning weight space with input space. Now if the weight values of a node are similar to the elements of an input vector then that node should be a good match for the input, and any other inputs that are similar. In order to start to see these ideas in practice we'll look at a simple clustering algorithm, the k-means algorithm, which has been around in statistics for a long time.

9.1 The k-Means Algorithm

If you have ever watched a group of tourists with a couple of tour guides who hold umbrellas up so that everybody can see them and follow them, then you have seen a dynamic version of the k-means algorithm. Our version is simpler, because the data (playing the part of the tourists) does not move, only the tour guides.

Suppose that we want to divide our input data into k categories, where we know the value of k (for example, we have a set of medical test results from lots of people for three diseases, and we want to see how well the tests identify the three diseases). We allocate k cluster centres to our input space, and we would like to position these centres so that there is one cluster centre in the middle of each cluster. However, we don't know where the clusters are, let alone where their 'middle' is, so we need an algorithm that will find them. Learning algorithms generally try to minimise some sort of error, so we need to think of an error criterion that describes this aim. The idea of the 'middle' is the first thing that we need to think about. How do we define the middle of a set of points? There are actually two things that we need to define:

A distance measure In order to talk about distances between points, we need some way to measure distances. It is often the normal Euclidean distance, but there are other alternatives; we've covered some other alternatives in Section 8.4.3.

The mean average Once we have a distance measure, we can compute the central point of a set of datapoints, which is the mean average (if you

Copyrighted material

aren't convinced, think what the mean of two numbers is, it is the point halfway along the line between them). Actually, this is only true in Euclidean space, which is the one you are used to, where everything is nice and flat. Everything becomes a lot trickier if we have to think about curved spaces; when we have to worry about curvature, the Euclidean distance metric isn't the right one, and there are at least two different definitions of the mean. So we aren't going to worry about any of these things, and we'll assume that space is flat. This is what statisticians do all the time.

We can now think about a suitable way of positioning the cluster centres: we compute the mean point of each cluster, $\boldsymbol{\mu}_{c(i)}$, and put the cluster centre there. This is equivalent to minimising the Euclidean distance (which is the sum-of-squares error again) from each datapoint to its cluster centre.

How do we decide which points belong to which clusters? It is important to decide, since we will use that to position the cluster centres. The obvious thing is to associate each point with the cluster centre that it is closest too. This might change as the algorithm iterates, but that's fine.

We start by positioning the cluster centres randomly though the input space, since we don't know where to put them, and then we update their positions according to the data. We decide which cluster each datapoint belongs to by computing the distance between each datapoint and all of the cluster centres, and assigning it to the cluster that is the closest. Note that we can reduce the computational cost of this procedure by using the KD-Tree algorithm that was described in Section 8.4.2. For all of the points that are assigned to a cluster, we then compute the mean of them, and move the cluster centre to that place. We iterate the algorithm until the cluster centres stop moving. Here is the algorithmic description:

The k-Means Algorithm

- **Initialisation**
  - choose a value for k
  - choose k random positions in the input space
  - assign the cluster centres $\boldsymbol{\mu}_j$ to those positions
- **Learning**
  - repeat
    - for each datapoint $\mathbf{x}_i$:
      - compute the distance to each cluster centre
      - assign the datapoint to the nearest cluster centre with distance

$$d_i = \min_j d(\mathbf{x}_i, \boldsymbol{\mu}_j). \tag{9.1}$$

Copyrighted material

        * for each cluster centre:
            · move the position of the centre to the mean of the points in that cluster (N_j is the number of points in cluster j):

$$\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{x}_i \tag{9.2}$$

    – until the cluster centres stop moving

- **Usage**

    – for each test point:

        * compute the distance to each cluster centre
        * assign the datapoint to the nearest cluster centre with distance

$$d_i = \min_j d(\mathbf{x}_i, \boldsymbol{\mu}_j). \tag{9.3}$$

The NumPy implementation follows these steps almost exactly, and we can take advantage of the `argmin()` function, which returns the index of the minimum value, to find the closest cluster. The code that computes the distances, finds the nearest cluster centre, and updates them can then be written as:

```
# Compute distances
distances = ones((1,self.nData))*sum((data-self.centres[0,:])
**2,axis=1)
for j in range(self.k-1):
    distances = append(distances,ones((1,self.nData))*sum((data-
    self.centres[j+1,:])**2,axis=1),axis=0)

# Identify the closest cluster
cluster = distances.argmin(axis=0)
cluster = transpose(cluster*ones((1,self.nData)))

# Update the cluster centres
for j in range(self.k):
    thisCluster = where(cluster==j,1,0)
    if sum(thisCluster)>0:
        self.centres[j,:] = sum(data*thisCluster,axis=0)/sum(
        thisCluster)
```

To see how this works in practice, Figures 9.1 and 9.2 show some data and some different ways to cluster that data computed by the k-means algorithm.

Copyrighted material

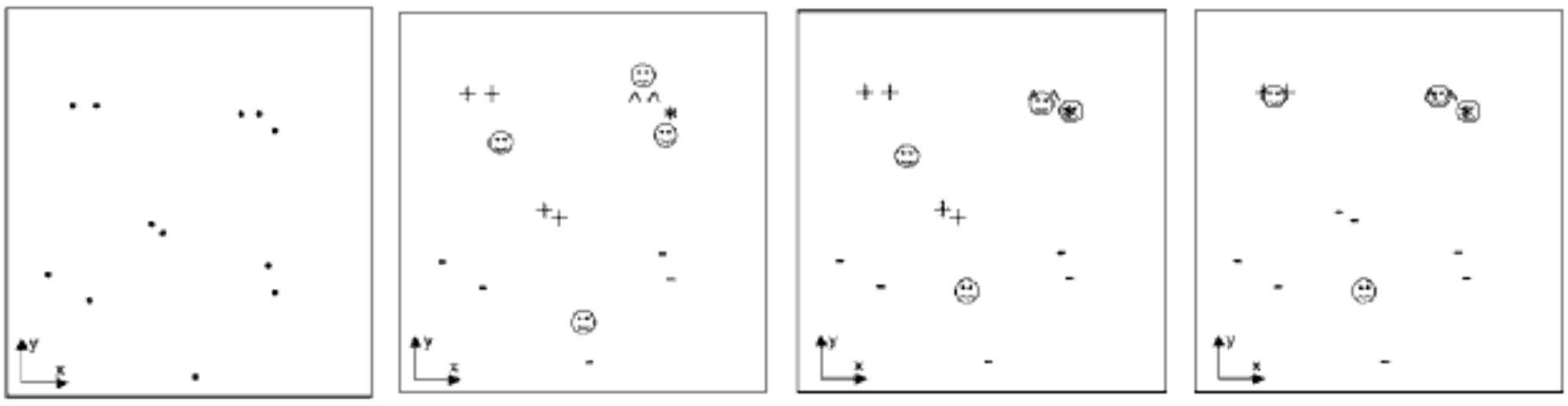


FIGURE 9.1: *Left:* A two-dimensional dataset. *Right:* Three possible ways to position 4 centres (drawn as faces) using the k-means algorithm, which is clearly susceptible to local minima.

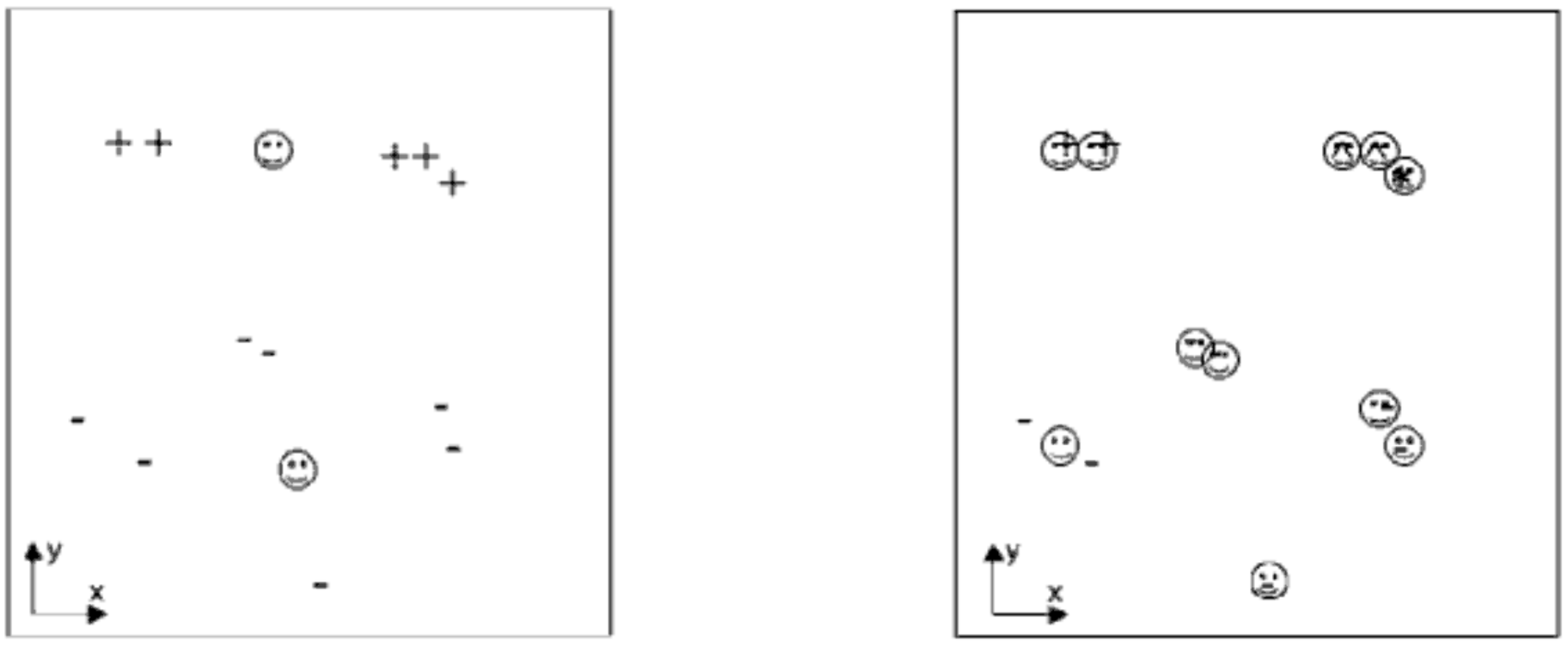


FIGURE 9.2: *Left:* A solution with only 2 classes, which does not match the data well. *Right:* A solution with 11 classes, showing severe overfitting.

It should be clear that the algorithm is susceptible to local minima: depending upon where the centres are initially positioned in the space, you can get very different solutions, and many of them look very unlikely to our eyes. Figure 9.2 shows examples of what happens when you choose the number of centres wrongly. There are certainly cases where we don't know in advance how many clusters we will see in the data, but the k-means algorithm doesn't deal with this at all well.

At the cost of significant extra computational expense, we can get around both of these problems by running the algorithm many different times. To find a good local optimum (or even the global one) we use many different initial centre locations, and the solution that minimises the overall sum-of-squares error is likely to be the best one.

By running the algorithm with lots of different values of k, we can see which values give us the best solution. Of course, we need to be careful with this. If we still just measure the sum-of-squares error, then when we set k to be equal to the number of datapoints, we can position one centre on every datapoint, and the sum-of-squares error will be zero (in fact, this won't happen, since the random initialisation will mean that several clusters will end up coinciding).

Copyrighted material

However, there is no generalisation in this solution (it is a case of serious overfitting). As usual, the answer is to use a validation set and monitor the error there.

9.1.1 Dealing with Noise

There are lots of reasons for performing clustering, but one of the more common ones is to deal with noisy data readings. These might be slightly corrupted, or occasionally just plain wrong. If we can choose the clusters correctly, then we have effectively removed the noise, because we replace each noisy datapoint by the cluster centre (we will use this way of representing datapoints for other purposes in Section 9.2). Unfortunately, the mean average, which is central to the k-means algorithm, is very susceptible to outliers, i.e., very noisy measurements. One way to avoid the problem is to replace the mean average with the median, which is what is known as a robust statistic, meaning that it is not affected by outliers (the mean of $(1, 2, 1, 2, 100)$ is 21.2, while the median is 2). The only change that is needed to the algorithm is to replace the computation of the mean with the computation of the median. This is computationally more expensive, as we've discussed previously, but it does remove noise effectively.

9.1.2 The k-Means Neural Network

The k-means algorithm clearly works, despite its problems with noise and the difficulty with choosing the number of clusters. Interestingly, while it might seem a long way from neural networks, it isn't. If we think about the cluster centres that we optimise the positions of as locations in weight space, then we could position neurons in those places and use neural network training. The computation that happened in the k-means algorithm was that each input decided which cluster centre it was closest to by calculating the distance to all of the centres. We could do this inside a neural network, too: the location of each neuron is its position in weight space, which matches the values of its weights. So for each input, we just make the activation of a node be the distance between that node in weight space and the current input, as we did for Radial Basis Functions in Chapter 4. Then training is just moving the position of the node, which means adjusting the weights.

So, we can implement the k-means algorithm using a set of neurons. We will use just one layer of neurons, together with some input nodes, and no bias node. The first layer will be the inputs, which don't do any computation, as usual, and the second layer will be a layer of competitive neurons, that is, neurons that 'compete' to fire, with only one of them actually succeeding. Only one cluster centre can represent a particular input vector, and so we will choose the neuron with the highest activation h to be the one that fires. This is known as winner-takes-all activation, and it is an example of competitive learning, since the set of neurons compete with each other to fire,

Copyrighted material

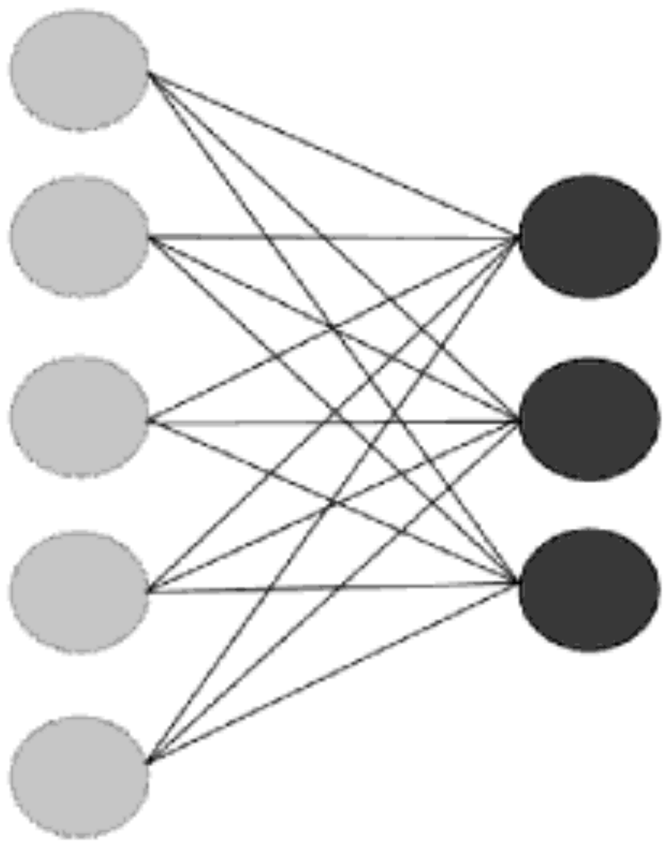


FIGURE 9.3: A single-layer neural network can implement the k-means solution.

with the winner being the one that best matches (i.e., is closest to) the input. Competitive learning is sometimes said to lead to grandmother cells, because each neuron in the network will learn to recognise one particular feature, and will fire only when that input is seen. You would then have a specific neuron that was trained to recognise your grandmother (and others for anybody else/anything else that you see often).

We will choose k neurons (for hopefully obvious reasons) and fully connect the inputs to the neurons, as usual. There is a picture of this network in Figure 9.3. We will use neurons with a linear transfer function, computing the activation of the neurons as simply the product of the weights and inputs:

$$h_i = \sum_j w_{ij} x_j. \tag{9.4}$$

Providing that the inputs are normalised so that their absolute size is the same (a point that we'll come back to in Section 9.1.3), this effectively measures the distance between the input vector and the cluster centre represented by that neuron, with larger numbers (higher activations) meaning that the two points are closer together.

So the winning neuron is the one that is closest to the current input. The question is how can we then change the position of that neuron in weight space, that is, how do we update its weights? In the k-means algorithm that was described earlier it was easy: we just set the cluster centre to be the mean of all the datapoints that were assigned to that centre. However, when we do neural network training, we often just feed in one input vector at a time and change the weights (that is, we use the algorithm on-line, rather than batch), we do not know the mean because we don't know about all the datapoints, just the current one. So we approximate it by moving the winning neuron closer to the current input, making that centre even more likely to be the best

Copyrighted material

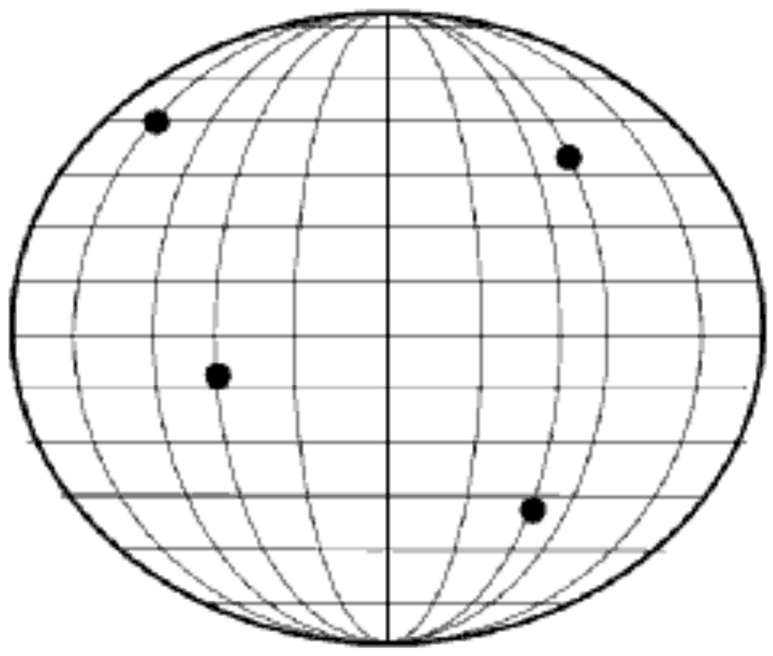


FIGURE 9.4: A set of neurons positioned on the unit sphere in 3D.

match next time that input is seen. This corresponds to:

$$\Delta w_{ij} = \eta x_j. \tag{9.5}$$

However, this is not good enough. To see why not, let's get back to that question of normalisation. This is important enough to need its own subsection.

9.1.3 Normalisation

Suppose that the weights of all the neurons are small (maybe less than 1) except for those to one particular neuron. We'll make those weights be 10 for the example. If an input vector with values $(0.2, 0.2, -0.1)$ is presented, and it happens to be an exact match for one of the neurons, then the activation of that neuron will be $0.2 \times 0.2 + 0.2 \times 0.2 + -0.1 \times -0.1 = 0.09$. The other neurons are not perfect matches, so their activations should all be less. However, consider the neuron with large weights. Its activation will be $10 \times 0.2 + 10 \times 0.2 + 10 \times -0.1 = 3$, and so it will be the winner. Thus, we can only compare activations if we know that the weights for all of the neurons are the same size. We do this by insisting that the weight vector is normalised so that the distance between the vector and the origin (the point $(0, 0, \ldots 0)$) is one. This means that all of the neurons are positioned on the unit hypersphere, which we described in Section 4.3 when we talked about the curse of dimensionality: it is the set of all points that are distance one from the origin, so it is a circle in 2D, a sphere in 3D (as shown in Figure 9.4), and a hypersphere in higher dimensions.

Computing this normalisation in NumPy takes a little bit of care because we are normalising the total Euclidean distance from the origin, and the sum and division are row-wise rather than column-wise, which means that the matrix has to be transposed before and after the division:

Copyrighted material

```
normalisers=sqrt(sum(data**2,axis=1))*ones((1,shape(data)[0]))
data = transpose(transpose(data)/normalisers)
```

The neuronal activation (Equation (9.4)) can be written as:

$$h_i = \mathbf{W}_i^T \cdot \mathbf{x}, \tag{9.6}$$

where, as usual, $\cdot$ refers to the inner product or scalar product between the two vectors, and $\mathbf{W}_i^T$ is the transpose of the ith row of W. The inner product computes $\|\mathbf{W}_i\|\|\mathbf{x}\|\cos\theta$, where θ is the angle between the two vectors and $\|\cdot\|$ is the magnitude of the vector. So if the magnitude of all the vectors is one, then only the angle θ affects the size of the dot product, and this tells us about the difference between the vector directions, since the more they point in the same direction, the larger the activation will be.

9.1.4 A Better Weight Update Rule

The weight update rule given in Equation (9.5) lets the weights grow without any bound, so that they do not lie on the unit hypersphere any more. If we normalise the inputs as well, which certainly seems reasonable, then we can use the following weight update rule:

$$\Delta w_{ij} = \eta(x_j - w_{ij}), \tag{9.7}$$

which has the effect of moving the weight w_{ij} directly towards the current input. Remember that the only weights that we are updating are those of the winning unit:

```
for i in range(self.nEpochs):
    for j in range(self.nData):
        activation = sum(self.weights*transpose(data[j:j+1,:]),
        axis=0)
        winner = argmax(activation)
        self.weights[:,winner] += self.eta * data[j,:] - self.
        weights[:,winner]
```

For many of our supervised learning algorithms we minimised the sum-of-squares difference between the output and the target. This was a global error criterion that affected all of the weights together. Now we are minimising a function that is effectively independent in each weight. So the minimisation that we are doing is actually more complicated, even though it doesn't look it. This makes it very difficult to analyse the behaviour of the algorithm, which is a general problem for competitive learning algorithms. However, they do tend to work well.

Copyrighted material

Now that we have a weight update rule that works, we can consider the entire algorithm for the on-line k-means network.

The On-Line k-Means Algorithm

- **Initialisation**
  - choose a value for k, which corresponds to the number of output nodes
  - initialise the weights to have small random values
- **Learning**
  - normalise the data so that all the points lie on the unit sphere
  - repeat:
    - for each datapoint:
      - compute the activations of all the nodes
      - pick the winner as the node with the highest activation
      - update the weights using Equation (9.7)
    - until number of iterations is above a threshold
  - **Usage**
    - for each test point:
      - compute the activations of all the nodes
      - pick the winner as the node with the highest activation

9.1.5 Example: The Iris Dataset Again

Now that we have a method of training the k-means algorithm we can use it to learn about data. Except we need to think about how to understand the results. If there aren't any labels in the data, then we can't really do much to analyse the results, since we don't have anything to compare them with. However, we might use unsupervised learning methods to cluster data where we know at least some of the labels. For example, we can use the algorithm on the iris dataset that we looked at in Section 3.4.3, where we classified three types of iris flowers using the MLP. All we need to do is to give some of the data to the algorithm and train it, and then use some more to test the output. However, the output of the algorithm isn't as clear now, because we don't use the labels that come with the data, since we aren't doing supervised learning anymore. To get around that, we need to work out some way of turning the results from the algorithm, which is the index of the cluster that best matches it, into a classification output that we can compare with the labels. This is relatively easy if we used three clusters in the algorithm, since there should

Copyrighted material

hopefully be a one-to-one correspondence between them, but it might turn out that using more clusters gets better results, although this will make the analysis more difficult. You can do this by hand if there are relatively small numbers of datapoints, or you could use a supervised learning algorithm to do it for you, as is discussed next.

To see how the k-means algorithm is used, we can see how it is used on the iris dataset:

```
import kmeansnet
net = kmeansnet.kmeans(3,train)
net.kmeanstrain(train)
cluster = net.kmeansfwd(test)
print cluster
print iris[3::4,4]
```

The output that is produced by this in an example run is (where the top line is the output of the algorithm and the bottom line is the classes from the dataset):

```
[ 0. 0. 0. 0. 0. 1. 1. 1. 1. 2. 1. 2. 2. 2. 0. 1. 2. 1. 0.
  1. 2. 2. 2. 1. 1. 2. 0. 0. 1. 0. 0. 0. 0. 2. 0. 2. 1.]
[ 1. 1. 1. 1. 1. 2. 2. 2. 1. 0. 2. 0. 0. 0. 1. 1. 0. 2. 2.
  2. 0. 0. 0. 2. 2. 0. 1. 2. 1. 1. 1. 1. 1. 0. 1. 0. 2.]
```

and then we can see that cluster 0 corresponds to label 1 and cluster 1 to label 2, in which case the algorithm gets 1 of cluster 0 wrong, 2 of cluster 1, and none of cluster 2.

9.1.6 Using Competitive Learning for Clustering

Deciding which cluster any datapoint belongs to is now an easy task: we present the datapoint to the trained algorithm and it will tell us. If we don't have any target data, then the problem is finished. However, for many problems we might want to interpret the best-matching cluster as a class label (alternatively, a set of cluster centres could all correspond to one class). This is fine, since if we have target data we can match the output classes to the targets, provided that we are a bit careful: there is no reason why the order of the nodes in the network should match the order in the data, since the algorithm knows nothing about that order. For that reason, when assigning class labels to the outputs, you need to check which numbers match up carefully, or the results will look a lot worse than they actually are.

There is an alternative solution to this problem of assigning labels, and it is one that we have seen before. In Chapter 4 we considered using the k-means

Copyrighted material

network in order to train the positions of the RBF nodes. It is now possible to see how this works. The k-means part positions the RBFs in the input space, so that they represent the input data well. A Perceptron is then used on top of this in order to provide the match to the outputs in the supervised learning part of the network. Since this is now supervised learning, it ensures that the output categories match the target data classes. It also means that you can use lots of clusters in the k-means network without having to work out which datapoints belong to which cluster, since the Perceptron will do this for you.

We are now going to look at another major algorithm in competitive learning, the Self-Organising Feature Map. As motivation for it, we are going to consider a sample problem for competitive learning, which is a problem in data compression called vector quantisation.

9.2 Vector Quantisation

We've already discussed using competitive learning for removing noise. There is a related application, data compression, which is used both for storing data and for the transmission of speech and image data. The reason that the applications are related is that both replace the current input by the cluster centre that it belongs to. For noise reduction we do this to replace the noisy input with a cleaner one, while for data compression we do it to reduce the number of datapoints that we send.

Both of these things can be understood by considering them as examples of data communication. Suppose that I want to send data to you, but that I have to pay for each data bit I transmit, so I want to keep the amount of data that I send to a minimum. I notice that there are lots of repeated datapoints, so I decide to encode my data before I send it, so that instead of sending the entire set, we agree a codebook of prototype vectors together. Now, instead of transmitting the actual data, I can transmit the index of that datapoint in the codebook, which is shorter. All you have to do is take the indices I send you and look them up, and you have the data. We can actually make the code even more efficient by using shorter indices for the datapoints that are more common. This is an important problem in information theory, and every kind of sound and image compression algorithm has a different method of solving it.

There is one problem with the scenario so far, which is that the codebook won't contain every possible datapoint. What happens when I want to send a datapoint and it isn't in the codebook? In that case we need to accept that our data will not look exactly the same, and I send you the index of the prototype vector that is closest to it (this is known as vector quantisation, and is the way that lossy compression works).

Copyrighted material

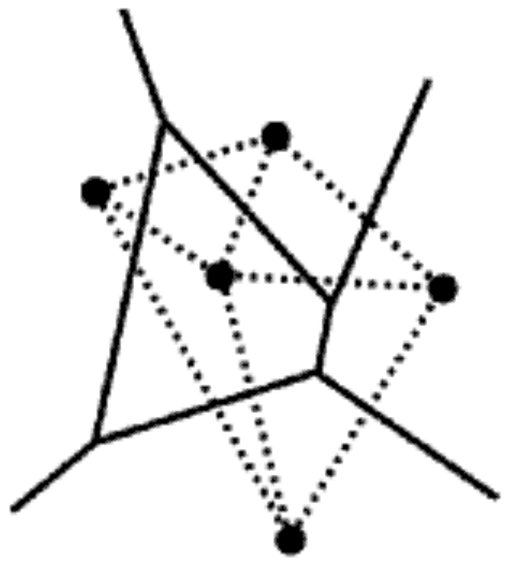


FIGURE 9.5: The Voronoi tesselation of space that performs vector quantisation. Any datapoint is represented by the dot within its cell, which is the prototype vector.

Figure 9.5 shows an interpretation of prototype vectors in two dimensions. The dots at the centre of each cell are the prototype vectors, and any datapoint that lies within a cell is represented by the dot. The name for each cell is the Voronoi set of a particular prototype. Together, they produce the Voronoi tesselation of the space. If you connect together every pair of points that share an edge, as is shown by the dotted lines, then you get the Delaunay triangulation, which is the optimal way to organise the space to perform function approximation.

The question is how to choose the prototype vectors, and this is where competitive learning comes in. We need to choose prototype vectors that are as close as possible to all of the possible inputs that we might see. This application is called learning vector quantisation because we are learning an efficient vector quantisation. The k-means algorithm can be used to solve the problem if we know how large we want our codebook to be. However, another algorithm turns out to be more useful, the Self-Organising Feature Map, which is described next.

9.3 The Self-Organising Feature Map

By far the most commonly used competitive learning algorithm is the Self-Organising Feature Map (often abbreviated to SOM), which was proposed by Teuvo Kohonen in 1988. Kohonen was considering the question of how sensory signals get mapped into the cerebral cortex of the brain with an order. For example, in the auditory cortex, which deals with the sounds that we hear, neurons that are excited (i.e., that are caused to fire) by similar sounds are positioned closely together, whereas two neurons that are excited by very different sounds will be far apart.

There are two novel departures in this for us: firstly, the relative locations

Copyrighted material

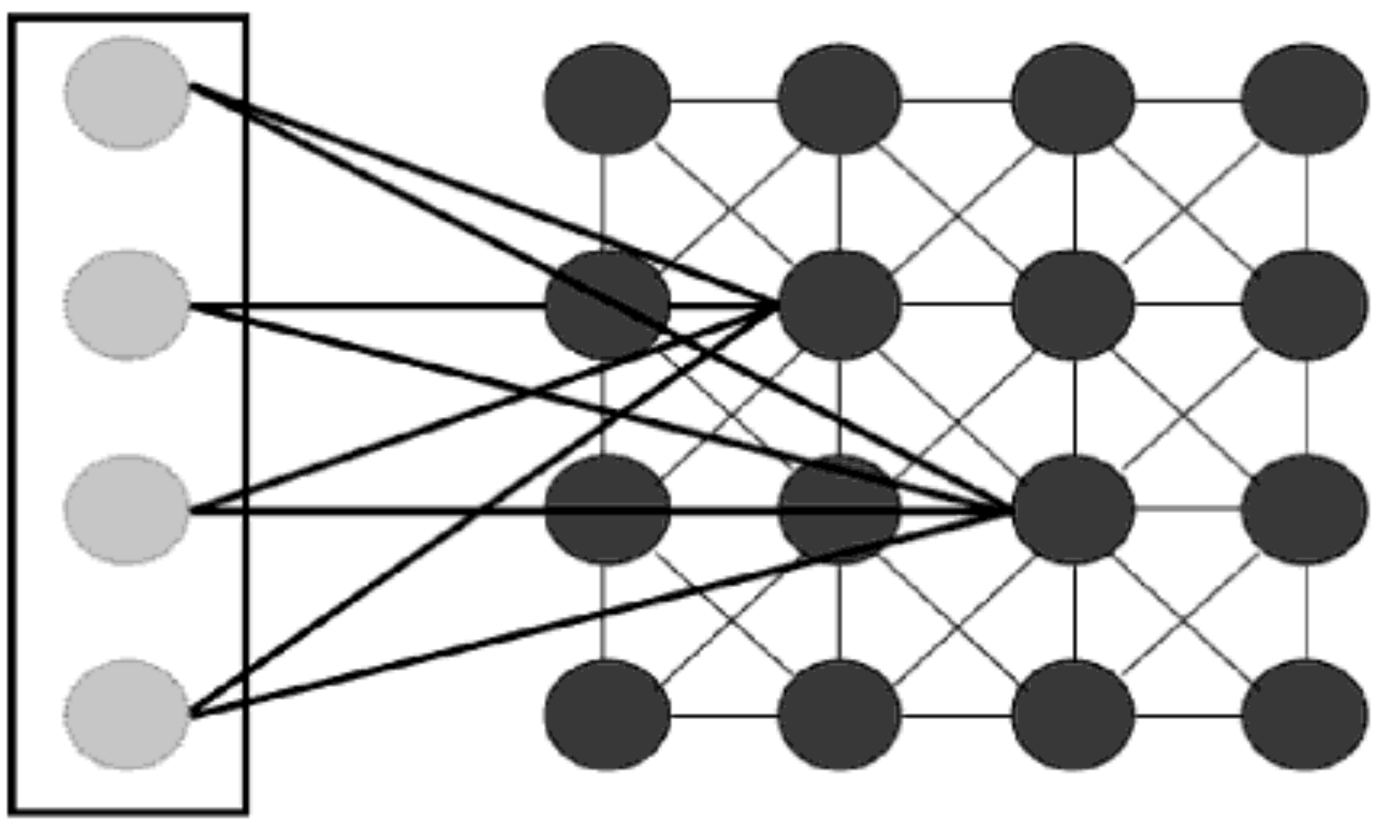


FIGURE 9.6: The Self-Organising Map network. As usual, input nodes (on the left) do no computation, and the weights are modified to change the activations of the neurons (weights are only shown to two nodes for clarity). However, the nodes within the SOM affect each other in that the winning node also changes the weights of neurons that are close to it. Connections are shown in the figure to the eight closest nodes, but this is a parameter of the network.

of the neurons in the network matters (this property is known as feature mapping—nearby neurons correspond to similar input patterns), and secondly, the neurons are arranged in a grid with connections between the neurons, rather than in layers with connections only between the different layers. In the auditory cortex there appears to be sheets of neurons arranged in 2D, and that is the typical arrangement of neurons for the SOM: a grid of neurons arranged in 2D, as can be seen in Figure 9.6. A 1D line of neurons is also sometimes used. In mathematical terms, the SOM demonstrates relative ordering preservation, which is sometimes known as topology preservation. The relative ordering of the inputs should be preserved by the ordering in the neurons, so that neurons that are close together represent inputs that are close together, while neurons that are far apart represent inputs that are far apart.

This topology preservation is not necessarily possible, because the SOM typically uses a 1D or 2D array of neurons, and most of our input spaces are of much higher dimensionality than that. This means that the ordering cannot be preserved. We have seen this in Figure 1.2, where one view of some wind turbines made it look like they are on top of each other, when they clearly are not, because we used a two-dimensional representation of three-dimensional reality. You've probably seen the same thing in other photos, where trees appear to be growing out of somebody's head. A different way to see the same thing is given in Figure 9.7, where mismatches between the topology of the input space and map lead to changes in the relative ordering. The best that can be said is that SOM is perfectly topology-preserving, which means that if the dimensionality of the input and the map correspond, then

Copyrighted material

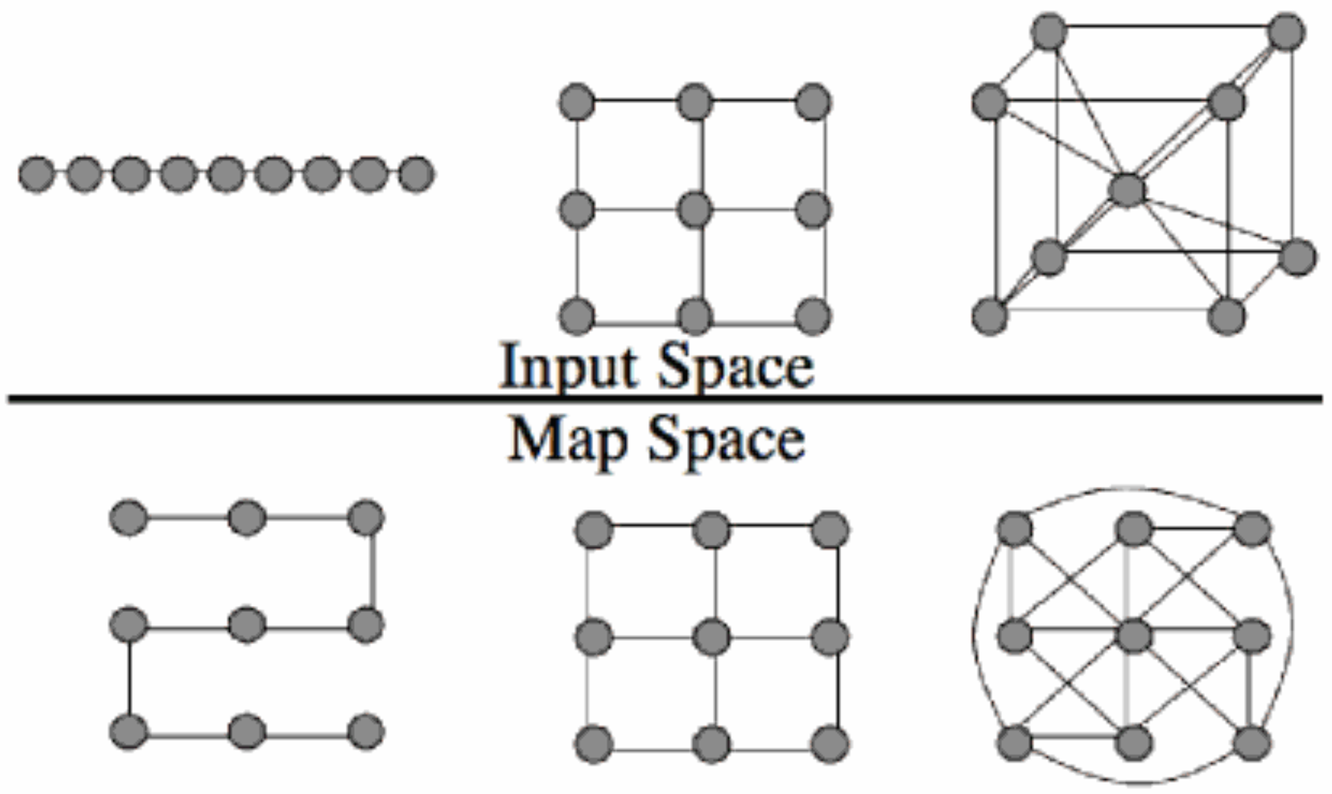


FIGURE 9.7: When inputs in 1D (a straight line), a 2D grid, and a 3D cube are represented by a 2D grid of neurons, the relative ordering is not perfectly preserved. The 1D line is bent, which means that points that used to be a long way apart (such as the first and sixth on the line) are now close together, while the cube becomes very complicated. The lines in the bottom part of the figure represent connections that are meant to be close.

the topology of the input space will be preserved. We are going to look at other methods of performing dimensionality reduction in Chapter 10.

The question, then, is how we can implement feature mapping in an unsupervised learning algorithm. The first thing to recognise is that we need some interaction between the neurons in the network, so that when one neuron fires, it affects what happens to those around it. We have seen something like this before, for example, between different layers of the MLP, but now we are thinking about neurons that are within a layer. These are known as lateral connections (i.e., within the layer of the network). How should this interaction work? We are trying to introduce feature mapping, so neurons that are close together in the map should represent similar features. This means that the winning neuron should pull other neurons that are close to it in the network closer to itself in weight space, which means that we need positive connections. Likewise, neurons that are further away should represent different features, and so should be a long way off in weight space, so the winning neuron 'repels' them, by using negative connections to push them away. Neurons that are very far away in the network should already represent different features, so we just ignore them. This is known as the 'Mexican Hat' form of lateral connections, for reasons that should be clear from the picture in Figure 9.8. We can then just use ordinary competitive learning, just like we did for the k-means network in Section 9.1.2. The Self-Organising Map does pretty much exactly this.

Copyrighted material

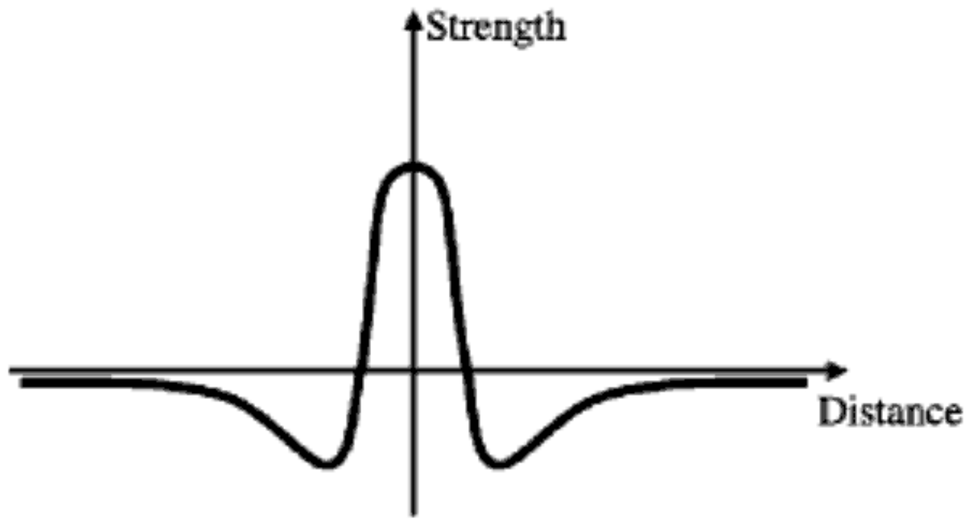


FIGURE 9.8: Graph of the strength of lateral connections for a feature mapping algorithm known as the 'Mexican Hat.'

9.3.1 The SOM Algorithm

Using the full Mexican hat lateral interactions between neurons is fine, but it isn't essential. In Kohonen's SOM algorithm, the weight update rule is modified instead, so that information about neighbouring neurons is included in the learning rule, which makes the algorithm simpler. The algorithm is a competitive learning algorithm, so that one neuron is chosen as the winner, but when its weights are updated, so are those of its neighbours, although to a lesser extent. Neurons that are not within the neighbourhood are ignored, not repelled.

We will now look at the SOM algorithm before examining some of the details further.

The Self-Organising Feature Map Algorithm

- **Initialisation**
  - choose a size (number of neurons) and number of dimensions d for the map
  - Either:
    - choose random values for the weight vectors so that they are all different OR
    - set the weight values to increase in the direction of the first d principal components of the dataset
- **Learning**
  - repeat:
    - for each datapoint:
      - select the best-matching neuron n_b using the minimum Euclidean distance between the weights and the input,

$$n_b = \min_j \|\mathbf{x} - \mathbf{w}_j^T\|. \tag{9.8}$$

Copyrighted material

* update the weight vector of the best-matching node using:

$$\mathbf{w}_j^T \leftarrow \mathbf{w}_j^T + \eta(t)(\mathbf{x} - \mathbf{w}_j^T), \tag{9.9}$$

where $\eta(t)$ is the learning rate.

* update the weight vector of all other neurons using:

$$\mathbf{w}_j^T \leftarrow \mathbf{w}_j^T + \eta_n(t)h(n_b, t)(\mathbf{x} - \mathbf{w}_j^T), \tag{9.10}$$

where $\eta_n(t)$ is the learning rate for neighbourhood nodes, and $h(n_b, t)$ is the neighbourhood function, which decides whether each neuron should be included in the neighbourhood of the winning neuron (so $h = 1$ for neighbours and $h = 0$ for non-neighbours)

* reduce the learning rates and adjust the neighbourhood function, typically by $\eta(t+1) = \alpha\eta(t)^{k/k_{\max}}$ where $0 \leq \alpha \leq 1$ decides how fast the size decreases, k is the number of iterations the algorithm has been running for, and $k_{\max}$ is when you want the learning to stop. The same equation is used for both learning rates (η, η_n) and the neighbourhood function $h(n_b, t)$.

– until the map stops changing or some maximum number of iterations is exceeded

• **Usage**

– for each test point:

* select the best-matching neuron n_b using the minimum Euclidean distance between the weights and the input:

$$n_b = \min_j \|\mathbf{x} - \mathbf{w}_j^T\| \tag{9.11}$$

9.3.2 Neighbourhood Connections

The size of the neighbourhood is thus another parameter that we need to control. How large should the neighbourhood of a neuron be? If we start our network off with random weights, as we did for the MLP, then at the beginning of learning, the network is pretty well unordered (as the weights are random, two nodes that are very close in weight space could be on opposite sides of the map, and vice versa) and so it makes sense that the neighbourhoods should be large, so that we get the rough ordering of the network correct. However, once the network has been learning for a while, the rough ordering has already been created, and the algorithm starts to fine-tune the individual local regions of the network. At this stage, the neighbourhoods should be small, as is shown in

Copyrighted material

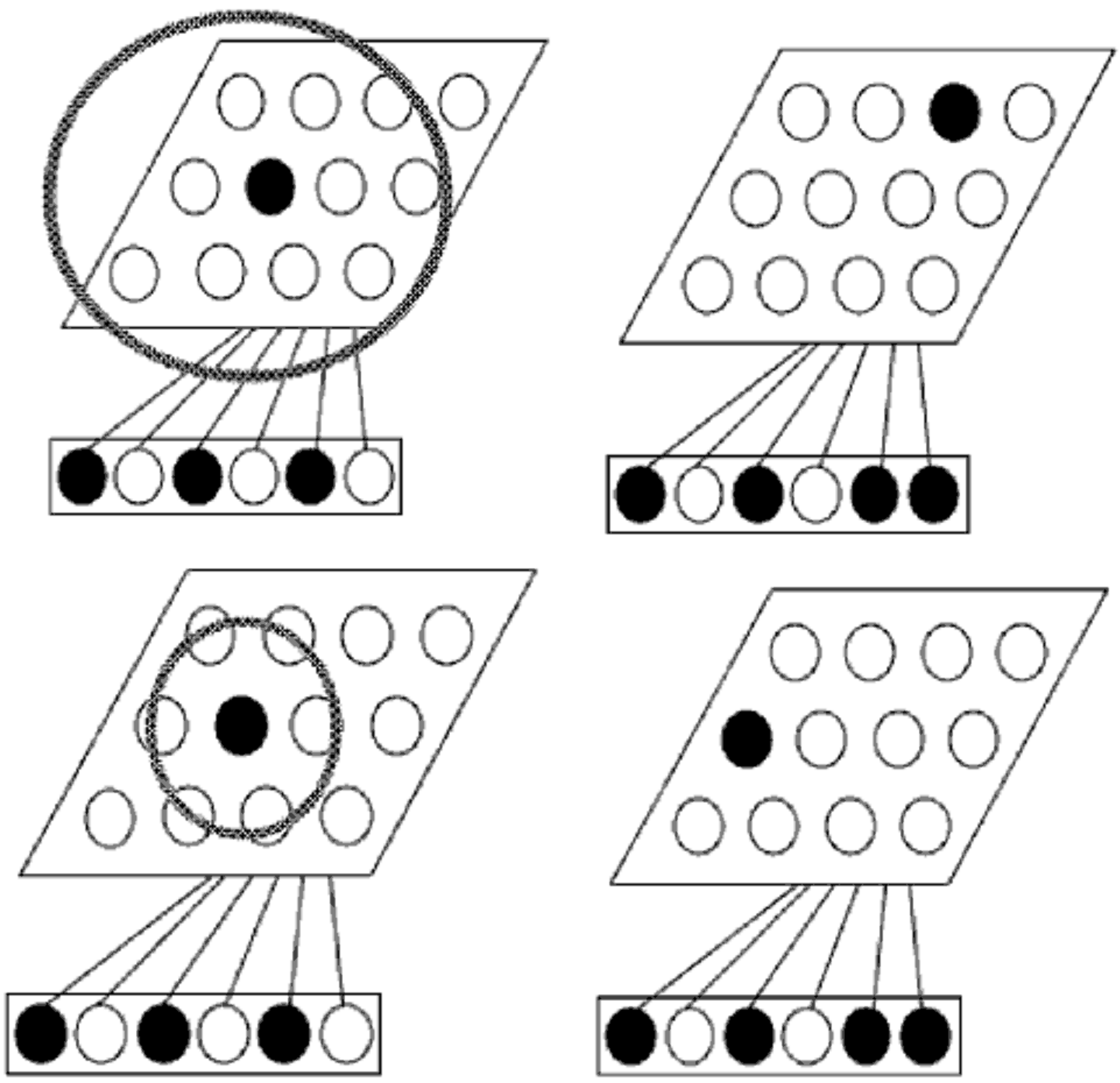


FIGURE 9.9: *Top:* Initially, similar input vectors excite neurons that are far apart, so that the neighbourhood (shown as a circle) needs to be large. *Bottom:* Later on during training the neighbourhood can be smaller, because similar input vectors excite neurons that are close together.

Figure 9.9. It therefore makes sense to reduce the size of the neighbourhood as the network adapts. These two phases of learning are also known as ordering and convergence. Typically, we reduce the neighbourhood size by a small amount at each iteration of the algorithm. We control the learning rate η in exactly the same way, so that it starts off large and decreases over time, as is shown in the algorithm below.

The fact that the size of the neighbourhood changes as the algorithm runs has consequences for an implementation. There is no point using actual connections between nodes, since the number of these will change as the algorithm runs. We therefore set up a matrix that measures the distances between nodes in the network and choose the nodes in the neighbourhood of a particular node as those within a neighbourhood radius that shrinks as the algorithm runs.

```
# Set up the map distance matrix
mapDist = zeros((self.x*self.y,self.x*self.y))
for i in range(self.x*self.y):
    for j in range(i+1,self.x*self.y):
```

Copyrighted material

Hidden page

9.3.3 Self-Organisation

You might be wondering what the self-organisation in the name of the SOM is. A particularly interesting aspect of feature mapping is that we get a global ordering of the neurons in the network, despite the fact that the interactions are all local, since neurons that are very far apart do not interact with each other. We thus get a global ordering of the space using only a set of local interactions, which is amazing. This is known as self-organisation, and it appears everywhere. It is part of the growing science of complexity. To see how common self-organisation is, consider a flock of birds flying in formation. The birds cannot possibly know exactly where each other are, so how do they keep in formation? In fact, simulations have shown that if each bird just tries to stay diagonally behind the bird to its right, and fly at the same speed, then they form perfect flocks, no matter how they start off and what objects are placed in their way. So the global ordering of the whole flock can arise from the local interactions of each bird looking to the one on its right (or left).

9.3.4 Network Dimensionality and Boundary Conditions

We typically think about applying the SOM algorithm to a 2D rectangular array of neurons (as shown in Figure 9.6), but there is nothing in the algorithm to force this. There are cases where a line of neurons (1D) works better, or where three dimensions are needed. It depends on the dimensionality of the inputs (actually on the intrinsic dimensionality, the number of dimensions that you actually need to represent the data), not the number that it is embedded in. As an example, consider a set of inputs spread through the room you are in, but all on the plane that connects the bottom of the wall to your left with the top of the wall to your right. These points have intrinsic dimensionality two since they are all on the plane, but they are embedded in your three-dimensional room. Noise and other inaccuracies in data often leads to it being represented in more dimensions than are actually required, and so finding the intrinsic dimensionality can help to reduce the noise.

We also need to consider the boundaries of the network. In some cases, it makes sense that the edges of the map of neurons is strictly defined — for example, if we are arranging sounds from low pitch to high pitch, then the lowest and highest pitches we can hear are obvious end points. However, it is not always the case that such boundaries are clearly defined. In this case we might want to remove the boundary conditions. We can do this by removing the boundary by tying the ends together. In 1D this means that we turn a line into a circle, while in 2D we turn a rectangle into a torus. To see this, try taking a piece of paper and bend it so that the top and bottom edges line up. You've now got a tube. If you bend the tube round so that the two open ends meet up you have a circle of tube known as a torus. Pictures of these effects are shown in Figure 9.10. In effect, it means that there are no neurons on the edge of the feature map. The choice of the number of dimensions and the

Copyrighted material

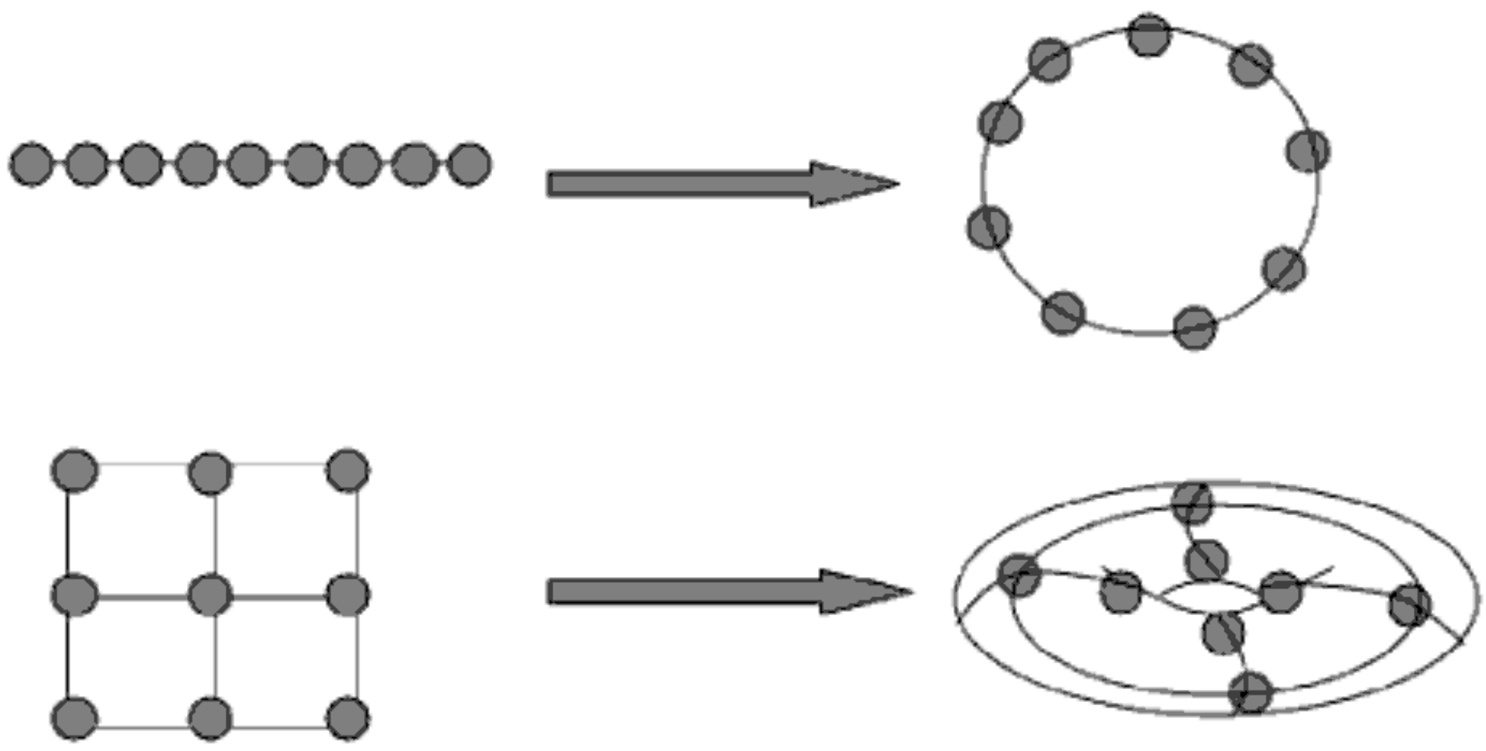


FIGURE 9.10: Using circular boundary conditions in 1D turns a line into a circle, while in 2D it turns a rectangle into a torus.

boundary conditions depends on the problem that we are considering, but it is usually the case that the torus works better than the rectangle, although it is not always clear why.

The one cost that this has is that the map distances get more complicated to calculate, since we now need to calculate the distances allowing for the wrap around. This can be done using modulo arithmetic, but it is easier to think about taking copies of the map and putting them around the map, so that the original map has copies of itself all around: one above, one below, to the right and left, and also diagonally above and below, as is shown in Figure 9.11. Now we keep one of the points in the original map, and the distance to the second node is the smallest of the distances between the first node and the copies of the second node in the different maps (including the original). By treating the distances in x and y separately, the number of distances that has to be computed can be reduced.

As with the competitive learning algorithm that we considered earlier, the size of the SOM is defined before we start learning. The size of the network (that is, the number of neurons that we put into it) decides how fine-grained the learning is. If there are very few neurons, then the best that the network can do is to find gross generalisations that link the data. However, if there are very large numbers of neurons, then the network can represent every input without ever needing to generalise at all. This is yet another example of overfitting. Clearly, then, choosing the correct size of network is important. The common approach is to test out several different sizes of network, such as 5×5 and 10×10 and see how well the network learns.

9.3.5 Examples of Using the SOM

As a first example of using the SOM, and one that shows the topological ordering of the network, consider training the network on a set of two-

Copyrighted material

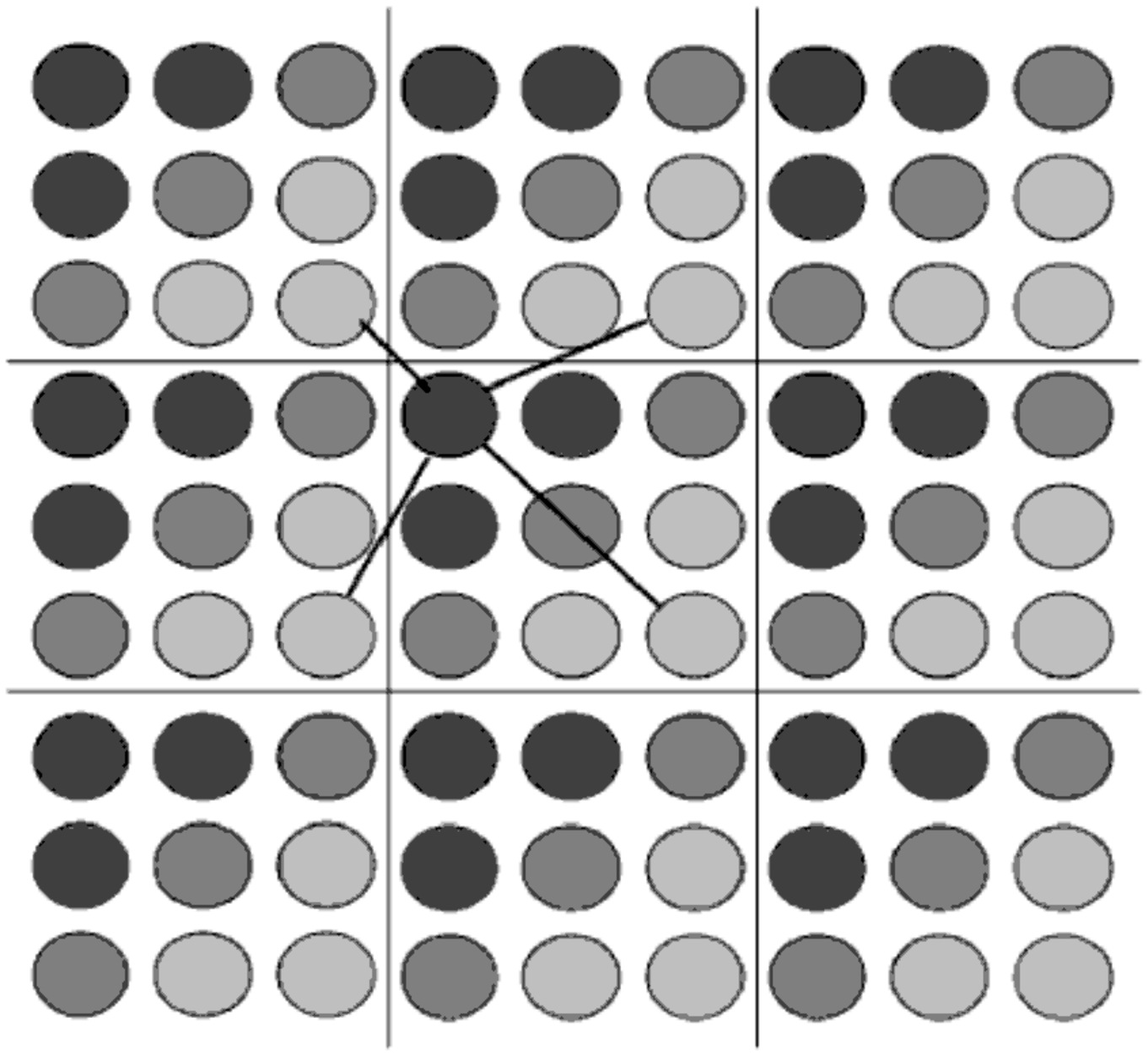


FIGURE 9.11: One way to compute distances between points without any boundary on the map is to imagine copies of the entire map being placed around the original, and picking the shortest of the distances between a node and any of the copies of the other node.

dimensional data drawn at random from a uniform distribution in $[-1, 1]$ in both directions. If the network weights are started off randomly, then initially the network is completely disordered (as shown in the top-left picture in Figure 9.12), but after 10 iterations of training the network is ordered so that neighbouring nodes map to data that is close together (bottom-left). Using PCA to initialise the map is not especially useful for this dataset, but it does speed things up: only five iterations through the dataset produce the output shown on the bottom-right of the figure, where it started from the version on the top-right.

For two examples of using the SOM on non-random data, where we can expect to see some actual learning, we will first look at the iris data that we used with the k-means algorithm earlier in this chapter. Figure 9.13 shows a plot of which node of a 5×5 Self-Organising Map was the best match on a set of test data after training for 100 iterations. The three different classes are shown as different shapes (squares, plus triangles pointing up and down), but remember that the network did not receive any information about these target classes. It can be seen that the examples in each of the three classes form different clusters in the map. Looking at the figure, you might be wondering if it is possible to use the plot to identify the different classes by assuming that they are separated in the map. This has been investigated—often by using methods similar to those of Linear Discriminant Analysis that are described

Copyrighted material

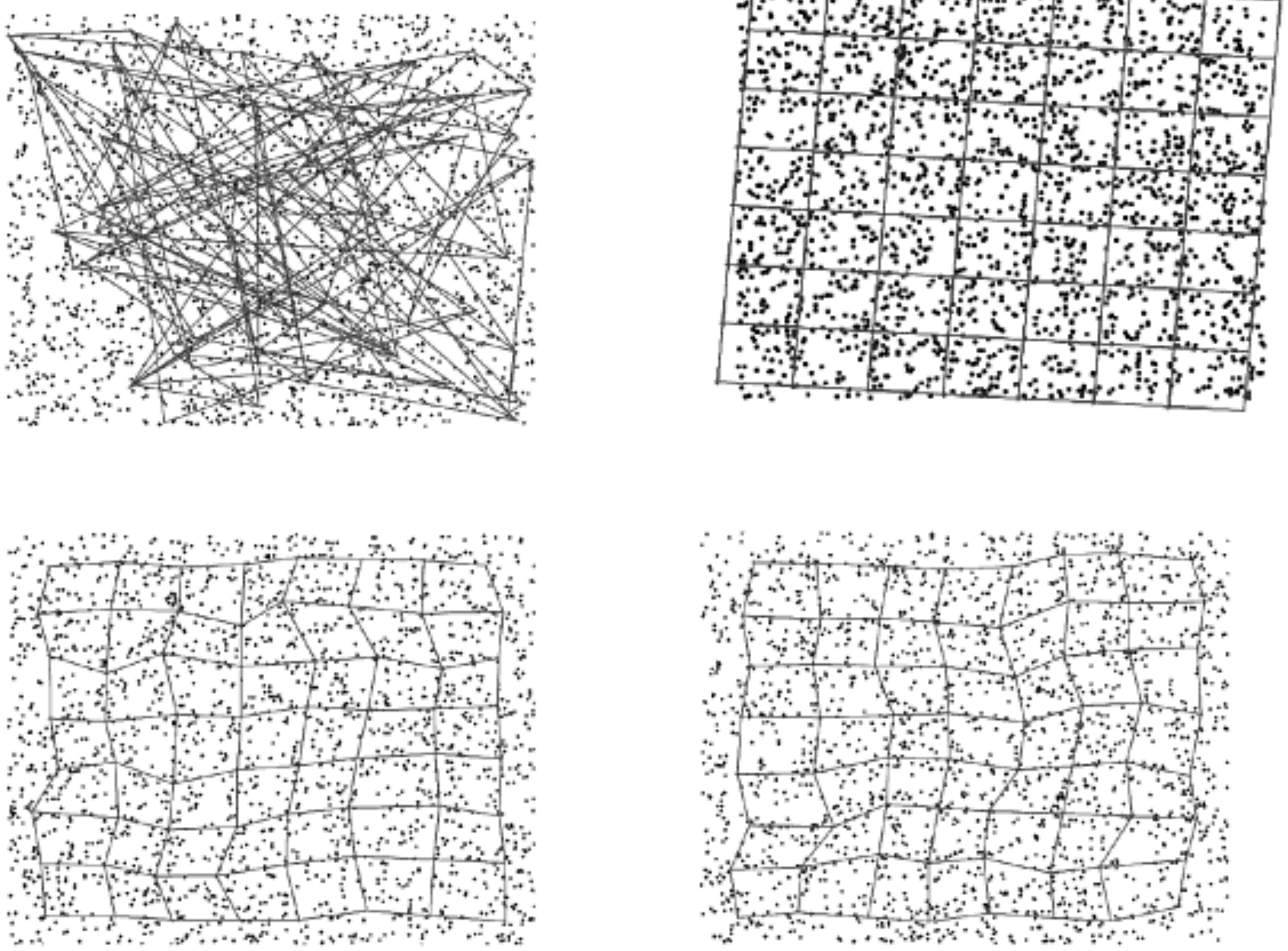


FIGURE 9.12: Training the SOM on a set of uniformly randomly sampled two-dimensional data in the range $[-1, 1]$ in both dimensions. *Top:* Initialisation of the map using *left:* random weights and *right:* PCA (the randomness in the data means that the directions of variation are not necessarily along the obvious directions). *Bottom:* The output after just 10 iterations of training on the left, and 5 on the right, both with typical parameter values.

Copyrighted material

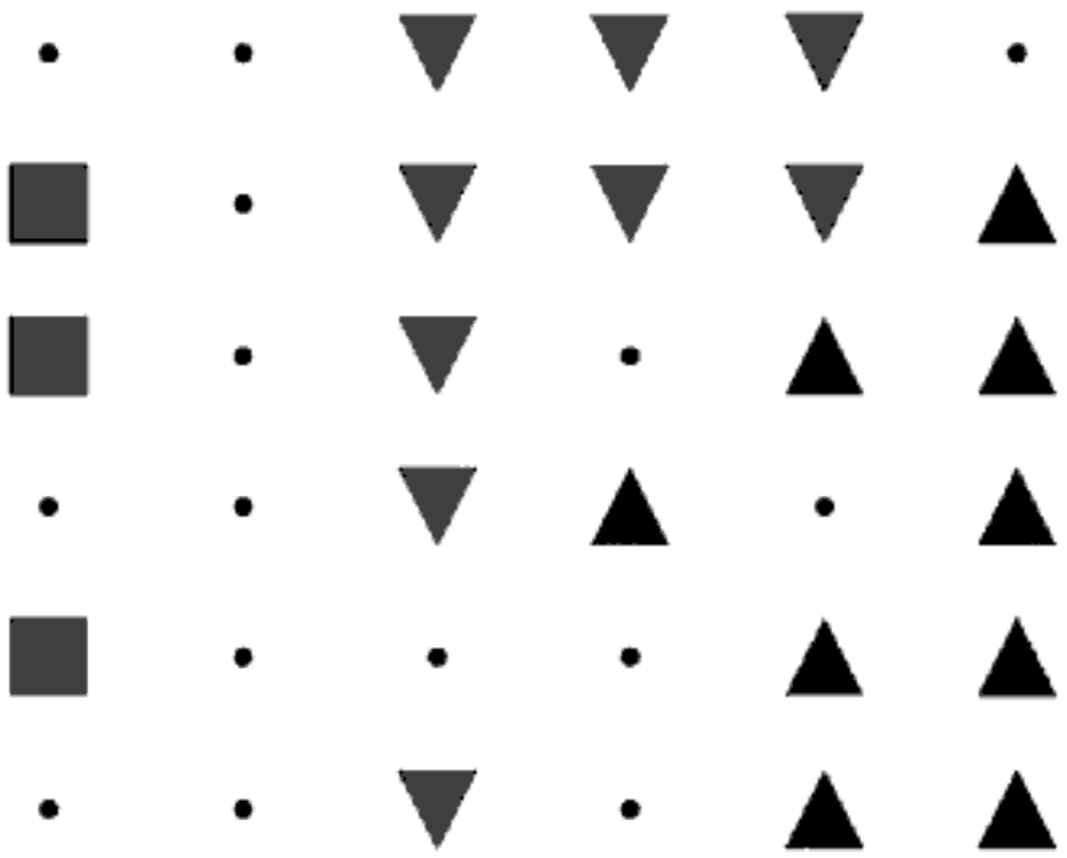


FIGURE 9.13: Plot showing which nodes are the best match according to class, with the three shapes corresponding to three different classes in the iris dataset. The small dots represent nodes that did not fire.

in Section 10.1—with some success, and a reference is provided at the end of the chapter.

A more difficult problem is shown in Figure 9.14. The data are the `ecoli` dataset from the UCI Machine Learning repository, and the class is the localisation site of the protein, based on a set of protein measurements. The results with this dataset when testing are not as clearly impressive (but note that the MLP gets about 50% accuracy on this dataset, and that has the target data, which the SOM doesn't). However, the clusters can still be seen to some extent, and they are very clear in the training data. Note that the boundary conditions can make things a little more complicated, since the cluster does not necessarily respect the edges of the map.

Further Reading

There is a book by Kohonen, the inventor of the SOM, that provides a very good overview of the area:

- T. Kohonen. *Self-Organisation and Associative Memory.* Springer, Berlin, Germany, 3rd edition, 1989.

The two on-line self-organising networks that were mentioned in the chapter were:

Copyrighted material

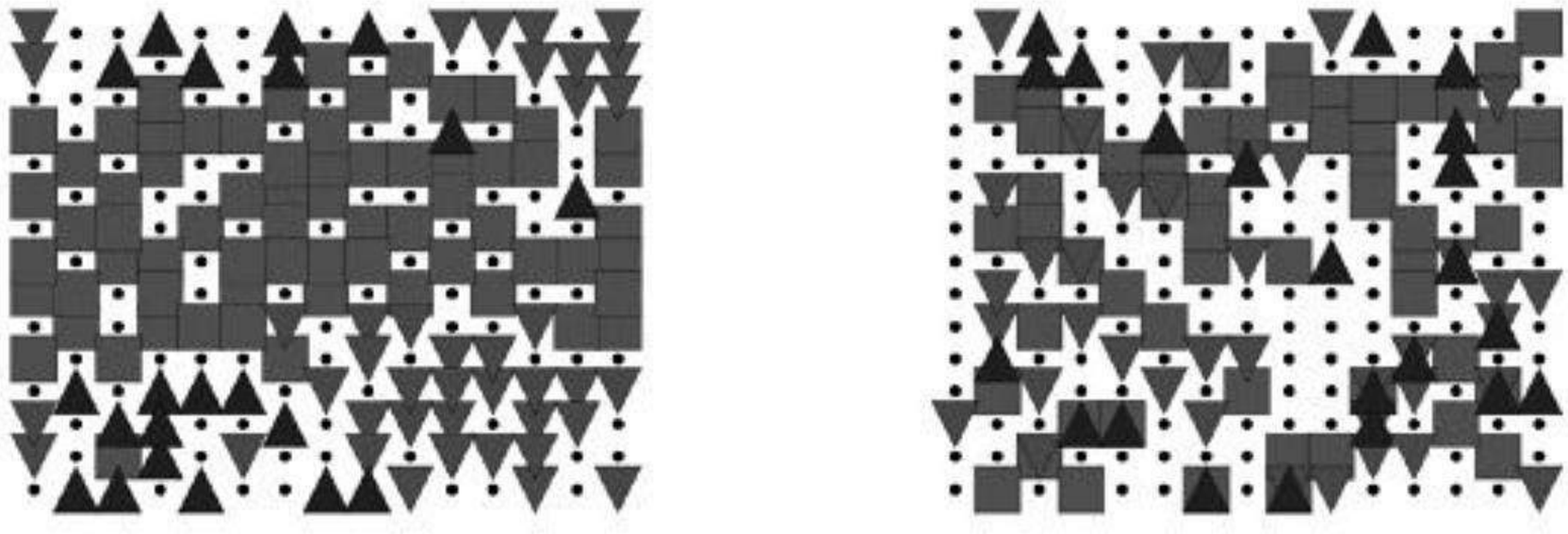


FIGURE 9.14: Plots showing which nodes are the best match according to class, with the three shapes corresponding to three different classes in the e-coli dataset, tested on *left:* the training set and *right:* a separate test set. The small dots represent nodes that did not fire.

- B. Fritzke. A growing neural gas network learns topologies. In Gerald Tesauro, David S. Touretzky, and Todd K. Leen, editors, *Advances in Neural Information Processing Systems*, volume 7, MIT Press, Cambridge, MA, USA, 1995.

- S. Marsland, J.S. Shapiro, and U. Nehmzow. A self-organising network that grows when required. *Neural Networks*, 15(8-9):1041–1058, 2002.

A possible reference on processing the data in the map in order to identify clusters is:

- S. Wu and T.W.S. Chow. Self-organizing-map based clustering using a local clustering validity index. *Neural Processing Letters*, 17(3):253–271, 2003.

Books that cover the area include:

- Section 10.14 of R.O. Duda, P.E. Hart, and D.G. Stork. *Pattern Classification.* Wiley-Interscience, New York, USA, 2nd edition, 2001.

- Chapter 9 of S. Haykin. *Neural Networks: A Comprehensive Foundation.* Prentice-Hall, New Jersey, USA, 2nd edition, 1999.

- Section 9.3 of B.D. Ripley. *Pattern Recognition and Neural Networks.* Cambridge University Press, Cambridge, UK, 1996.

Copyrighted material

Practice Questions

Problem 9.1 What is the purpose of the neighbourhood function in the SOM? How does it change the learning?

Problem 9.2 A simplistic intruder detection system for a computer network consists of an attempt to categorise users according to (i) the time of day they log in, (ii) the length of time they log in for, (iii) the types of programs they run while logged in, (iv) the number of programs they run while logged in. Suggest how you would train a SOM and the naïve Bayes' classifier to perform the categorisation. What preprocessing of the data would you do, how much data would you need, and how large would you make the SOM? Do you think that such a system would work for intruder detection?

Problem 9.3 The Music Genome Project (`http://www.pandora.com`) does not work by using a SOM. But it could. Describe how you would implement it.

Problem 9.4 A bank wants to detect fraudulent credit card transactions. They have data for lots and lots of transactions (each transaction is an amount of money, a shop, and the time and date) and some information about when credit cards were stolen, and the transactions that were performed on the stolen card. Describe how you could use a competitive learning method to cluster people's transactions together to identify patterns, so that stolen cards can be detected as changes in pattern. How well do you think this would work? There is much more data of transactions when cards are not stolen, compared to stolen transactions. How does this affect the learning, and what can you do about it?

Problem 9.5 It is possible to use any competitive learning method to position the basis functions of a Radial Basis Function network. The example code used k-means. Modify it to use the SOM instead and compare the results on the `wine` and `yeast` datasets.

Problem 9.6 For the `wine` dataset, experiment with different sizes of map, and boundary conditions. How much difference does it make? Can you use the principal components in order to set the size automatically?

Copyrighted material

Chapter 10

Dimensionality Reduction

In Chapter 9 we saw that the Self-Organising Map (SOM) reduced the number of dimensions in the data to the two dimensions of the map. The choice of two dimensions was imposed arbitrarily by the fact that the neurons were arranged in a two-dimensional grid, and we saw in Section 9.3 that this can cause problems, since projecting the data into two dimensions usually changes the relative ordering of the datapoints. However, there are many reasons why this dimensionality reduction is useful. The most obvious justification is that it reduces the curse of dimensionality, and also the computational cost of many of the algorithms, since the dimensionality is usually an explicit factor. However, it can also remove noise, significantly improve the results of the learning algorithm, make the dataset easier to work with, and make the results easier to understand. In extreme cases such as the Self-Organising Map, where the number of dimensions becomes three or less, we can also plot the data, which makes it much easier to understand and interpret.

With this many good things to say about dimensionality reduction, clearly it is something that we need to understand. The importance of the field for machine learning and other forms of data analysis can be seen from the fact that in the year 2000 there were three articles related to dimensionality reduction published together in the prestigious journal *Science*. At the end of the chapter we are going to see two of the algorithms that were described in those papers: Locally Linear Embedding and Isomap.

There are three different ways to do dimensionality reduction. The first is feature selection, which typically means looking through the features that are available and seeing whether or not they are actually useful, i.e., correlated to the output variables. While many people use neural networks precisely because they don't want to 'get their hands dirty' and look at the data themselves, as we have already seen the results will be better if you check for correlations and other simple things before using the neural network or other learning algorithm. The second method is feature derivation, which means deriving new features from the old ones, generally by applying transforms to the dataset that simply change the axes (coordinate system) of the graph by moving and rotating them, which can be written simply as a matrix that we apply to the data. The reason why this performs dimensionality reduction is that it enables us to combine features, and to identify which are useful and which are not. The third method is simply to use clustering in order to group

Copyrighted material

x	y
2.00	-1.43
2.37	-2.80
1.00	-3.17
0.63	-1.80

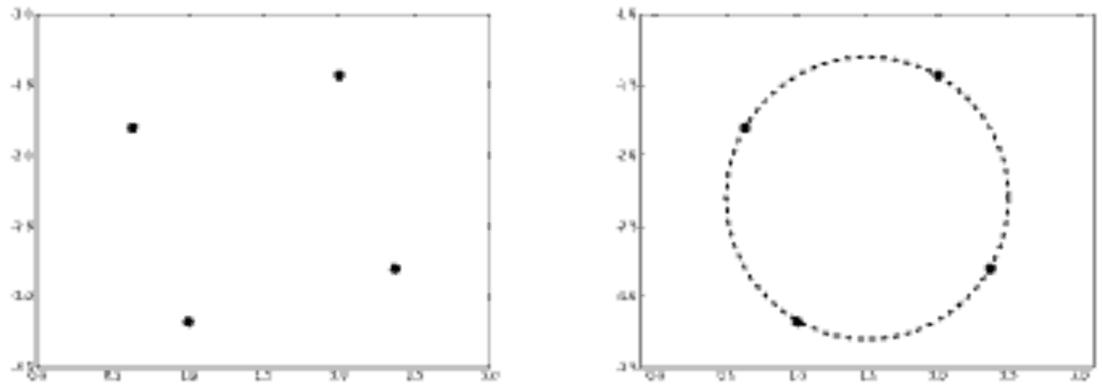


FIGURE 10.1: Three views of the same four points. *Left:* As numbers, where the links are unclear. *Centre:* As four plotted points. *Right:* As four points that lie on a circle.

together similar datapoints, and to see whether this allows fewer features to be used.

To see how choosing the right features can make a problem significantly simpler, have a look at the table on the left of Figure 10.1. It shows the x and y coordinates of 4 points. Looking at the numbers it is hard to see any correlation between the points, and even when they are plotted it simply looks like they might form corners of a rotated rectangle. However, the plot on the right of the figure shows that they are simply a set of four points from a circle, (in fact, the points at $(\pi/6, 4\pi/6, 7\pi/6, 11\pi/6)$) and using this one coordinate, the angle, makes the data a lot easier to understand and analyse.

Once we have worked out how to represent the data, we can suppress dimensions that aren't useful to the algorithm. Even before we get into any form of analysis at all, we can try to perform feature selection, looking at the possible inputs that we have for the problem, and deciding which are useful. Many of the methods that we will see in this chapter merge this idea with transformations of the data, so that combinations of the different inputs, rather than the inputs themselves, are used. However, even before using any of the algorithms identified here, input features can be ignored if they do not seem to be useful.

We have already seen a method of doing feature selection, since it is inherent to the way that the decision tree (Chapter 6) works: at each stage of the algorithm it decides which feature to add next. This is the constructive way to decide on the features: start with none, and then iteratively add more, testing the error at each stage to see whether or not it changed at all when that feature was added. The destructive method is the pruning that was applied to the decision tree, lopping off branches and checking whether the error changed at all.

In general, selecting the features is a search problem. We take the best system so far, and then search over the set of possible next features to add. This can be computationally very expensive, since for d features there are $2^d - 1$ possible sets of features to search over, from any individual feature up to the full set. In general, greedy methods (Section 11.4) are employed, although backtracking can also be employed to check whether the search gets stuck.

Copyrighted material

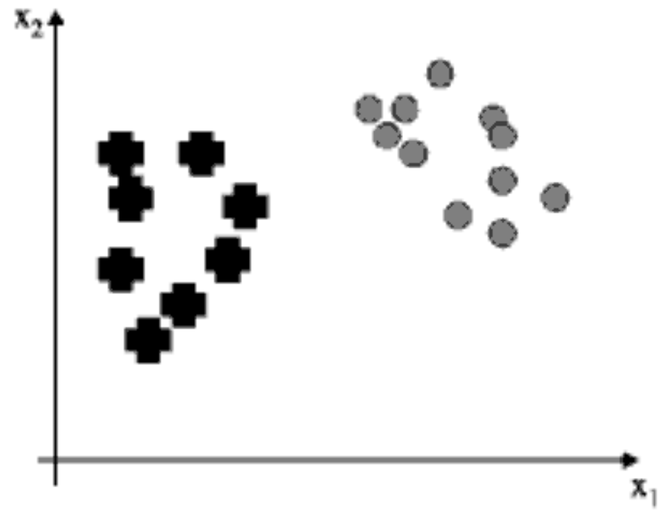


FIGURE 10.2: A set of datapoints in two dimensions, with two classes.

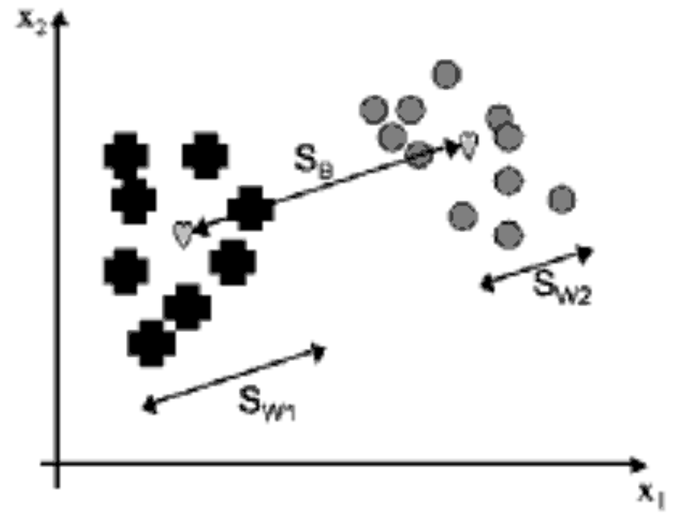


FIGURE 10.3: The mean-ing of the between-class and within-class scatter. The hearts mark the means of the two classes.

Many of the algorithms that we will see in this chapter are unsupervised. The disadvantage of this is that we are not then able to use the knowledge of their classes in order to reduce the problem further. However, we will start off by considering a method of dimensionality reduction that is aimed at supervised learning, Linear Discriminant Analysis. This method is credited to one of the best-known statisticians of the 20th century, R.A. Fisher, and dates from 1936.

10.1 Linear Discriminant Analysis (LDA)

Figure 10.2 shows a simple two-dimensional dataset consisting of two classes. We can compute various statistics about the data, but we will settle for the means of the two classes in the data, $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$, the mean of the entire dataset ($\boldsymbol{\mu}$), and the covariance of each class with itself (see Section 8.2.2 for a description of covariance), which is $\sum_j (\mathbf{x}_j - \boldsymbol{\mu})(\mathbf{x}_j - \boldsymbol{\mu})^T$. The question is what we can do with these pieces of data. The principal insight of LDA is that the covariance matrix can tell us about the scatter within a dataset, which is the amount of spread that there is within the data. The way to find this scatter is to multiply the covariance by the p_c, the probability of the class (that is, the number of datapoints there are in that class divided by the total number). Adding the values of this for all of the classes gives us a measure of the within-class scatter of the dataset:

$$S_W = \sum_{\text{classes } c} \sum_{j \in c} p_c (\mathbf{x}_j - \boldsymbol{\mu}_c)(\mathbf{x}_j - \boldsymbol{\mu}_c)^T. \tag{10.1}$$

If our dataset is easy to separate into classes, then this within-class scatter should be small, so that each class is tightly clustered together. However, to

Copyrighted material

be able to separate the data, we also want the distance *between* the classes to be large. This is known as the **between-classes scatter** and is a significantly simpler computation, simply looking at the difference in the means:

$$S_B = \sum_{\text{classes } c} (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T. \tag{10.2}$$

The meanings of these two measurements is shown in Figure 10.3. The argument about good separation suggests that datasets that are easy to separate into the different classes (i.e., the classes are **discriminable**) should have S_B/S_W as large as possible.

This seems perfectly reasonable, but it hasn't told us anything about dimensionality reduction. However, we can say that the rule about making S_B/S_W as large as possible is something that we want to be true for our data when we reduce the number of dimensions. Figure 10.4 shows two **projections** of the dataset onto a straight line. For the projection on the left it is clear that we can't separate out the two classes, while for the one on the right we can. So we just need to find a way to compute a suitable projection.

Remember from Chapter 2 that any line can be written as a vector $\mathbf{w}$ (which we used as our weight vector in Section 2.3; it is one row of weight matrix $\mathbf{W}$). The projection of the data can be written as $z = \mathbf{w}^T \cdot \mathbf{x}$ for datapoint $\mathbf{x}$. This gives us a scalar that is the distance along the $\mathbf{w}$ vector that we need to go to find the projection of point $\mathbf{x}$. To see this, remember that $\mathbf{w}^T \cdot \mathbf{x}$ is the sum of the vectors multiplied together element-wise, and is equal to the size of $\mathbf{w}$ times the size of $\mathbf{x}$ times the cosine of the angle between them. We can make the size of $\mathbf{w}$ be 1, so that we don't have to worry about it, and all that is then described is the amount of $\mathbf{x}$ that lies along $\mathbf{w}$.

So we can compute the projection of our data along $\mathbf{w}$ for every point, and we will have projected our data onto a straight line, as is shown in the two examples in Figure 10.4. Since the mean can be treated as a datapoint, we can project that as well: $\mu'_c = \mathbf{w}^T \cdot \boldsymbol{\mu}_c$. Now we just need to work out what happens to the within-class and between-class scatters. Replacing $\mathbf{x}_j$ with $\mathbf{w}^T \cdot \mathbf{x}_j$ in Equations (10.1) and (10.2) we can use some linear algebra (principally the fact that $(\mathbf{A}^T\mathbf{B})^T = \mathbf{B}^T\mathbf{A}^{T^T} = \mathbf{B}^T\mathbf{A}$) to get:

$$\sum_{\text{classes } c} \sum_{j \in c} p_c(\mathbf{w}^T \cdot (\mathbf{x}_j - \boldsymbol{\mu}_c))(\mathbf{w}^T \cdot (\mathbf{x}_j - \boldsymbol{\mu}_c))^T = \mathbf{w}^T S_W \mathbf{w} \tag{10.3}$$

$$\sum_{\text{classes } c} \mathbf{w}^T(\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T\mathbf{w} = \mathbf{w}^T S_B \mathbf{w}. \tag{10.4}$$

So our ratio of within-class and between-class scatter looks like $\frac{\mathbf{w}^T S_W \mathbf{w}}{\mathbf{w}^T S_B \mathbf{w}}$. In order to find the maximum value of this with respect to $\mathbf{w}$, we differentiate it and set the derivative equal to 0. This tells us that:

Copyrighted material

Hidden page

Hidden page

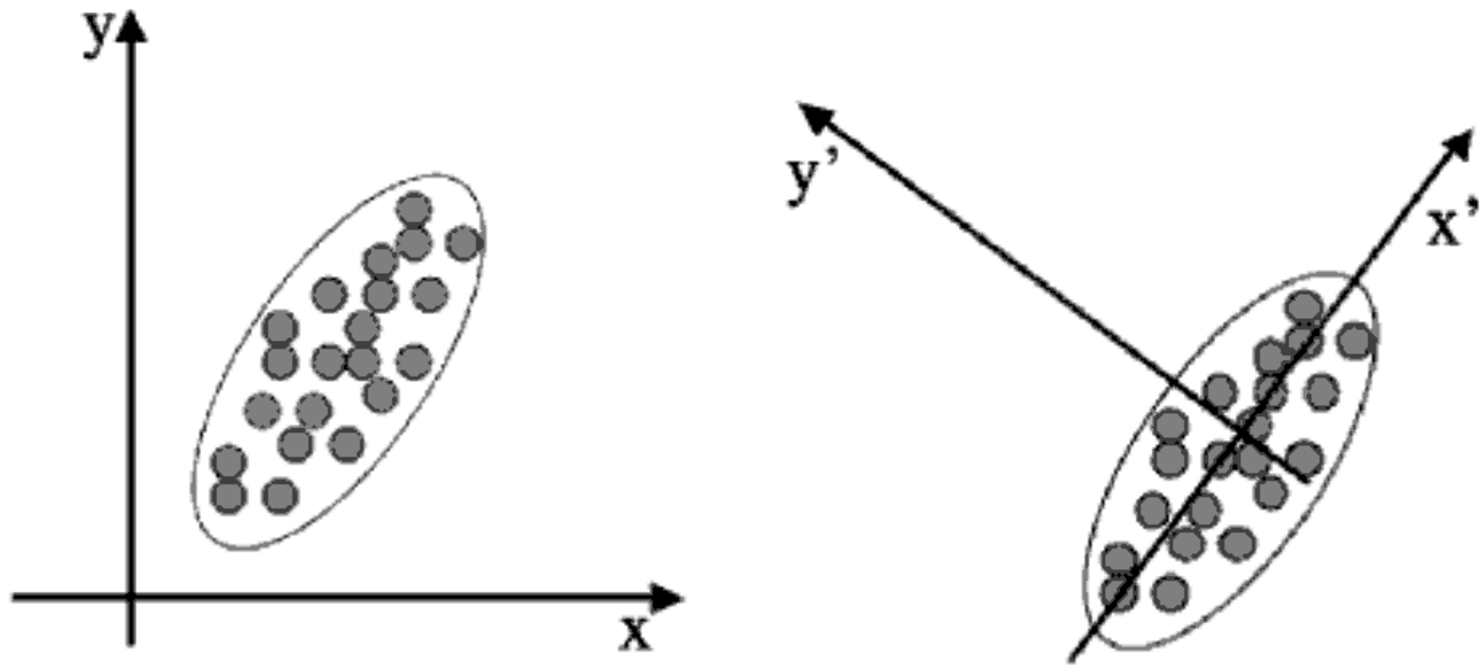


FIGURE 10.6: Two different sets of coordinate axes. The second consists of a rotation and translation of the first and was found using Principal Components Analysis.

does not stop them being used for labelled data, since the learning that takes place in the lower dimensional space can still use the target data, although it does mean that they miss out on any information that is contained in the targets. The idea is that by finding particular sets of coordinate axes, it will become clear that some of the dimensions are not required. This is demonstrated in Figure 10.6, which shows two versions of the same dataset. In the first the data are arranged in an ellipse that runs at 45° to the axes, while in the second the axes have been moved so that the data now runs along the $x-$axis and is centred on the origin. The potential for dimensionality reduction is in the fact that the y dimension does not now demonstrate much variability, and so it might be possible to ignore it and use the x axis values alone without compromising the results of a learning algorithm. In fact, it can make the results better, since we are often removing some of the noise in the data.

The question is how to choose the axes. The first method we are going to look at is Principal Components Analysis (PCA). The idea of a principal component is that it is a direction in the data with the largest variation. The algorithm first centres the data by subtracting off the mean, and then chooses the direction with the largest variation and places an axis in that direction, and then looks at the variation that remains and finds another axis that it is orthogonal to the first and covers as much of the remaining variation as possible. It then iterates this until it has run out of possible axes. The end result is that all the variation is along the axes of the coordinate set, and so the covariance matrix is diagonal—each new variable is uncorrelated with every variable except itself. Some of the axes that are found last have very little variation, and so they can be removed without affecting the variability in the data.

Putting this in more formal terms, we have a data matrix $\mathbf{X}$ and we want to rotate it so that the data lies along the directions of maximum variation. This

Copyrighted material

means that we multiply our data matrix by a rotation matrix (often written as $\mathbf{P}^T$) so that $\mathbf{Y} = \mathbf{P}^T\mathbf{X}$, where $\mathbf{P}$ is chosen so that the covariance matrix of $\mathbf{Y}$ is diagonal, i.e.,

$$\text{cov}(\mathbf{Y}) = \text{cov}(\mathbf{P}^T\mathbf{X}) = \begin{pmatrix} \lambda_1 & 0 & 0 & \dots & 0 \\ 0 & \lambda_2 & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & \lambda_N \end{pmatrix} . \tag{10.7}$$

We can get a different handle on this by using some linear algebra and the definition of covariance to see that:

$$\begin{aligned} \text{cov}(\mathbf{Y}) &= E[\mathbf{Y}\mathbf{Y}^T] && (10.8)\\ &= E[(\mathbf{P}^T\mathbf{X})(\mathbf{P}^T\mathbf{X})^T] && (10.9)\\ &= E[(\mathbf{P}^T\mathbf{X})(\mathbf{X}^T\mathbf{P})] && (10.10)\\ &= \mathbf{P}^T E(\mathbf{X}\mathbf{X}^T)\mathbf{P} && (10.11)\\ &= \mathbf{P}^T \text{cov}(\mathbf{X})\mathbf{P}. && (10.12) \end{aligned}$$

The two extra things that we needed to know were that $(\mathbf{P}^T\mathbf{X})^T = \mathbf{X}^T\mathbf{P}^{T^T} = \mathbf{X}^T\mathbf{P}$ and that $E[\mathbf{P}] = \mathbf{P}$ (and obviously the same for $\mathbf{P}^T$) since it is not a data-dependent matrix. This then tells us that:

$$\mathbf{P}\text{cov}(\mathbf{Y}) = \mathbf{P}\mathbf{P}^T\text{cov}(\mathbf{X})\mathbf{P} = \text{cov}(\mathbf{X})\mathbf{P}, \tag{10.13}$$

where there is one tricky fact, namely that for a rotation matrix $\mathbf{P}^T = \mathbf{P}^{-1}$. This just says that to invert a rotation we rotate in the opposite direction by the same amount that we rotated forwards.

As $\text{cov}(\mathbf{Y})$ is diagonal, if we write $\mathbf{P}$ as a set of column vectors $\mathbf{P} = [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N]$ then:

$$\mathbf{P}\text{cov}(\mathbf{Y}) = [\lambda_1\mathbf{p}_1, \lambda_2\mathbf{p}_2, \dots, \lambda_N\mathbf{p}_N], \tag{10.14}$$

which (by writing the λ variables in a matrix as $\boldsymbol{\lambda} = (\lambda_1, \lambda_2, \dots, \lambda_N)^T$ and $\mathbf{Z} = \text{cov}(\mathbf{X})$) leads to a very interesting equation:

$$\boldsymbol{\lambda}\mathbf{P} = \mathbf{Z}\mathbf{P}. \tag{10.15}$$

At first sight it doesn't look very interesting, but the important thing is to realise that $\boldsymbol{\lambda}$ is a column vector, while $\mathbf{Z}$ is a full matrix. Since $\boldsymbol{\lambda}$ is only a column vector, all it does is rescale the components of $\mathbf{P}$; it cannot rotate it or do anything complicated like that. So this tells us that somehow we have found a matrix $\mathbf{P}$ so that for the directions that $\mathbf{P}$ is written in, the matrix $\mathbf{Z}$ does not twist or rotate those directions, but just rescales them. These directions are special enough that they have a name: they are **eigenvectors**, and the amount that they rescale the axes (the $\boldsymbol{\lambda}$s) by are known as **eigenvalues**.

Copyrighted material

All eigenvectors of a square symmetric matrix $\mathbf{A}$ are unit length and are orthogonal to each other. This tells us that the eigenvectors define a space. If we make a matrix $\mathbf{E}$ that contains the eigenvectors of a matrix $\mathbf{A}$ as columns then this matrix will take any vector and rotate it into what is known as the eigenspace. Since $\mathbf{E}$ is a rotation matrix, $\mathbf{E}^{-1} = \mathbf{E}^T$, so that rotating the resultant vector back out of the eigenspace requires multiplying it by $\mathbf{E}^T$. So what should we do between rotating the vector into the eigenspace, and rotating it back out? The answer is that we can stretch the vectors along the axes. This is done by multiplying the vector by a diagonal matrix that has the eigenvalues along its diagonal, $\mathbf{D}$. So we can decompose any square symmetric matrix $\mathbf{A}$ into the following set of matrices: $\mathbf{A} = \mathbf{EDE}^T$, and this is what we have done to our covariance matrix above. This is called the spectral decomposition.

Before we get on to the algorithm, there is one other useful thing to note. The eigenvalues tell us how much stretching we need to do along their corresponding eigenvector dimensions. The more of this rescaling is needed, the larger the variation along that dimension (since if the data was already spread out equally then the eigenvalue would be close to 1), and so the dimensions with large eigenvalues have lots of variation and are therefore useful dimensions, while for those with small eigenvalues, all the datapoints are very tightly bunched together, and there is not much variation in that direction. This means that we can throw away dimensions where the eigenvalues are very small (usually smaller than some chosen parameter).

It is time to see the algorithm that we need.

The Principal Components Analysis Algorithm

- write N datapoints $\mathbf{x}_i = (\mathbf{x}_{1i}, \mathbf{x}_{2i}, \ldots, \mathbf{x}_{Mi})$ as row vectors
- put these vectors into a matrix $\mathbf{X}$ (which will have size $N \times M$)
- centre the data by subtracting off the mean of each column, putting it into matrix $\mathbf{B}$
- compute the covariance matrix $\mathbf{C} = \frac{1}{N}\mathbf{B}^T\mathbf{B}$
- compute the eigenvalues and eigenvectors of $\mathbf{C}$, so $\mathbf{V}^{-1}\mathbf{CV} = \mathbf{D}$, where $\mathbf{V}$ holds the eigenvectors of $\mathbf{C}$ and $\mathbf{D}$ is the $M \times M$ diagonal eigenvalue matrix
- sort the columns of $\mathbf{D}$ into order of decreasing eigenvalues, and apply the same order to the columns of V
- reject those with eigenvalue less than some η, leaving L dimensions in the data

---

Copyrighted material

Hidden page

Hidden page

look like then? A full answer is complicated, but we can speculate that the first layer is computing some non-linear transformation of the data, while the second (bottleneck) layer is computing the PCA of those non-linear functions. Then the third layer reconstructs the data, which appears again in the fourth layer. So the network is still doing PCA, just on a non-linear version of the inputs. This might be useful, since now we are not assuming that the data are linearly separable. However, to understand it better we will look at it from a different viewpoint, thinking of the actions of the first layer as kernels, which we saw in Section 5.2.

10.2.2 Kernel PCA

One problem with PCA is that it assumes that the directions of variation are all straight lines. This is often not true. We can use the auto-associator with multiple hidden layers as just discussed, but there is a very nice extension to PCA that uses the kernel trick (which we saw in Section 5.2) to get around this problem, just as the SVM got around it for the Perceptron. Just as we did there, we apply a (possibly non-linear) function $\Phi(\cdot)$ to each datapoint $\mathbf{x}$ that transforms the data into the kernel space, and then perform normal linear PCA in that space. The covariance matrix is defined in the kernel space and is:

$$\mathbf{C} = \frac{1}{N}\sum_{n=1}^{N}\Phi(\mathbf{x}_n)\Phi(\mathbf{x}_N)^T, \tag{10.16}$$

which produces the eigenvector equation:

$$\lambda\left(\Phi(\mathbf{x}_i)\mathbf{V}\right) = \left(\Phi(\mathbf{x}_i)\mathbf{C}\mathbf{V}\right) \;\; i = 1\ldots N, \tag{10.17}$$

where $\mathbf{V} = \sum_{j=1}^{N}\boldsymbol{\alpha}_j\Phi(\mathbf{x}_j)$ are the eigenvectors of the original problem and the $\boldsymbol{\alpha}_j$ will turn out to be the eigenvectors of the 'kernelized' problem. It is at this point that we can apply the kernel trick and produce an $N \times N$ matrix $\mathbf{K}$, where:

$$\mathbf{K}_{(i,j)} = \left(\Phi(\mathbf{x}_i)\cdot\Phi(\mathbf{x}_j)\right). \tag{10.18}$$

Putting these together we get the equation $N\lambda\mathbf{K}\boldsymbol{\alpha} = \mathbf{K}^2\boldsymbol{\alpha}$, and we left-multiply by $\mathbf{K}^{-1}$ to reduce it to $N\lambda\boldsymbol{\alpha} = \mathbf{K}\boldsymbol{\alpha}$. Computing the projection of a new point $\mathbf{x}$ into the kernel PCA space requires:

$$\left(\mathbf{V}^k\cdot\Phi(\mathbf{x})\right) = \sum_{i=1}^{N}\boldsymbol{\alpha}_i^k\left(\Phi(\mathbf{x}_i)\cdot\Phi(\mathbf{x}_j)\right). \tag{10.19}$$

This is all there is to the algorithm.

Copyrighted material

Hidden page



FIGURE 10.9: Plot of the first two non-linear principal components of the iris data, showing that the three classes are clearly distinguishable.

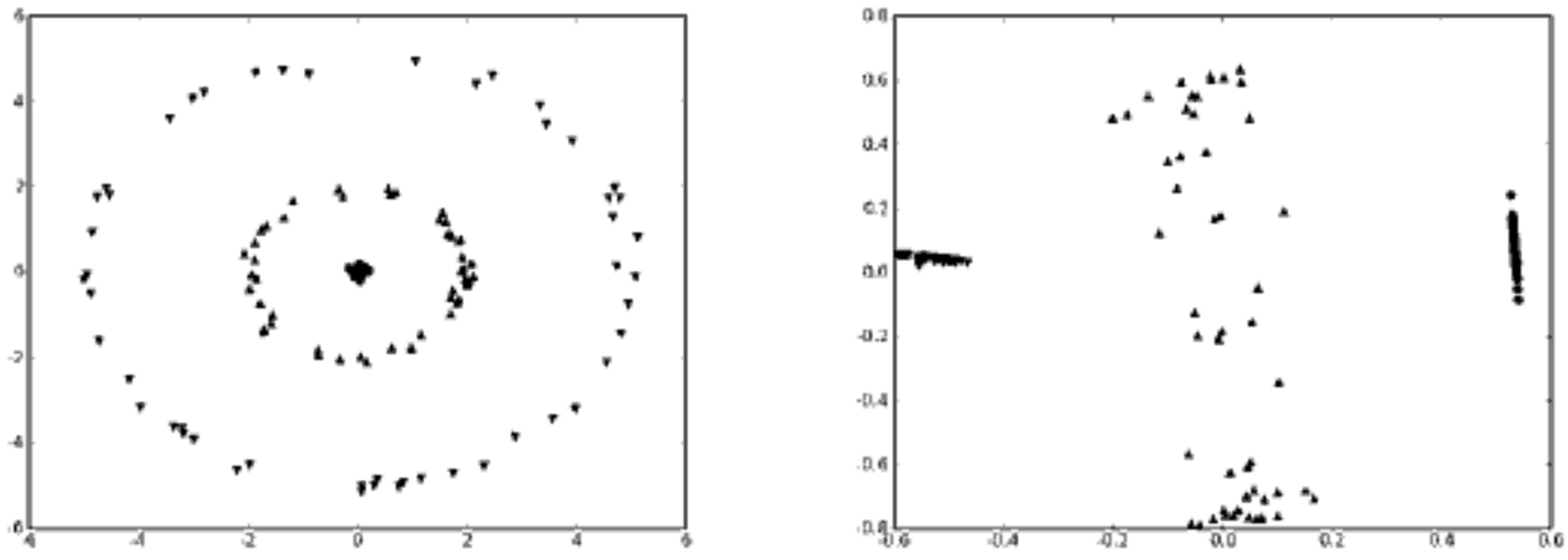


FIGURE 10.10: A very definitely non-linear dataset consisting of three concentric circles, and the kernel PCA mapping of the data, which requires only one component to separate the data.

are sampled from three concentric circles. Clearly, linear PCA would not be able to separate this data, but applying kernel PCA to this example separates the data using only one component.

10.3 Factor Analysis

The idea of factor analysis is to ask whether the data that is observed can be explained by a smaller number of uncorrelated factors or latent variables. The assumption is that the data comes from some underlying data source (or set of data sources) that are not directly known. The problem of factor analysis is to find those independent factors, and the noise that is inherent in the measurements of each factor. Factor analysis is commonly used in psychology and other social sciences, and the factors are generally chosen to have some particular meanings: in psychology, they can be related to IQ and

Copyrighted material

Hidden page

$$\mathbf{W}_{new} = \left(\mathbf{y}E(\mathbf{x}|\mathbf{y})^T\right)\left(E(\mathbf{x}\mathbf{x}^T|\mathbf{y})\right)^{-1}, \tag{10.23}$$

$$\Psi_{new} = \frac{1}{N}\text{diagonal}\left(\mathbf{x}\mathbf{x}^T - WE(\mathbf{x}|\mathbf{y})\mathbf{y}^T\right), \tag{10.24}$$

where diagonal() ensures that the matrix retains values only on the diagonal and the expectations are:

$$E(\mathbf{x}|\mathbf{y}) = \mathbf{W}^T(\mathbf{W}\mathbf{W}^T + \Psi)^{-1}\mathbf{b} \tag{10.25}$$

$$E(\mathbf{x}\mathbf{x}^T|\mathbf{x}) - E(\mathbf{x}|\mathbf{y})E(\mathbf{x}|\mathbf{y})^T = \mathbf{I} - \mathbf{W}^T(\mathbf{W}\mathbf{W}^T + \Psi)^{-1}W. \tag{10.26}$$

The only other things that we need to add to the algorithm is some way to decide when to stop, which involves computing the log likelihood and stopping the algorithm when it stops descending. This leads to an algorithm where the basic steps in the loop are:

```
# E-step
A = dot(W,transpose(W)) + diag(Psi)
logA = log(abs(linalg.det(A)))
A = linalg.inv(A)

WA = dot(transpose(W),A)
WAC = dot(WA,C)
Exx = eye(nRedDim) - dot(WA,W) + dot(WAC,transpose(WA))

# M-step
W = dot(transpose(WAC),linalg.inv(Exx))
Psi = Cd - (dot(W,WAC)).diagonal()

tAC = (A*transpose(C)).sum()

L = -N/2*log(2.*pi) -0.5*logA - 0.5*tAC
if (L-oldL)<(1e-4):
    print "Stop",i
    break
```

The output of using factor analysis on the iris dataset are shown in Figure 10.11.

Copyrighted material

Hidden page

Hidden page

Hidden page

Hidden page

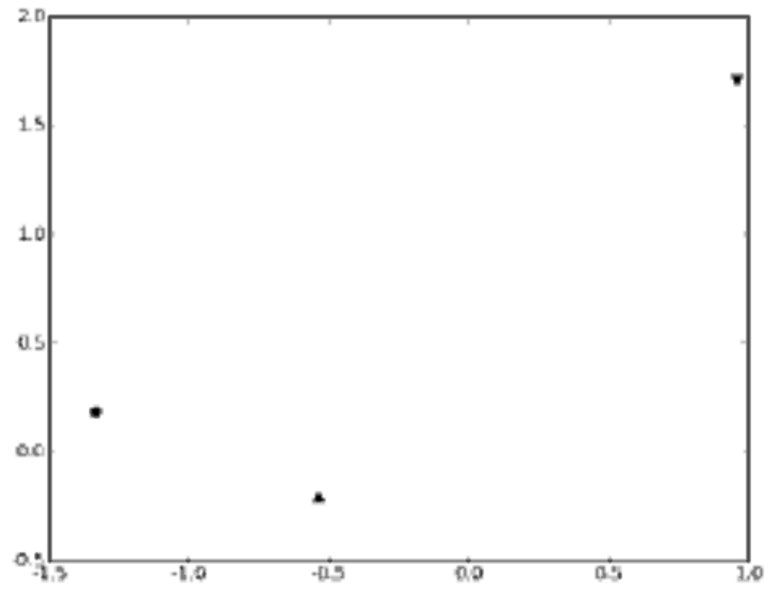


FIGURE 10.12: The Locally Linear Embedding algorithm with $k = 12$ neighbours transforms the iris dataset into three points, separating the data perfectly.

```
for i in range(ndata):
    Z  = data[neighbours[i,:],:] - kron(ones((K,1)),data[i,:])
    C = dot(Z,transpose(Z))
    C = C+identity(K)*1e-3*trace(C)
    W[:,i] = transpose(linalg.solve(C,ones((K,1))))
    W[:,i] = W[:,i]/sum(W[:,i])

M = eye(ndata,dtype=float)
for i in range(ndata):
    w = transpose(ones((1,shape(W)[0]))*transpose(W[:,i]))
    j = neighbours[i,:]
    ww = dot(w,transpose(w))
    for k in range(K):
 M[i,j[k]] -= w[k]
 M[j[k],i] -= w[k]
 for l in range(K):
            M[j[k],j[l]] += ww[k,l]

evals,evecs = linalg.eig(M)
ind = argsort(evals)
y = evecs[:,ind[1:nRedDim+1]]*sqrt(ndata)
```

The LLE algorithm produces a very interesting result on the iris dataset: it separates the three groups into three points (Figure 10.12). This shows that the algorithm works very well on this type of data, but doesn't give us any hints as to what else it can do. Figure 10.13 shows a common demonstration dataset for these algorithms. Known as the swissroll for obvious reasons, it is tricky to find a 2D representation of the 3D data because it is rolled up. The right of Figure 10.13 shows that LLE can successfully unroll it.

Copyrighted material

Hidden page

Hidden page

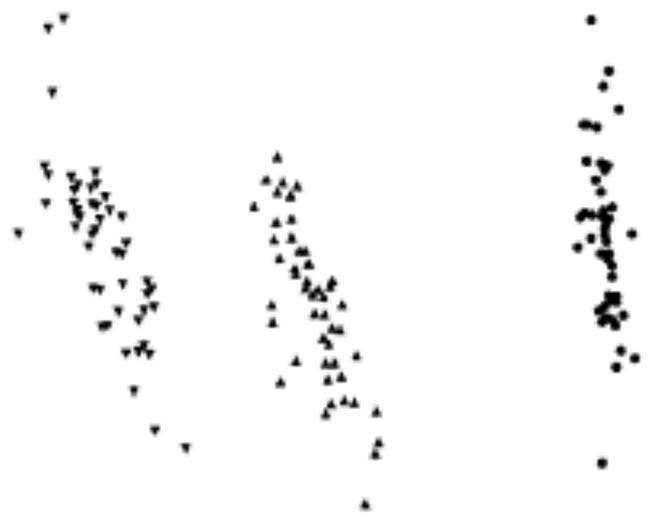


FIGURE 10.14: Isomap transforms the iris data in a similar way to factor analysis, provided that the neighbourhood size is large enough to avoid points becoming disconnected.

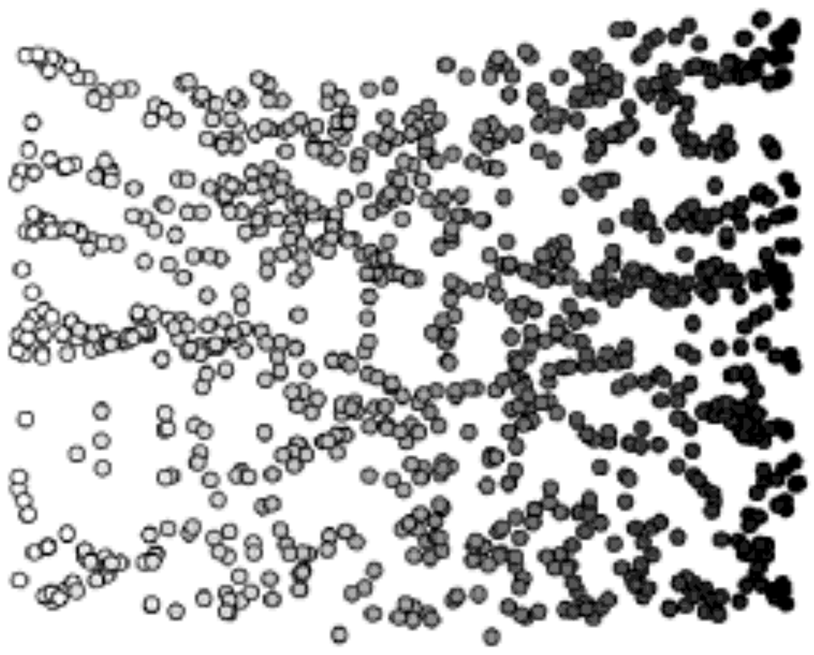


FIGURE 10.15: Isomap also produces a good remapping of the swissroll dataset.

respectively. Any good algorithms textbook provides the details if you don't know them.

There is one practical aspect of Isomap, which is that getting the number of neighbours right can be important, otherwise the graph splits into separate components (that is, segments of the graph that are not linked to each other), which have infinite distance between them. You then have to be careful to deal only with the largest component, which means that you end up with less data than you started with. Otherwise the implementation is fairly simple.

Figure 10.14 shows the results of applying Isomap to the iris dataset. Here, the default neighbourhood size of 12 produced a largest component that held only one of the three classes, and the other two were deleted. By increasing the neighbourhood size over 50, so that each point had more neighbours than were in its class, the results shown in the figure were produced. On the swissroll dataset shown on the left of Figure 10.13, Isomap produces qualitatively similar results to LLE, as can be seen in Figure 10.15.

Although the two algorithms produce similar mappings of the swissroll dataset, they are based on different principles. Isomap attempts to find a

Copyrighted material

mapping that preserves the distances between pairs of points within the manifold, no matter how far apart they are, while LLE focuses only on local regions of the manifold. This means that the computational cost of LLE is significantly less, but it can make errors by putting points close together that should be far apart. The choice of which algorithm to use often depends upon the dataset, and trying both of them out for your particular dataset is often a good idea.

Further Reading

Surveys of the area of dimensionality reduction include:

- L.J.P. van der Maaten. An introduction to dimensionality reduction using MATLAB. Technical Report MICC 07-07, Maastricht University, Maastricht, the Netherlands, 2007.
- F. Camastra. Data dimensionality estimation methods: a survey. *Pattern Recognition*, 36:2945–2954, 2003.

For more information about many of the methods desribed here, there are books or papers that contain a lot of information. Notable references include:

- (for LDA) Section 4.3 of T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, Berlin, Germany, 2001.
- (for PCA) I.T. Jolliffe. *Principal Components Analysis*. Springer, Berlin, Germany, 1986.
- (for kernel PCA) J. Shawe-Taylor and N. Cristianini. *Kernel Methods for Pattern Analysis*. Cambridge University Press, Cambridge, UK, 2004.
- (for ICA) J.V. Stone. *Independent Components Analysis: A Tutorial Introduction*. MIT Press, Cambridge, MA, USA, 2004.
- (for ICA) A. Hyvrinen and E. Oja. Independent components analysis: Algorithms and applications. *Neural Networks*, 13(4–5):411–430, 2000.
- (for LLE) S. Roweis and L. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.
- (for MDS) T.F. Cox and M.A.A. Cox. *Multidimensional Scaling*. Chapman & Hall, London, UK, 1994.

Copyrighted material

- (for Isomap) J.B. Tenenbaum, V. de Silva, and J.C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

- Chapter 12 of C.M. Bishop. *Pattern Recognition and Machine Learning*. Springer, Berlin, Germany, 2006.

Practice Questions

Problem 10.1 Use LDA on the Iris dataset (which is what Fisher originally tested LDA on).

Problem 10.2 Compare the results with using PCA, which is not supervised and will not therefore be able to find the same space.

Problem 10.3 Compute the eigenvalues and eigenvectors of:

$$\begin{pmatrix} 5 & 7 \\ -2 & -4 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \begin{pmatrix} 1 & 2 & 1 \\ 6 & -1 & 0 \\ -1 & -2 & -1 \end{pmatrix} \tag{10.33}$$

Problem 10.4 Compare the algorithms described in this chapter on a variety of different datasets, including the `yeast` dataset and the `wine` dataset. Input the results of the data reduction method to the MLP and SOM. Are the results better than before this preprocessing?

Problem 10.5 Modify the Isomap code to use Dijkstra's algorithm rather than Floyd's algorithm.

Problem 10.6 Another dataset that the Isomap and LLE algorithms are commonly demonstrated on is the 'S' shape that is available on the website. Download it and test various algorithms, not just Isomap and LLE on it. For Isomap and LLE, try different numbers of neighbours to see the effect that this has.

Copyrighted material

Hidden page

Hidden page

Hidden page

Hidden page

Hidden page

value of a function at a point in terms of its derivatives. So, for a function $f(\mathbf{x})$:

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \nabla f(\mathbf{x})|_{\mathbf{x}_0}(\mathbf{x}-\mathbf{x}_0) + \frac{1}{2}(\mathbf{x}-\mathbf{x}_0)^T \nabla^2 f(\mathbf{x})|_{\mathbf{x}_0}(\mathbf{x}-\mathbf{x}_0) + \ldots, \quad (11.2)$$

where $\mathbf{x}_0$ is a common, but potentially slightly confusing notation for the initial guess $\mathbf{x}(0)$, the $|_{\mathbf{x}_0}$ notation means that the function is evaluated at that point, and $\mathbf{J}(\cdot)$ is the Jacobian matrix of first derivatives:

$$\mathbf{J}(\mathbf{x}) = \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ & \cdots & \\ \frac{\partial f_n(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_n(\mathbf{x})}{\partial x_n} \end{pmatrix} \quad (11.3)$$

and $\mathbf{H}(\mathbf{x})$ is the Hessian matrix of second derivatives defined in a similar way to $\mathbf{J}(\mathbf{x})$, but using second derivatives. If $f(\mathbf{x})$ is a scalar function (so that it returns just 1 number) then $\mathbf{J}(\mathbf{x}) = \nabla f(\mathbf{x})$ and is a vector and $\mathbf{H}(\mathbf{x}) = \nabla^2 f(\mathbf{x})$ is a two-dimensional matrix. For a vector $\mathbf{f}(\mathbf{x})$, $\mathbf{J}(\mathbf{x})$ is a two-dimensional matrix and $\mathbf{H}(\mathbf{x})$ is three-dimensional.

If we choose to minimise Equation (11.2) exactly as it is written (i.e., ignoring third derivatives and higher), then we find the Newton direction at the kth iteration to be: $\mathbf{p}_k = -(\nabla^2 f(\mathbf{x}_k))^{-1} \nabla f(\mathbf{x}_k)$. There is something important to notice about this equation, which is that we actually use the inverse of the Hessian. Computing this is generally of order $\mathcal{O}(N^3)$ (where N is the number of elements in the matrix) which makes this a computationally expensive method. The compensation for this cost is that we don't really have to worry about the stepsize at all; it is always set to 1.

Implementing this requires only 1 line of change to our basic steepest descent algorithm, plus the addition of a function that computes the Hessian. The line to change is the one that computes $\mathbf{p}_k$, which becomes:

```
p = -dot(linalg.inv(Hessian(x)),Jacobian(x))
```

For this simple example, this algorithm goes straight to the correct answer in one step, which is much better than the steepest descent method that we saw earlier. However, for more complicated functions it won't work as well, and we can do better, as we shall see.

11.2.2 The Levenberg-Marquardt Algorithm

For least-squares problems, the objective function that we are optimising is:

$$f(\mathbf{x}) = \frac{1}{2}\sum_{j=1}^{m} r_j^2(\mathbf{x}) = \frac{1}{2}\|\mathbf{r}(\mathbf{x})\|_2^2, \quad (11.4)$$

Copyrighted material

where the $\frac{1}{2}$ makes the derivative nicer, and $\mathbf{r}(\mathbf{x}) = (r_1(\mathbf{x}), r_2(\mathbf{x}), \ldots, r_m(\mathbf{x}))^T$. In this last version, we can write the Jacobian of $\mathbf{r}$ as:

$$\mathbf{J}(\mathbf{x}) = \begin{Bmatrix} \frac{\partial r_1}{\partial x_1} & \frac{\partial r_2}{\partial x_1} & \cdots & \frac{\partial r_m}{\partial x_1} \\ \frac{\partial r_1}{\partial x_2} & \frac{\partial r_2}{\partial x_2} & \cdots & \frac{\partial r_m}{\partial x_2} \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial r_1}{\partial x_n} & \frac{\partial r_2}{\partial x_n} & \cdots & \frac{\partial r_m}{\partial x_n} \end{Bmatrix} = \left[\frac{\partial r_j}{\partial x_i}\right]_{j=1,\ldots,m,\ i=1,\ldots,n}. \tag{11.5}$$

This is useful because the function gradients that we want can mostly be computed directly:

$$\nabla f(\mathbf{x}) = \mathbf{J}(\mathbf{x})^T \mathbf{r}(\mathbf{x}) \tag{11.6}$$

$$\nabla^2 f(\mathbf{x}) = \mathbf{J}(\mathbf{x})^T \mathbf{J}(\mathbf{x}) + \sum_{j=1}^{m} r_j(\mathbf{x}) \nabla^2 r_j(\mathbf{x}). \tag{11.7}$$

The upshot of this is that knowing the Jacobian gives you the first (and usually, most important) part of the Hessian effectively without any additional computational cost, and it is this that special algorithms can exploit to solve least-squares problems efficiently. To see this, remember that, as in all of the other gradient-descent algorithms that we have looked at, we are approximating the function by the Taylor series (Equation (11.2)) up to second-order (Hessian) terms.

If $\|\mathbf{r}(\mathbf{x})\|$ is linear (which means that $f(\mathbf{x})$ is quadratic), then the Jacobian is constant and $\nabla^2 r_j(\mathbf{x}) = 0$ for all j. In this case, $f(\mathbf{x}) = \frac{1}{2}\|\mathbf{J}\mathbf{x} + \mathbf{r}\|^2$, and at a solution:

$$\nabla f(\mathbf{x}) = \mathbf{J}^T(\mathbf{J}\mathbf{x} + \mathbf{r}) = 0, \tag{11.8}$$

and so:

$$\mathbf{J}^T\mathbf{J}\mathbf{x} = \mathbf{J}^T\mathbf{r}(\mathbf{x}). \tag{11.9}$$

We can use this along with some linear algebra to find $\mathbf{x}$ in a variety of different ways, such as Cholesky factorisation, QR factorisation, and using the Singular Value Decomposition. We will look at the last of these methods, since it uses eigenvectors, which we have already seen in Chapter 10. The Singular Value Decomposition (SVD) is the decomposition of a matrix $\mathbf{A}$ of size $m \times n$ into:

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T, \tag{11.10}$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices (i.e., the inverse of the matrix is its transpose, so $\mathbf{U}^T\mathbf{U} = \mathbf{U}\mathbf{U}^T = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix). $\mathbf{U}$ is of size $m \times m$ and $\mathbf{V}$ is of size $n \times n$. $\mathbf{S}$ is a diagonal matrix of size $m \times n$, with the elements of this matrix, σ_i, being known as singular values.

Copyrighted material

Hidden page

The Levenberg-Marquardt Algorithm

- given start point $\mathbf{x}_0$
- while $\mathbf{J}^T\mathbf{r}(\mathbf{x})$ >tolerance and maximum number of iterations not exceeded:
  - repeat
    - solve $(\mathbf{J}^T\mathbf{J} + \nu\mathbf{I})\mathbf{dx} = -\mathbf{J}^T\mathbf{r}$ for $\mathbf{dx}$ using linear least-squares
    - set $\mathbf{x}_{\text{new}} = \mathbf{x} + \mathbf{dx}$
    - compute the ratio of the actual and prediction reductions:
      - actual $= \|f(\mathbf{x}) - f(\mathbf{x}_{new})\|$
      - predicted $= \nabla f^T(\mathbf{x}) \times \mathbf{x}_{new} - \mathbf{x}$
      - ρ = actual/predicted
    - if $0 < \rho < 0.25$:
      - accept step: $\mathbf{x} = \mathbf{x}_{\text{new}}$
    - else if $\rho > 0.25$:
      - accept step: $\mathbf{x} = \mathbf{x}_{\text{new}}$
      - increase trust region size (reduce ν)
    - else:
      - reject step
      - reduce trust region (increase ν)
  - until $\mathbf{x}$ is updated or maximum number of iterations is exceeded

We will look at two examples of using non-linear least-squares. One is a simple case of finding the minimum of a function that consists of two quadratic terms added together, i.e., a sum-of-squares problem, while the second is to minimise the fitting of a function to data.

The function that we will attempt to minimise is Rosenbrock's function:

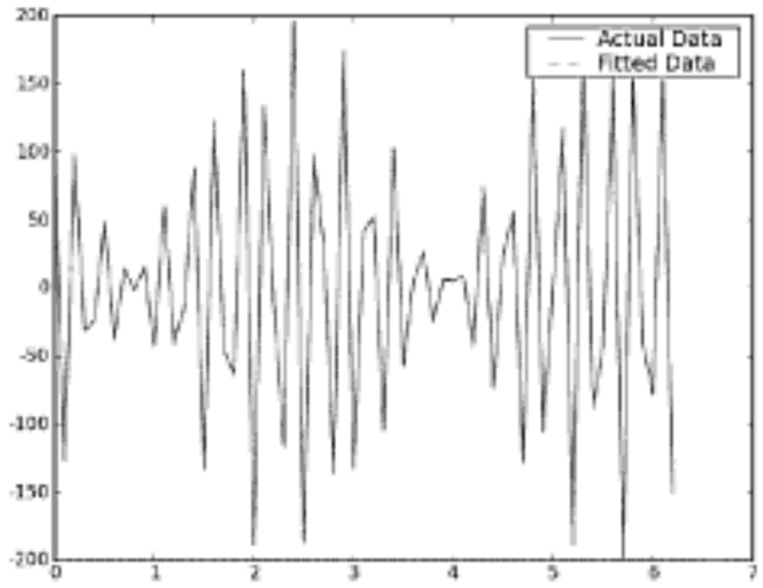
$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2. \tag{11.15}$$

This is a common problem to try since it has a long narrow valley, so finding the optimal solution is not especially easy (except by hand: if you look at the problem, then guessing that $x_1 = 1, x_2 = 1$ is the minimum is fairly obvious). You need to work out how to encode this in the form required for a sum-of-squares problem, which is basically to write:

$$\mathbf{r} = (10(x_2 - x_1^2), 1 - x_1)^T. \tag{11.16}$$

Copyrighted material

Hidden page



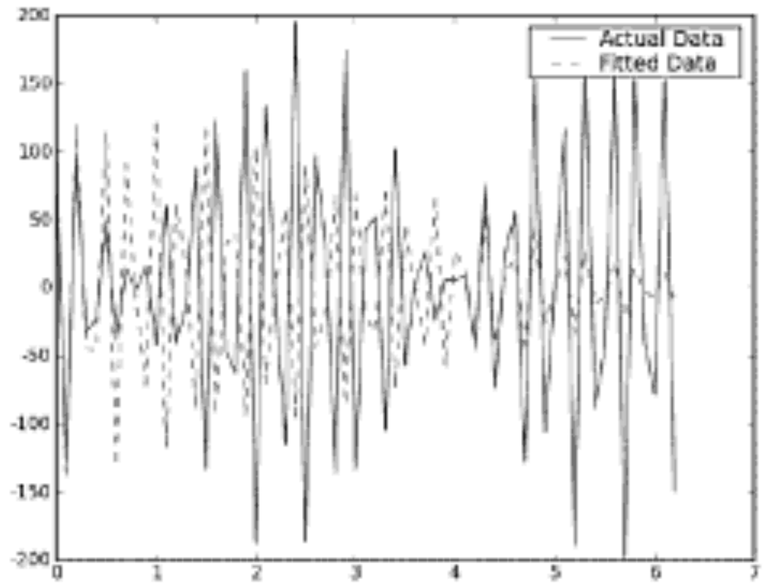


FIGURE 11.4: Using Levenberg-Marquardt for least-squares data fitting of data from Equation (11.18). The example on the left converges to the correct solution, while the one on the right, which still starts from a point close to the correct solution, fails to find it, resulting in significantly different output.

starting point is $(100.5, 102.5)$, while on the right it is $(101, 101)$. It can be seen that on this problem, Levenberg-Marquardt is very susceptible to local minima, since while the example on the left works (converging after only 8 iterations), the example on the right, which still starts with parameter values very close to the correct ones, gets stuck and fails, with final parameter values $(100.89, 101.13)$.

11.3 Conjugate Gradients

Not every problem that we want to solve is a least-squares problem. The good news is that we can do rather better than steepest descent even when we want to minimise an arbitrary objective function. The key to this is to look again at Figure 11.3, where you can see that there are several of the steepest gradient lines that are in pretty much the same direction. We would only need to go in that direction once if we knew how far to go the first time. And then we would go in a direction orthogonal (at right angles) to that one and, in two dimensions, we would be finished, as is shown on the right of Figure 11.5, where one step in the x direction and one in the y direction are enough to complete the minimisation. In n dimensions we would have to take n steps, and then we would have finished. This amazing scenario is the aim of the method of conjugate gradients. It manages to achieve it in the linear case, but in most non-linear cases, which are the kind we are usually interested in, it usually requires a few more iterations that it theoretically should, although still many less steps than most other methods for real problems.

It turns out that making the lines be orthogonal is generally impossible, since you don't have enough information about the solution space. However,

Copyrighted material

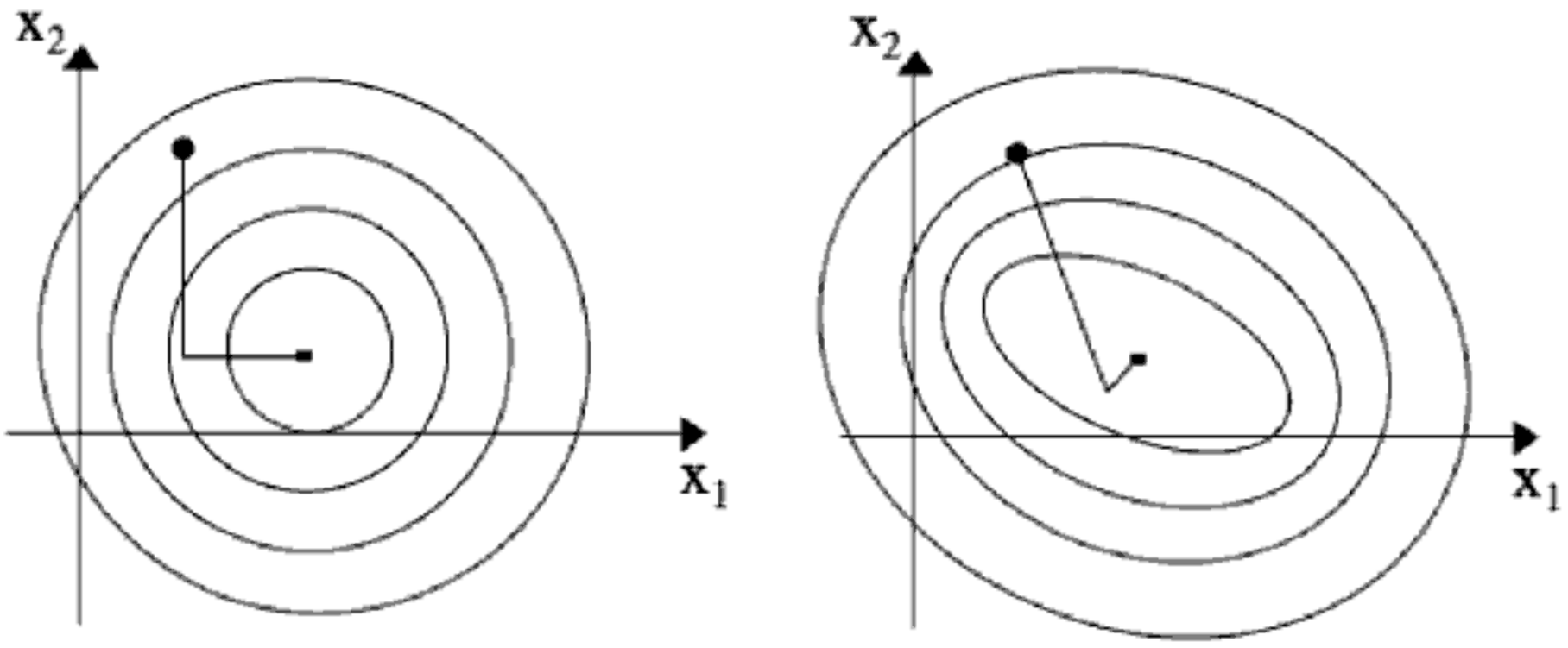


FIGURE 11.5: *Left:* If the directions are orthogonal to each other and the stepsize is correct, then only one step is needed for each dimension in the data, here two. *Right:* The conjugate directions are not orthogonal to each other on the ellipse.

it is possible to make them conjugate or A-orthogonal. Two vectors $\mathbf{p}_i, \mathbf{p}_j$ are conjugate if $\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0$ for some matrix $\mathbf{A}$. Conjugate lines for the ellipse contours in Figure 11.2 are shown on the right of Figure 11.5. Amazingly, the line search that we wrote down in Equation (11.1) is soluble along these directions, since they do not interfere with each other, with solution:

$$\alpha_i = \frac{\mathbf{p}_i^T(-\nabla f(\mathbf{x}_{i-1})}{\mathbf{p}_i^T \mathbf{A} \mathbf{p}_i}). \tag{11.20}$$

We then need to use a function to find the zeros of this. The Newton-Raphson iteration, which is one method that will do it, is described below. So if we can find conjugate directions, then the line search is much better. The only question that remains is how to find them. This requires a Gram-Schmidt process, which constructs each new direction by taking a candidate solution and then subtracting off any part that lies along any of the directions that have already been used. We start by picking a set of mutually orthogonal vectors $\mathbf{u}_i$ (the basic coordinate axes will do; there are better options, but they are beyond the scope of this book) and then using:

$$\mathbf{p}_k = \mathbf{u}_k + \sum_{i=0}^{k-1} \beta_{ki} \mathbf{p}_i. \tag{11.21}$$

There are two possible β terms that can be used. They are both based on the ratios between the squared Jacobian before and after an update. The Fletcher-Reeves formula is:

$$\beta_{i+1} = \frac{\nabla f(\mathbf{x}_{x+1})^T \nabla f(\mathbf{x}_{i+1})}{\nabla f(\mathbf{x}_i)^T \nabla f(\mathbf{x}_i)}, \tag{11.22}$$

Copyrighted material

Hidden page

The Conjugate Gradients Algorithm

- given start point $\mathbf{x}_0$, and stopping parameter ϵ, set $\mathbf{p}_0 = -\nabla f(\mathbf{x})$
- set $\mathbf{p}_{\text{new}} = \mathbf{p}_0$
- while $\mathbf{p}_{\text{new}} > \epsilon^2 \mathbf{p}_0$:
  - compute α_k and $\mathbf{x}_{\text{new}} = \mathbf{x} + \alpha_k \mathbf{p}$ using the Newton-Raphson iteration:
    - while $\alpha^2 dp > \epsilon^2$:
      - $\alpha = -(\nabla f(\mathbf{x})^T \mathbf{p})/(\mathbf{p}^T \mathbf{H}(\mathbf{x})\mathbf{p})$
      - $\mathbf{x} = \mathbf{x} + \alpha \mathbf{p}$
      - $dp = \mathbf{p}^T \mathbf{p}$
  - evaluate $\nabla f(\mathbf{x}_{\text{new}})$
  - compute β_{k+1} using Equation (11.22) or (11.23)
  - update $\mathbf{p}_{\text{new}} = \nabla f(\mathbf{x}_{\text{new}}) + \beta_{k+1}\mathbf{p}$
  - check for restarts

11.3.1 Conjugate Gradients Example

Computing the conjugate gradients solution to the same function as above: $f(\mathbf{x}) = (0.5x_1^2 + 0.2x_2^2 + 0.6x_3^2)$ makes use of the Jacobian and Hessian again. The first Newton-Raphson step yields an α value of 0.931, so that the next step is:

$$\mathbf{x}(1) = \begin{pmatrix} -2 \\ 2 \\ 0 \end{pmatrix} + 0.931 \times \begin{pmatrix} 2 \\ -0.8 \\ 2.4 \end{pmatrix} = \begin{pmatrix} -0.138 \\ 1.255 \\ 0.235 \end{pmatrix} \tag{11.27}$$

Then $\beta = 0.0337$, so that the direction is:

$$\mathbf{p}(1) = \begin{pmatrix} 0.138 \\ -0.502 \\ -0.282 \end{pmatrix} + 0.0337 \times \begin{pmatrix} 2 \\ -0.8 \\ 2.4 \end{pmatrix} = \begin{pmatrix} 0.205 \\ -0.529 \\ -0.201 \end{pmatrix} \tag{11.28}$$

In the second step, $\alpha = 1.731$,

$$\mathbf{x}(2) = \begin{pmatrix} -0.138 \\ 1.255 \\ 0.235 \end{pmatrix} + 1.731 \times \begin{pmatrix} 0.205 \\ -0.529 \\ -0.201 \end{pmatrix} = \begin{pmatrix} -0.217 \\ -0.136 \\ 0.136 \end{pmatrix} \tag{11.29}$$

and the update is:

Copyrighted material

$$p(2) = \begin{pmatrix} -0.217 \\ -0.136 \\ 0.136 \end{pmatrix} + 0.240 \times \begin{pmatrix} 0.205 \\ -0.529 \\ -0.201 \end{pmatrix} = \begin{pmatrix} -0.168 \\ -0.263 \\ 0.088 \end{pmatrix} \tag{11.30}$$

A third step then gives the final answer as $(0, 0, 0)$.

There are other methods of doing gradient descent, some of which are more effective on certain problems (but note that the No Free Lunch theorem tells us that no one solver will be the most effective for every problem). However, they are not necessary for an understanding of machine learning. The scientific Python libraries SciPy include a set of optimisation algorithms in `scipy.optimize` that might be worth a look if you want to find more complete algorithms in this area.

We will next consider what happens when the problems that we wish to solve are discrete, and so there is no gradient to find.

11.4 Search: Three Basic Approaches

We are going to discuss three different ways to attempt optimisation without gradients. For each one, we will see how it works on the Travelling Salesman Problem (TSP), which is a classic discrete optimisation problem that consists of trying to find the shortest route through a set of cities that visits each city exactly once and returns to the start. For the first (starting) city we can choose any of the N that are available. For the next, there are $N - 1$ choices, and for the next $N - 2$. Using a brute force search in this way provides a $\mathcal{O}(N!)$ solution, which is obviously infeasible. In fact, the TSP is an NP-hard problem. The best known solution that is guaranteed to find the global maximum is using dynamic programming and its computational cost is $\mathcal{O}(n^2 2^n)$, but we won't be considering that here—the TSP is an example, not a problem we really want to solve here. The basic search methods are described next.

11.4.1 Exhaustive Search

Try out every solution and pick the best one. While this is obviously guaranteed to find the global optimum, because it checks every single solution, it is impractical for any reasonable size problem. For the TSP it would involve testing out every single possible way of ordering the cities, and calculating the distance for each ordering, so the computational complexity is $\mathcal{O}(N!)$, which is worse than exponential. It is computationally infeasible to do the computations for more than about $N = 10$ cities. The basic part of the algorithm

Copyrighted material

Hidden page

Hidden page



FIGURE 11.6: A one-armed bandit machine. It has one arm, and it steals your money.

it will take a very long time to reach the optimal solution. The second is on a plateau, where no changes that the algorithm makes affect the solution. In this case the solution will just change randomly, if at all, and the maximum will probably not be found. The third case is when there is a very gently sloping ridge in the data. Most directions that the algorithm looks in will be downhill, and so it may decide that it has already reached the maximum.

11.5 Exploitation and Exploration

The search methods above can be separated into methods that perform exploration of the search space, always trying out new solutions, like exhaustive search, and those performing exploitation of the current best solution, by trying out local variations of that current best solution, like hill climbing. Ideally, we would like some combination of the two—we should be trying to improve on the current best solution by local search, and also looking around in case there is an even better solution hiding elsewhere in the search space.

One way to think about this is known as the n-armed bandit problem. Suppose that we have a room full of one-armed bandit machines in some tacky Las Vegas casino (for those who don't know, a one-armed bandit is a slot machine with a lever that you pull, as in Figure 11.6). You don't know anything about the machines in advance, such as what the payouts are, and how likely you are to get the payout. You enter the room with a fistful of 50 cent coins from your student loan, aiming to generate enough beer money to get through the year. How do you choose which machine to use?

At first, you have no information at all, so you choose randomly. However, as you explore, you pick up information about which machines are good (here,

Copyrighted material

Hidden page

Hidden page

Hidden page

Problem 11.2 Experiment with the Fletcher-Reeves and Polak-Ribiere formulas (Equations (11.22) and (11.23)) when solving Rosenbrock's function using conjugate gradients. Can you find places where one works better than the other?

Problem 11.3 Generate data from the equation $a(1 - \exp(-b(x - c)))$ for choice of parameters a, b, c and x in the range -5 to 5 (with noise). Use Levenberg-Marquardt to fit the parameters.

Problem 11.4 It is possible to use the exact gradient descent methods described in this chapter for an MLP. The first thing you have to do is to put the various sets of weights into one data structure (which is easy: simply put the two sets into one array). You then need to work out what the derivatives are for these different functions, and then compute the Hessian. The structure can then be passed to the optimisation code. Implement this and compare the results to using back-propagation.

Problem 11.5 By incorporating back-tracking into hill climbing, it is possible to escape from some poor local maxima. Add this into the code and test the results on the Travelling Salesman problem.

Problem 11.6 The logical satisfiability problem is an NP-complete problem that consists of finding truth assignments to sets of logical statements (e.g., $(a_1 \wedge a_2) \vee (\neg a_1 \vee a_3)$) so that they are true. It is an NP-complete problem to find truth assignments. Devise a way to use hill climbing and simulated annealing on the problem.

Copyrighted material

Chapter 12

Evolutionary Learning

In this chapter we are going to start by treating evolution the same way that we treated neuroscience earlier in the book—by cherry-picking a few useful concepts, and then filling in the gaps with computer science in order to make an effective learning method. To see why this might be interesting, you need to view evolution as a search problem. We don't generally think of it in this way, but animals are competing with each other in all kinds of ways—for example, eating each other—which encourages them to try to find camouflage colours, become toxic to certain predators, etc.

Evolution works on a population through an imaginary fitness landscape, which has an implicit bias towards animals that are 'fitter,' i.e., those animals that live long enough to reproduce, are more attractive, and so get more mates, and generate more and healthier offspring. You can find out more from hundreds of books, such as Charles Darwin's "The Origin of Species" (the original book on the topic, still in print and very interesting) and Richard Dawkin's "The Blind Watchmaker."

The genetic algorithm models the genetic process that gives rise to evolution. In particular, it models sexual reproduction, where both parents give some genetic information to their offspring. As is sketched in Figure 12.1, in biological organisms, each parent passes on one chromosome out of their two, and so there is a 50% chance of any gene making it into the offspring. Of the two versions of each gene (one from each parent) one allele (variation) is selected. Hence, children have similarities with their parents, and there is lots of genetic inheritance. However, there are also random mutations, caused by copying errors when the chromosome material is reproduced, which mean that some things do change over time. Real genetics is obviously a lot more complicated than this, but we are taking only the things that we want for our model.

The genetic algorithm shows many of the things that are best and worst about machine learning: it is often, but not always, very effective, it has an array of parameters that are crucial, but hard to set, and it is impossible to guarantee that it will find a result that is any good at all. Having said all that, it often works very well, and it has become a very popular algorithm for people to use when they have no idea of any other way to find a reasonable solution.

In the terms that we saw at the end of the previous chapter, genetic al-

Copyrighted material

Hidden page

- a way to calculate the fitness of a solution
- a selection method to choose parents
- a way to generate offspring by breeding the parents

These items are all described in the following sections, and the basic algorithm is described. We are going to use an example to describe the methods, which is an NP-complete problem (if you are not familiar with the term NP-complete, its practical implication is that the problem runs in exponential time in the number of inputs) known as the knapsack problem (a knapsack is a rather old name for a rucksack or bag). Sections 12.3.1 and 12.3.4 provide other examples. The knapsack problem is easy to describe, but difficult to solve in general. Here is the version of it that we will use:

Suppose that you are packing for your holidays. You've bought the biggest and best rucksack that was for sale, but there is still no way that you are going to fit in everything you want to take (camera, money, addresses of friends, etc.) and the things that your mum is insisting you take (spare underwear, phrasebook, stamps to write home with, etc.). As a good computer scientist you decide to assign a value to each item, and measure how much space it takes up. Then you want to maximise the value of the items you will take with you, with the constraint that everything has to fit into the bag.

This problem, and variations of it, appear in various disguises in cryptography, combinatorics, applied mathematics, logistics, and business, so it is an important problem. Unfortunately, it is also NP-complete, so finding the optimal solution for interesting cases is computationally impossible. We are going to find approximations to the correct solution using a Genetic Algorithm.

12.1.1 String Representation

The first thing that we need is some way to represent the individual solutions, in analogy to the chromosome. GAs use a string, with each element of the string (equivalent to the gene) being chosen from some alphabet. The different values in the alphabet, which is often just binary, are analogous to the alleles. For the problem we are trying to solve we have to work out a way of encoding the description of a solution as a string. We then create a set of random strings to be our initial population.

It is possible to modify the GA so that the alphabet it uses runs over the real numbers. While purists don't think that this is a GA at all, it is quite popular, because of the number of applications, but it is not as elegant as using a discrete alphabet. It also makes the mutation operator that we will see later less useful.

For the knapsack problem the alphabet is very simple, since we can make it binary. We make the string L units long, where L is the total number of things we would like to take with us, and make each unit a binary digit. We then encode a solution using 0 for the things we will not take and 1 for the

Copyrighted material

things we will. So if there were four things we wanted to take, then $(0, 1, 1, 0)$ would mean that we take the middle two, but not the first or last.

Note that this does not tell us whether or not this string is possible (that is, whether it will fit into the knapsack), nor whether it is a good string (whether it fills the knapsack). To work these out we need some way to decide how well each string fulfills the problem criteria. This is known as the fitness of the string.

12.1.2 Evaluating Fitness

The fitness function can be seen as an oracle that takes a string as an argument and returns a value for that string. It is the only problem-specific part of the algorithm. It is worth thinking about what we want from our fitness function. Clearly, the best string should have the highest fitness, and the fitness should decrease as the strings do less well on the problem. In general, fitness should always be a positive function—even the least fit strings should have fitness of at least zero. In real evolution, the fitness landscape is not static: there is competition between different species, such as predators and prey, or medical cures for certain diseases, and so the measure of fitness changes over time. We'll ignore that in the genetic algorithm.

For the knapsack problem, we could decide that we want to make the bag as full as possible. So we would need to know the volume of each item that we want to put into the knapsack, and then for a given string that says which things should be taken, and which should not, we can compute the total volume. This is then a possible fitness function. However, it does not tell us anything about whether they will fit into the bag—with this fitness function the optimal solution is to take everything. So we need to check that they will fit, and if they will not, reduce the fitness of that solution. One option would be to set the fitness to 0 if it will not fit. However, suppose that the solution is almost perfect, it is just that there is one thing too many in the knapsack. By setting the fitness to 0 we are reducing the chance of this solution being allowed to evolve and improve during later iterations. For this reason we will make the fitness function be the sum of the values of the items to be taken if they fit into the knapsack, but if they do not we will subtract twice the amount by which they are too big for the knapsack from the size of the knapsack. This allows solutions that are only just over to be considered for improvement, but tries to ensure that they are not the fittest solutions around.

There is an obvious greedy algorithm that finds solutions to the knapsack problem. At each stage it takes the largest thing that hasn't been packed yet and that will still fit into the bag, and iterates that rule. This will not necessarily return the optimal solution (unless each thing is larger than the sum of all the ones smaller than it, in which case it will), but it is very quick and simple. So a GA should be getting a much better solution than the greedy rule in general to be worth all the effort involved in writing and running it.

Copyrighted material

12.1.3 Population

We can now measure the fitness of any string. The GA works on a population of strings, with the first generation usually being created randomly. The fitness of each string is then evaluated, and that first generation is bred together to make a second generation, which is then used to generate a third, and so on. After the initial population is chosen randomly, the algorithm evolves in such a way that the fitness of individuals in the population increases over the generations.

So for the knapsack problem, we will now create a set of random binary strings of length L by using the random number generator. We'll arbitrarily decide to make 100 strings. We now need to choose parents out of this population, and start breeding them. At every iteration the population stays the same size, something else that is unlike real evolution. Creating the initial population is very easy in NumPy using the uniform random number generator and the `where()` function:

```
pop = random.rand(popSize,stringLength)
pop = where(pop<0.5,0,1)
```

12.1.4 Generating Offspring: Parent Selection

For the current generation we need to select those strings that will be used to generate new offspring. The idea here is that fitness will improve if we select strings that are already relatively fit compared to the other members of the population (following natural selection). This is exploitation of our current population. However, it is also good to allow some exploration in there, which means that we have to allow some possibility of weak strings being considered. The basic idea is that we choose strings proportionally to their fitness, so that fitter strings are more likely to be chosen to enter the 'mating pool.' There are two commonly employed ways to do this, although the second one is better:

Truncation Selection A simple method is just to pick some fraction f of the best strings and ignore the rest. For example, $f = 0.5$ is often used, so the best 50% of the strings are put into the mating pool, and chosen with equal probability. This is obviously very easy to implement, but it does limit the amount of exploration that is done, biasing the GA towards exploitation.

Fitness Proportional Selection The better option is to select strings probabilistically, with the probability of a string being selected being proportional to its fitness. The function that is generally used is (for string α):

Copyrighted material

$$p^\alpha = \frac{F^\alpha}{\sum_{\alpha'} F^{\alpha'}}, \tag{12.1}$$

where F^α is the fitness. This probabilistic interpretation is the reason why fitness should be positive. If they aren't guaranteed positive, then Boltzmann selection can be used to make them so (where s is the selection strength, a parameter, and you might recognise the equation as the softmax activation from Chapter 3):

$$p^\alpha = \frac{\exp(sF^\alpha)}{\sum_{\alpha'} \exp(sF^{\alpha'})}. \tag{12.2}$$

There is an implementation issue here. We want to pick each string with probability proportional to its fitness, but if we only have one copy of each string, then the probability of picking each string is the same. One way around this is to add more copies of the fitter strings, so that they are more likely to get chosen. This is sometimes called 'roulette selection,' because if you imagine that each string gets an area on a roulette wheel, then the larger the area associated to one number, the more likely it is that the ball will land there. You can then just randomly pick strings from this larger set. A method of doing this is shown in the following code snippet, which uses the `kron()` function. We've seen this before (in Section 10.5); it is a NumPy function that multiples each element of its first array argument by every element of the second, putting all of the results together into one multi-dimensional output array. It is useful here in order to populate the new and much larger `newPopulation` array, which contains multiple copies of each string.

```
# Put in repeated copies of each string according to fitness
# Deal with strings with very low fitness
j=0
while round(fitness[j])<1:
 j = j+1

newPop = kron(ones((round(fitness[j]),1)),pop[j,:])

# Add multiple copies of strings into the newPop
for i in range(j+1,self.popSize):
 if round(fitness[i])>=1:
  newPop = concatenate((newPop,kron(ones((round(fitness[i]),1))
  ,pop[i,:])),axis=0)

# Shuffle the order (note that there are still too many)
indices = range(shape(newPop)[0])
```

Copyrighted material

```
random.shuffle(indices)
newPop = newPop[indices[:popSize],:]
return newPop
```

However we select the strings to put into the mating pool, the next operation is to put them into pairs. Since the order that they are in is random, we can simply pair up the strings so that each even-indexed string takes the following odd-indexed one as its mate.

12.2 Generating Offspring: Genetic Operators

Having selected our breeding pairs, we now need to decide how to combine their two strings to generate the offspring, which is the genetics part of the algorithm. There are two genetic operators that are generally used, and they are discussed now. There are others, but these were the original choices, and are far and away the most common.

12.2.1 Crossover

In biology, organisms have two chromosomes, and each parent donates one of them. Members of our GA population only have one chromosome-equivalent, the string. Thus, we generate the new string as part of the first parent and part of the second. The most common way of doing this is to pick one point at random in the string, and to use parent 1 for the first part of the string, up to the crossover point and parent 2 for the rest. We actually generate two offspring, with the second one consisting of the first part of parent 2 and the second part of parent 1. This scheme is known as single point crossover, and the extension to multi-point crossover is hopefully obvious. The most extreme version is known as uniform crossover and consists of independently selecting each element of the string at random from the two parents. The three types of crossover are shown in Figure 12.2.

Crossover is the operator that performs global exploration, since the strings that are produced are radically different to both parents. The hope is that sometimes we will take good parts of both solutions and put them together to make an even better solution. A nice picture example is to imagine a bird that has webbed feet for good swimming, but that cannot fly, breeding with a bird that can fly, but not swim. The offspring? A duck! Obviously, this is not biologically plausible, but it is a good picture of how crossover works. One interesting feature of the GA that obviously isn't true in real genetics is that in addition to the duck the algorithm would produce the bird that can't fly or

Copyrighted material

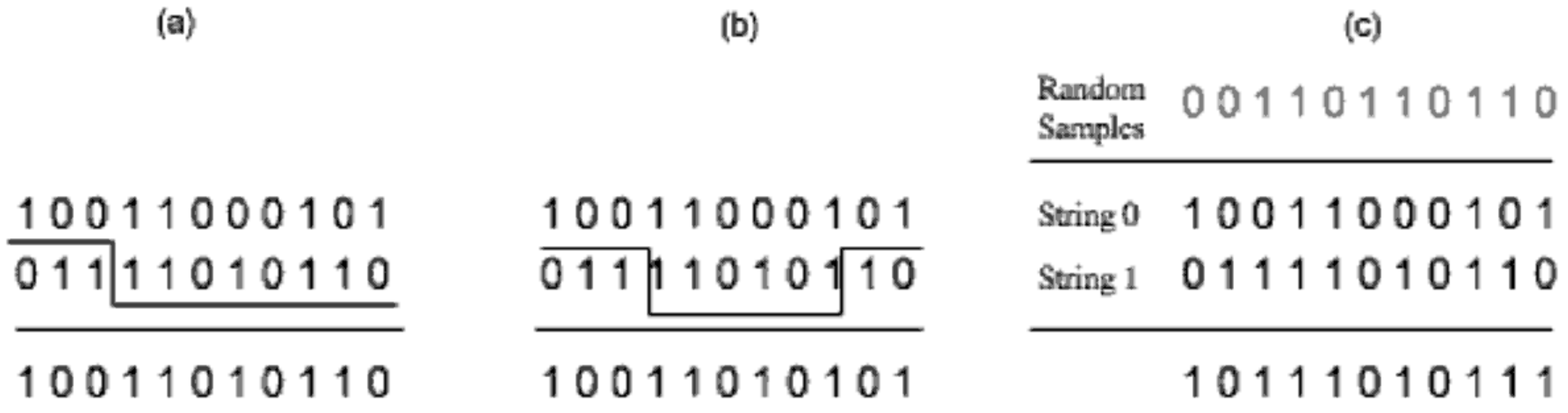


FIGURE 12.2: The different forms of the crossover operator. (a) Single point crossover. A position in the string is chosen at random, and the offspring is made up of the first part of parent 1 and the second part of parent 2.(b) Multi-point crossover. Multiple points are chosen, with the offspring being made in the same way. (c) Uniform crossover. Random numbers are used to select which parent to take each element from.

swim, although it is unlikely to last long since its fitness will presumably not be high. In fact, there are exceptions to this, such as the great New Zealand Kiwi, which can neither swim nor fly, but is happily not extinct.

The following code snippet shows a NumPy implementation of single point crossover. The extension to multi-point and uniform crossover is not particularly difficult.

```
def spCrossover(pop):
    newPop = zeros(shape(pop))
    crossoverPoint = random.randint(0,stringLength,popSize)
    for i in range(0,self.popSize,2):
 newPop[i,:crossoverPoint[i]] = pop[i,:crossoverPoint[i]]
 newPop[i+1,:crossoverPoint[i]] = pop[i+1,:crossoverPoint[i]]
 newPop[i,crossoverPoint[i]:] = pop[i+1,crossoverPoint[i]:]
 newPop[i+1,crossoverPoint[i]:] = pop[i,crossoverPoint[i]:]
    return newPop
```

Crossover is not always useful, depending upon the problem; for example, in the Travelling Salesman Problem that we talked about in Chapter 11, the strings that are generated by crossover might not even be valid tours. However, when it is useful, it is often the more powerful of the genetic operators, and has led to the building block hypothesis of how GAs work. The idea is that GAs work well on problems where the solution comes from putting together lots of little solutions, so that different strings assemble each separate building block, and then crossover puts those substrings together to make the final solution.

Copyrighted material

Hidden page

Hidden page

The Basic Genetic Algorithm

- **Initialisation**
  - generate N random strings of length L with our chosen alphabet
- **Learning**
  - repeat:
    - create an (initially empty) new population
    - repeat:
      - select two strings from current population by fitness
      - recombine them to produce two new strings
      - mutate the offspring
      - either add the two offspring to the population or use elitism or tournaments
      - keep track of the best string in the population
    - until N strings for the new population are generated
    - replace the current population with the new population
  - until stopping criteria met

12.3 Using Genetic Algorithms

12.3.1 Map Colouring

Graph colouring is a typical discrete optimisation problem. We want to colour a graph using only k colours, and choose them in such a way that adjacent regions have different colours. It has been mathematically proven that any two-dimensional planar graph can be coloured with four colours, which was the first ever proof that used a computer program to check the cases. Even though it might be impossible, we are going to try to solve the three-colour problem using a genetic algorithm, we just won't be upset if the solution isn't perfect (this is a good idea with a GA anyway, of course). With all problems where you want to apply a genetic algorithm, there are three basic tasks that need to be performed:

Encode possible solutions as strings For this problem, we'll choose our alphabet to consist of the three possible shades (black (b), dark (d), and light (l), say). So for a six-region map, a possible string is $\alpha = \{bdblbb\}$. This says that the first region is black, the second dark grey, etc. We

Copyrighted material

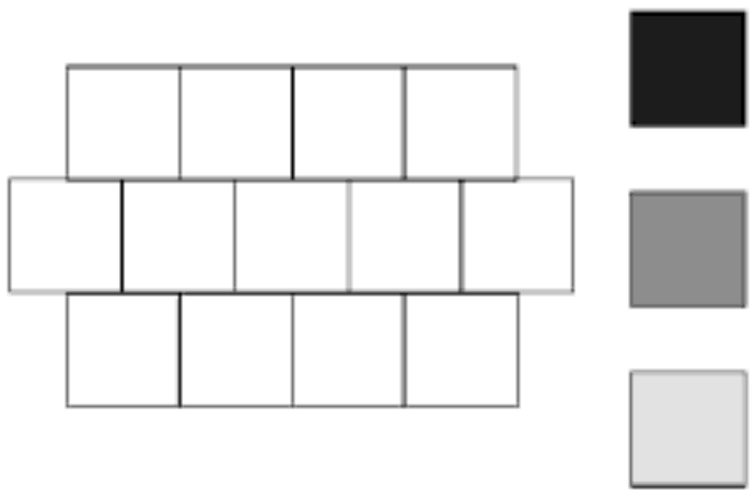


FIGURE 12.4: A sample map that we wish to colour using the three colours shown, without any two adjacent squares having the same colour.

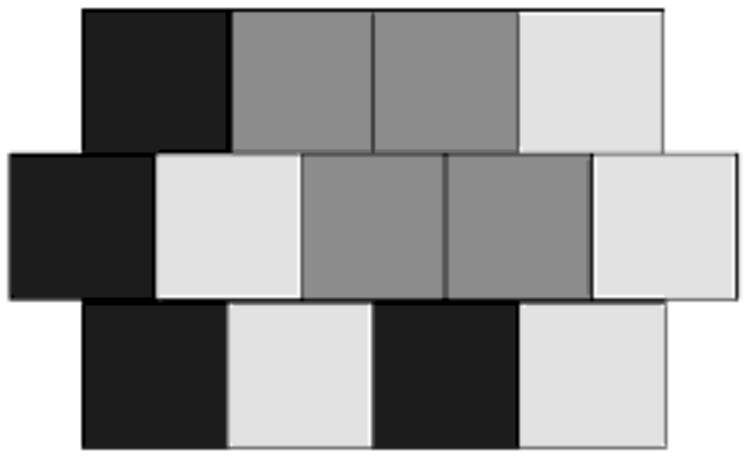


FIGURE 12.5: A possible colouring with several adjacent squares having the same colour.

choose an order to record the regions in and stick to it for all the strings, and now we can encode any way of colouring in those six regions. An example problem and a colouring are given in Figures 12.4 and 12.5.

Choose a suitable fitness function The thing that we want to minimise (a cost function) is the number of times that two adjacent regions have the same colour. We could count these up fairly simply, but it is not a fitness function, because the best solution has the lowest number, not the highest. One easy way to turn it into a fitness function would be to use the Boltzmann selection described earlier (Equation (12.2)), or to count the total number of lines between regions and subtract off the number where the two regions on either side of the line have the same colour. However, we could also just count the number of correct edges. The example in Figure 12.5 has 16 out of the 26 boundaries correct (where a boundary is the intersection between any two squares), so its fitness is 16.

Choose suitable genetic operators We'll use the standard genetic operators for this, since this example makes the operations of crossover and mutation clear. The way that they are used is shown in Figures 12.6 and 12.7. In general, people just use the standard operators for most problems, but if they don't work well, it can be worth putting some effort into thinking of new ones.

Having made those choices, we can let the GA run on the problem, with a possible population and their offspring shown in Figure 12.8, and look at the best solutions after some preset number of iterations. The GA produces good solutions to this problem, and implementing it for yourself is one of the suggested exercises for this chapter.

Copyrighted material

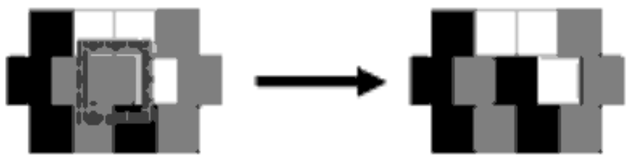


FIGURE 12.6: The way that mutation is performed on a colour, changing it into one of the other colours.

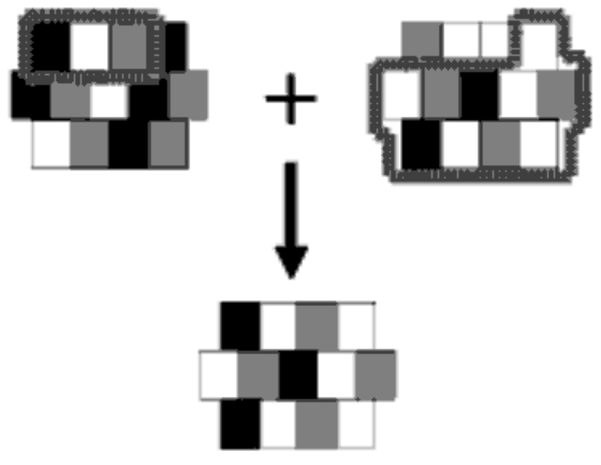


FIGURE 12.7: The effects of crossover on a map.

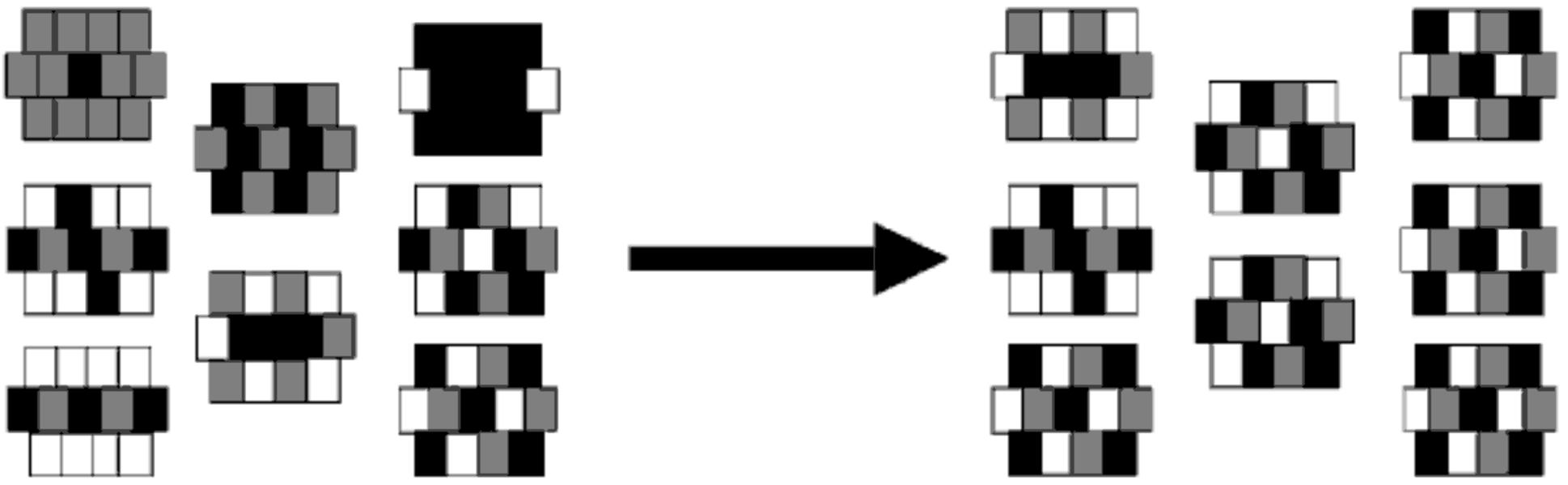


FIGURE 12.8: One generation of the GA working on the map colouring problem.

12.3.2 Punctuated Equilibrium

For a long time, one thing that creationists and others who did not believe in evolution used as an argument against it was the problem of the lack of intermediate animals in the fossil record. The argument runs that if humans evolved from apes, then there should be some evidence of a whole set of intermediary species that existed during the transition phase, and there aren't. Interestingly, GAs demonstrate one of the explanations why this is not correct, which is that the way that evolution actually seems to work is known as punctuated equilibrium. There is basically a steady population of some species for a long time, and then something changes and over a very short (in evolutionary terms... still hundreds or thousands of years) period, there is a big change, and then everything settles down again. So the chance of finding fossils from the intermediary stage is quite small. There is a graph showing this effect in Figure 12.9.

12.3.3 Example: The Knapsack Problem

We used the knapsack problem as an example while we were looking at components of the GA. It is now time to see it being solved. Before we do that, we can use some of the methods from Section 11.4 to solve it. We've

Copyrighted material

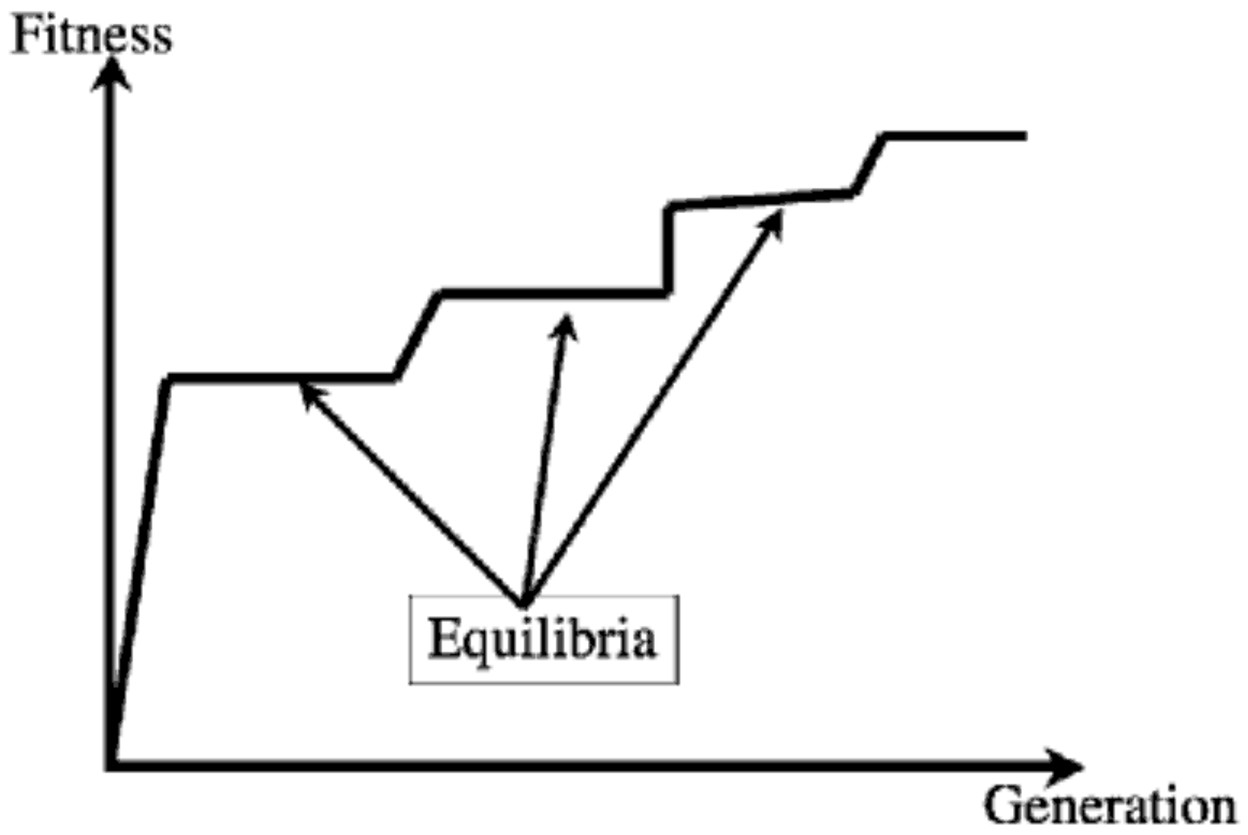


FIGURE 12.9: A graph showing punctuated equilibrium in a genetic algorithm. There is an effectively steady state where fitness does not improve, followed by rapid improvements in fitness until another steady state is reached.

already mentioned the greedy algorithm solution, and we can of course use exhaustive search, as well, or any of the other methods we discussed in the last chapter, such as simulated annealing or hillclimbing.

The website has a simple example with 20 different packages, which have a total size of 2436.77 and a maximum knapsack size of 500. The greedy algorithm finds a solution of 487.47, while the optimal solution is eventually found by the exhaustive search as 499.98. The question is how well the GA does on the same problem. We will use the fitness function that was described in Section 12.1.2, where solutions that are too large are penalised by having twice the amount they are over subtracted from the maximum size. Figure 12.10 shows a graph of the output when the GA is run on this problem for 100 iterations. The GA rapidly finds a near-optimal solution (of 499.94) to this relatively simple problem, although in this run it did not find the global optimum.

12.3.4 Example: The Four Peaks Problem

The four peaks is a toy problem (that is, simple problem that isn't useful itself, but is good for testing algorithms) that is quite often used to test out GAs and various developments of them. It is an invented fitness function that rewards strings with lots of consecutive 0s at the start of the string, and lots of consecutive 1s at the end. The fitness consists of counting the number of 0s at the start, and the number of 1s at the end and returning the maximum of them as the fitness. However, if both the number of 0s and the number of 1s are above some threshold value T then the fitness function gets a bonus of 100 added to it. This is where the name 'four peaks' comes from: there are two small peaks where there are lots of 0s, or lots of 1s, and then there

Copyrighted material

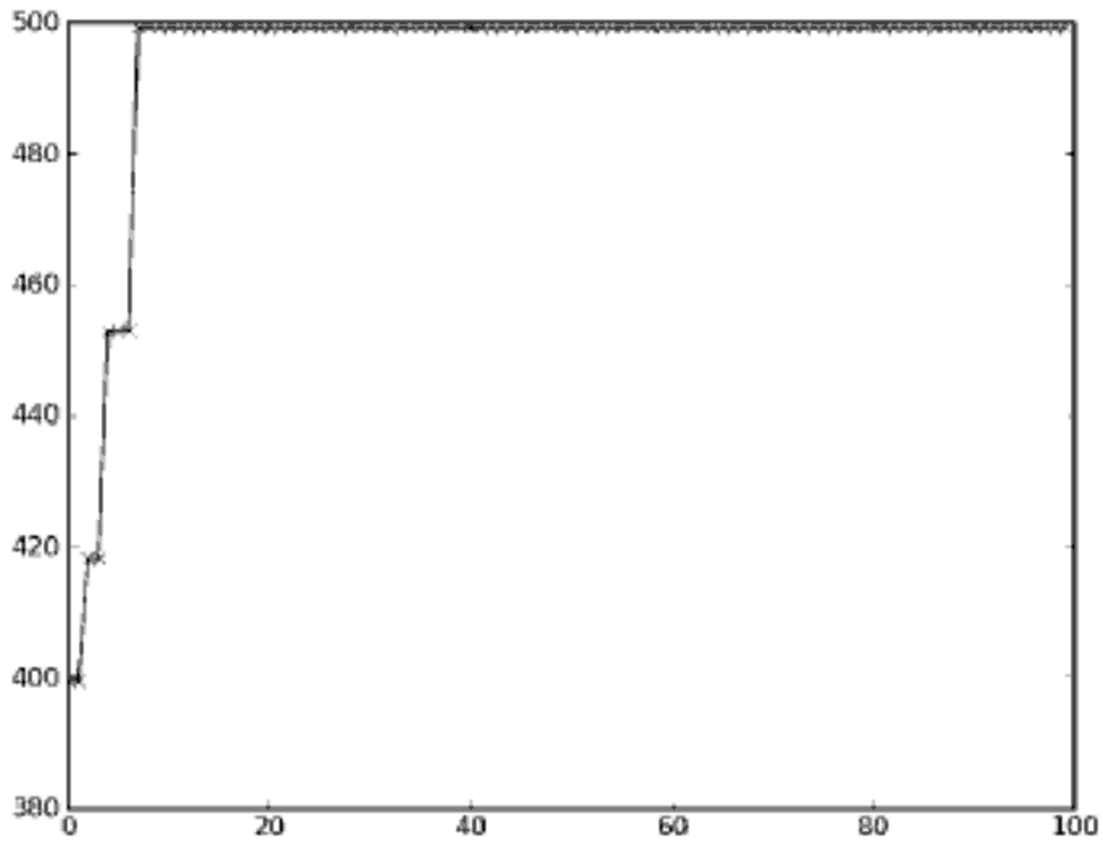


FIGURE 12.10: Evolution of the solution to the knapsack problem. The GA finds a very good solution to this simple problem within a few iterations, but never finds the optimal solution.

are two larger peaks, where the bonus is included. The GA should find these larger peaks for a successful run.

In NumPy the four peaks fitness function can be written as:

```
def fourpeaks(pop,T=10):

    start = zeros((shape(pop)[0],1))
    finish = zeros((shape(pop)[0],1))

    fitness = zeros((shape(pop)[0],1))

    for i in range(shape(pop)[0]):
        s = where(pop[i,:]==1)
        f = where(pop[i,:]==0)
        if size(s)>0:
            start = s[0][0]
        else:
            start = 0

    if size(f)>0:
        finish = shape(pop)[1] - f[-1][-1] -1
    else:
        finish = 0

    if start>T and finish>T:
        fitness[i] = maximum(start,finish)+100
```

Copyrighted material

Hidden page

guarantee that the algorithm will converge at all, and certainly not to the optimal solution. This bothers a lot of researchers. That said, genetic algorithms are widely used when other methods do not work, and they are usually treated as a black box—strings are pushed in one end, and eventually an answer emerges. This is risky, because without knowledge of how the algorithm works it is not possible to improve it, nor do you know how cautiously you should treat the results.

12.3.6 Training Neural Networks with Genetic Algorithms

We trained our neural networks, most notably the MLP, using gradient descent. However, we could encode the problem of finding the correct weights as a set of strings, with the fitness function measuring the sum-of-squares error. This has been done, and with good reported results. However, there are some problems with this approach. The first is that we turn all the local information from the targets about the error at each output node of the network into just one number, the fitness, which is throwing away useful information, and the second is that we are ignoring the gradient information, which is also throwing away useful information.

A more sensible use for GAs with neural networks is to use the GA to choose the topology of the network. Previously, we chose the structure in a completely ad hoc way by trying out different structures and choosing the one that worked best. We can use a GA for this problem, although the crossover operator doesn't make a great deal of sense, so we just consider mutation. However, we allow for four different types of mutation: delete a neuron, delete a weight connection, add a neuron, add a connection. The deletion operators bias the learning towards simple networks. Making the GA more complicated by adding extra mutation operators might make you wonder if you can make it more complicated again. And you can; one example of where this can lead is discussed next.

12.4 Genetic Programming

One extension of genetic algorithms that has had a lot of attention is the idea of **genetic programming**. This was introduced by John Koza, and the basic idea is to represent a computer program as a tree (imagine a flow chart of the code). For certain programming languages, notably LISP, this is actually a very natural way to represent a program, but it doesn't work very well in Python, so we will have a quick look at the idea, but not get into writing any explicit algorithms for the method.

Tree-based variants on mutation and crossover are defined (replace sub-

Copyrighted material

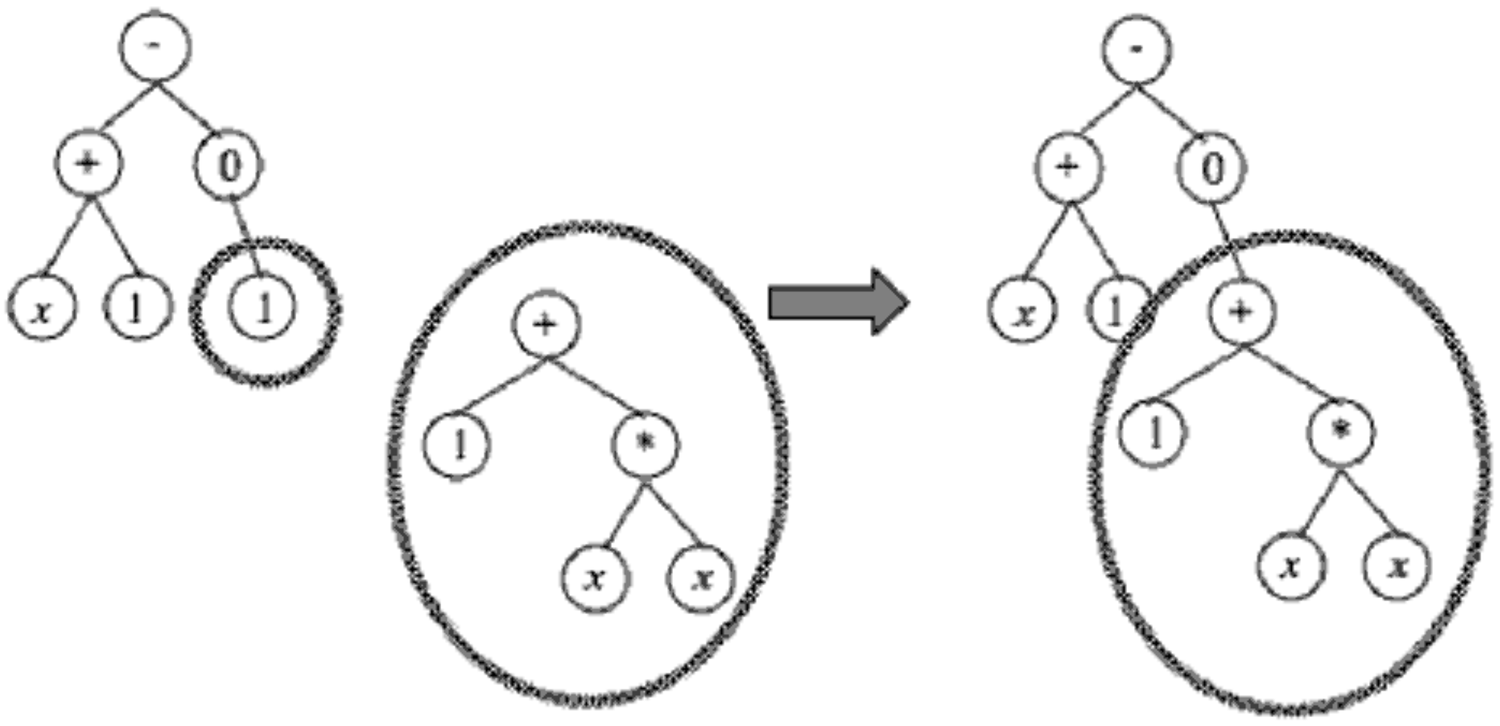


FIGURE 12.13: Example of a mutation in genetic programming.

trees by other subtrees, either randomly generated (mutation, Figure 12.13) or swapped from another tree (crossover, Figure 12.14)), and then the genetic program runs just like a normal genetic algorithm, but acting on these program trees rather than strings. Figure 12.15 shows a set of simple trees that perform arithmetic operations, and some possible developments of them, made using these operators.

Genetic programming has been used for many different tasks, from recognising skin melanomas to circuit design, and lots of very impressive results have been claimed for it. However, the search space is unbelievably large, and the mutation operator not especially useful, and so a lot depends upon the initial population. A set of possibly useful subtrees are usually chosen by the system developer first in order to give the system a head start. There are a couple of places where you can find more information on genetic programming in the Further Reading section.

12.5 Combining Sampling with Evolutionary Learning

The last machine learning method in this chapter is an interesting variation on the theme of evolutionary learning, combined with probabilistic models of the type that are described in Chapter 15, namely Bayesian networks. They are often known as estimation of distribution algorithms (EDA).

The most basic version is known as Population-Based Incremental Learning (PBIL), and it is amazingly simple. It works on a binary alphabet, just like the basic GA, but instead of maintaining a population, instead it keeps a probability vector p that gives the probability of each element being a 0 or 1. Initially, each value of this vector is 0.5, so that each element has equal

Copyrighted material

Hidden page

chance of being 0 or 1. A population is then constructed by sampling from the distribution specified vector, and the fitness of each member of the population is computed. A subset of this population (typically just the two fittest vectors) is chosen to update the probability vector, using a learning rate η, which is often set to 0.005 (where `best` and `second` represent the best and second-best elements of the population):

$$p = p \times (1 - \eta) + \eta(\text{best} + \text{second})/2. \tag{12.3}$$

The population is then thrown away, and a new one sampled from the updated probability vector. The results of using this simple algorithm on the four-peaks problem with $T = 11$ are shown in Figure 12.16 using strings of length 100 with 200 strings in each population. This is directly comparable with Figure 12.12.

The centre of the algorithm is simply the code to find the strings with the two highest fitnesses and use them to update the vector. Everything else is directly equivalent to the genetic algorithm.

```
# Pick best
best[count] = max(fitness)
bestplace = argmax(fitness)
fitness[bestplace] = 0
secondplace = argmax(fitness)

# Update vector
p  = p*(1-eta) + eta*((pop[bestplace,:]+pop[secondplace,:])/2)
```

The probabilistic model that is used in PBIL is very simple: it is assumed that each element of the probability vector is independent, so that there is no interaction. However, there is no reason why more complicated interactions between variables cannot be considered, and several methods have been developed that do exactly this. The first option is to construct a chain, so that each variable depends only on the one to its left. This might involve sorting the order of the probability vector first, but then the algorithm simply needs to measure the mutual information (see Section 6.2.1) between each pair of neighbouring variables. This use of mutual information gives the algorithm its name: MIMIC. There are also more complicated variants using full Bayesian networks, such as the Bayesian Optimisation Algorithm (BOA) and Factorised Distribution Algorithm (FDA).

The power of these developments of the GA is that they use probabilistic models and are therefore more amenable to analysis than normal GAs, which have steadfastly withstood many attempts to better understand their behaviour. They also enable the algorithm to discover correlations between input variables, which can be useful if you want to understand the solution rather than just apply it.

Copyrighted material

Hidden page

- D.E. Goldberg. *Genetic Algorithms in Search, Optimisation, and Machine Learning.* Addison-Wesley, Reading, MA, USA, 1999.

There are also entire books on genetic programming, including:

- J.R. Koza. *Genetic Programming: On the Programming of Computers by the Means of Natural Selection.* MIT Press, Cambridge, MA, USA, 1992.

- Z. Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs.* Springer, Berlin, Germany, third edition, 1999.

For more on Estimation of Distribution algorithms, look at:

- S. Baluja and R. Caruana. Removing the genetics from the standard genetic algorithm. In A. Prieditis and S. Russel, editors, *The International Conference on Machine Learning*, pages 38–46, San Mateo, CA, USA, 1995. Morgan Kaufmann Publishers.

- M. Pelikan, D.E. Goldberg, and F. Lobo. A survey of optimization by building and using probabilistic models. *Computational Optimization and Applications*, 21(1):5–20, 2002. Also IlliGAL Report No. 99018.

Details of the two books mentioned about real evolution are:

- C. Darwin. *On the Origin of Species By Means of Natural Selection.* Wordsworth, London, UK, 6th edition, 1872.

- R. Dawkins. *The Blind Watchmaker: Why the Evidence of Evolution Reveals a Universe Without Design.* Penguin, London, UK, 1996.

Practice Questions

Problem 12.1 Suppose that you want to archive your data files, but you have only got one CD, and more data files than will fit on it. You decide to choose the files you will save so as to try to maximise the amount of space you fill on the disk, so that the most data is backed up, but you can't split a data file. Write a greedy algorithm and a hill-climbing algorithm to solve this problem. What guarantees can you make about efficiency of the solutions?

Problem 12.2 (from Jon Shapiro)
In video poker, you are dealt five cards face up. You have one chance to replace any of the cards (or all or none) with cards drawn from the

Copyrighted material

Hidden page

is the sum of the fitness for each block. Implement this fitness function and test it on strings of length 16, with blocks of lengths 1, 2, 4, 8. Run your GAs for 10,000 iterations. Compare the results to using PBIL.

Copyrighted material

Hidden page

> those which are accompanied or closely followed by discomfort to the animal will, other things being equal, have their connections with that situation weakened, so that, when it recurs, they will be less likely to occur. The greater the satisfaction or discomfort, the greater the strengthening or weakening of the bond. (E. L. Thorndike, "Animal Intelligence," page 244.)

This is where the name 'reinforcement learning' comes from, since you repeat actions that are reinforced by a feeling of satisfaction. To see how it can be applied to machine learning, we will need some more notation.

13.1 Overview

Reinforcement learning maps states or situations to actions in order to maximise some numerical reward. That is, the algorithm knows about the current input (the state), and the possible things it can do (the actions), and its aim is to maximise the reward. There is a clear distinction drawn between the agent that is doing the learning and the environment, which is where the agent acts, and which produces the state and the rewards. The most common way to think about reinforcement learning is on a robot. The current sensor readings of the robot, or processed versions of them, could define the state. They are a representation of the environment around the robot in some way. Note that the state doesn't necessarily tell us everything that it would be useful to know (the robot's sensors don't tell it its location, only what it can see about it), and there can be noise and inaccuracies in the state data. The possible ways that the robot can drive its motors are the actions, which move the robot in the environment, and the reward could be how well it does its task without crashing into things. Figure 13.1 shows the idea of state, actions, and environment to a robot, while Figure 13.2 shows how they are linked, together with the reward.

In reinforcement learning the algorithm gets feedback in the form of the reward about how well it is doing. In contrast to supervised learning, where the algorithm is 'taught' the correct answer, the reward function evaluates the current solution, but does not suggest how to improve it. Just to make the situation a little more difficult, we need to think about the possibility that the reward can be delayed, which means that you don't actually get the reward until a long time in the future (for example, think about a robot that is learning to traverse a maze. It doesn't know whether it has found the centre of the maze until it gets there, and it doesn't get the reward until it reaches the centre of the maze.) We therefore need to allow for rewards that don't appear until long after the relevant actions have been taken. Sometimes we think of the immediate reward and the total expected reward into the future.

Copyrighted material

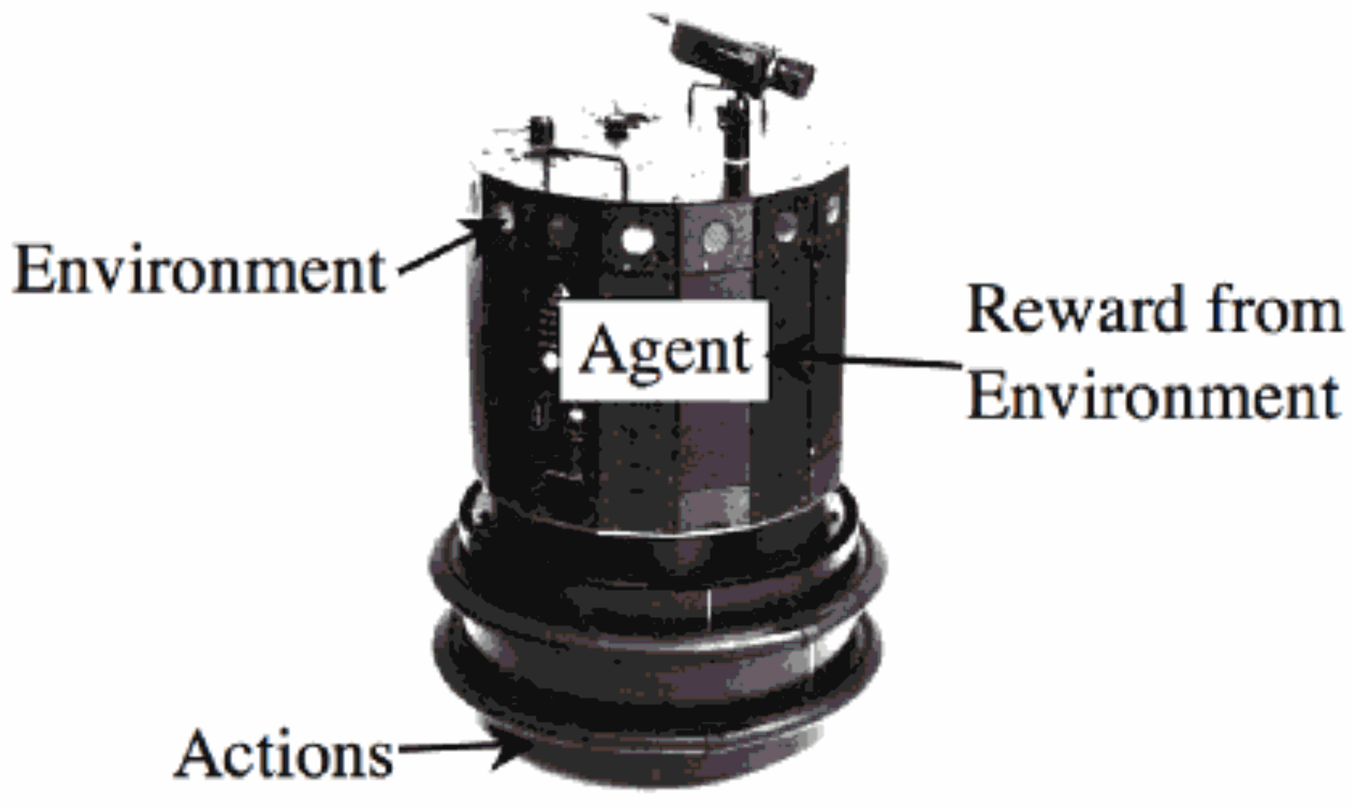


FIGURE 13.1: A robot perceives the current state of its environment through its sensors, and performs actions by moving its motors. The reinforcement learner (agent) within the robot tries to predict the next state and reward.

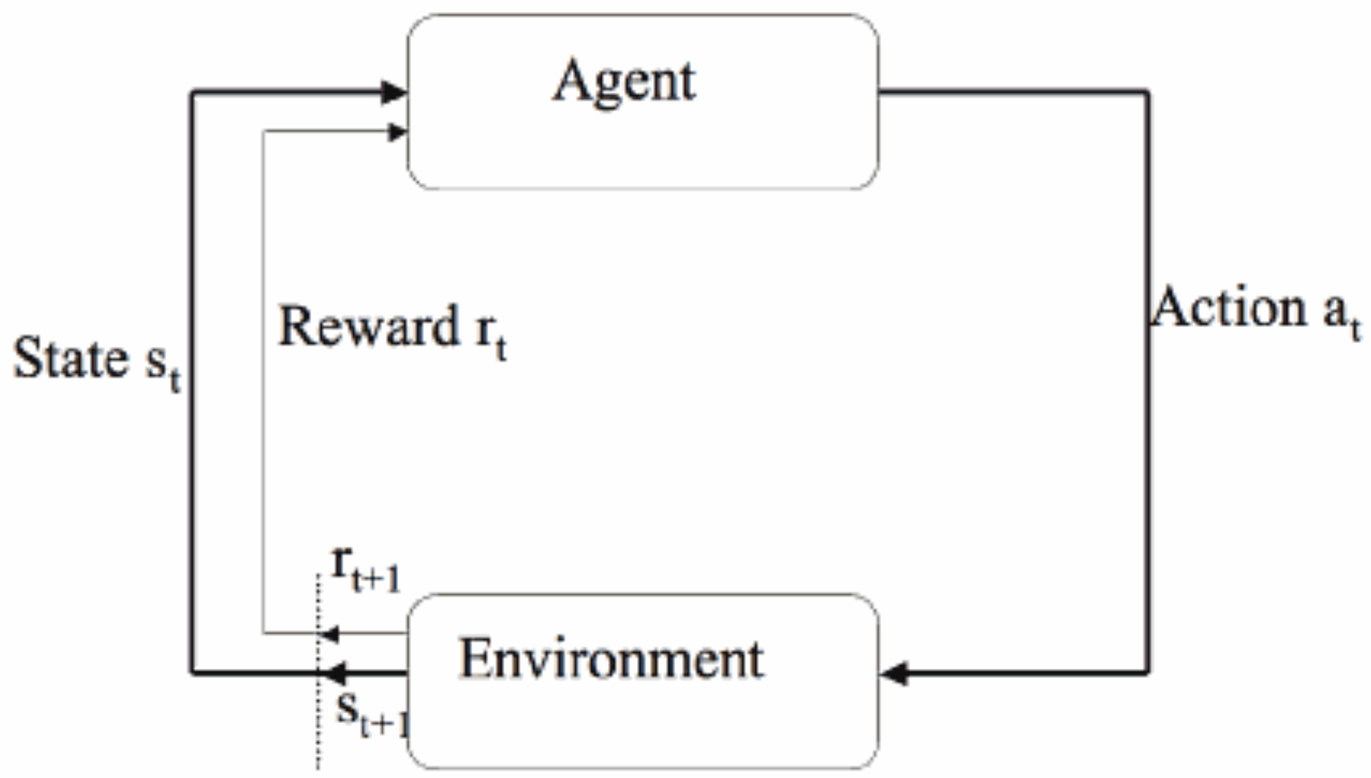


FIGURE 13.2: The reinforcement learning cycle: the learning agent performs action a_t in state s_t and receives reward r_{t+1} from the environment, ending up in state s_{t+1}.

Copyrighted material

Hidden page

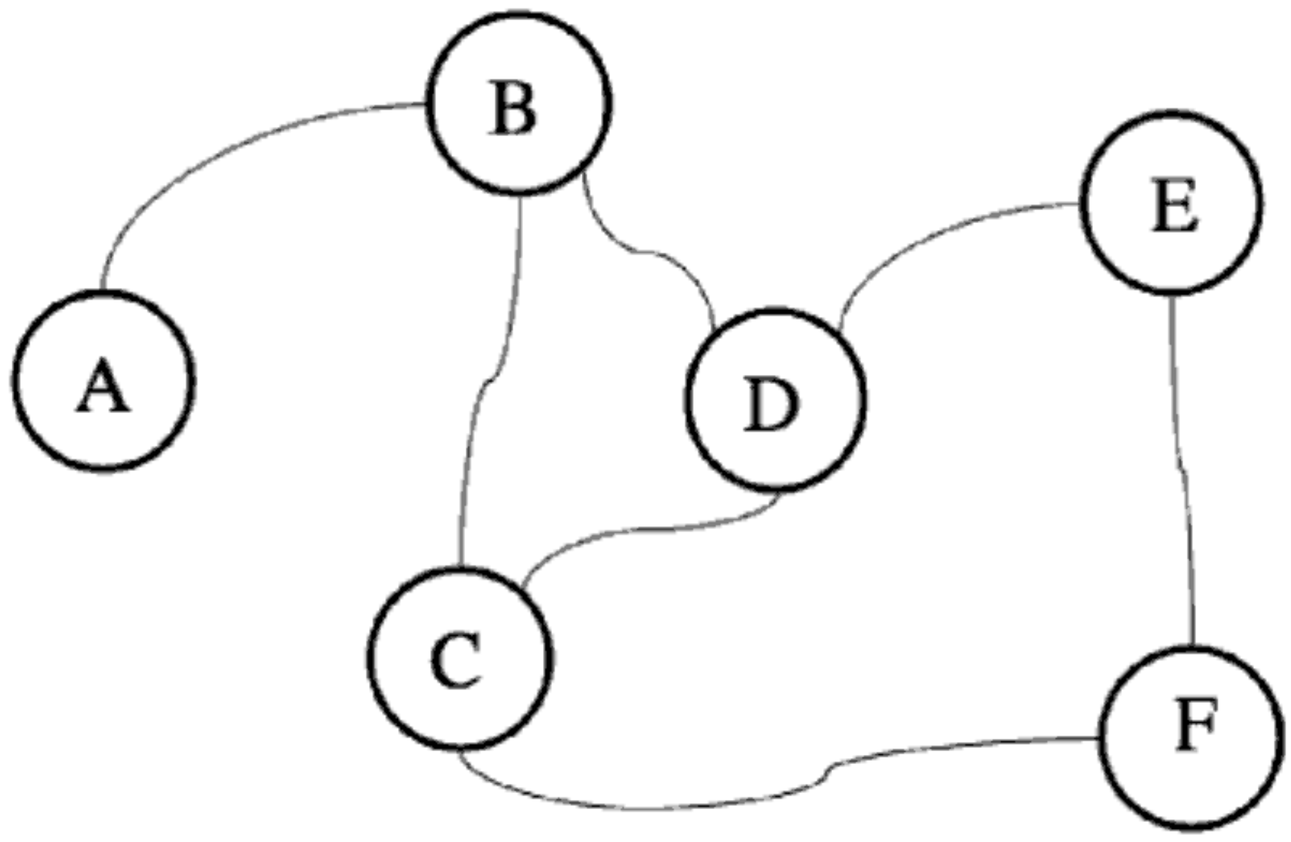


FIGURE 13.3: The old town that you find yourself lost in.

ment can be viewed as punishment, it doesn't necessarily correspond clearly, but you might want to imagine it as pinching yourself so that you stay awake). Of course, once you reach state F you are in the backpacker's and will therefore stay there. This is known as an absorbing state, and is the end of the problem, when you get the reward of eating all the chips you bought. Now moving between two squares could be good, because it might take you closer to F. But without looking at the map you won't know that, so you decide to just apply a reward when you actually reach F, and leave everything else as neutral. Where there is no direct road between two squares (so that no action takes you from one to the other) there is no reward because it is not a viable action. This results in the reward matrix R shown below (where '-' shows that there is no link) and also in Figure 13.4.

	Next State					
Current State	A	B	C	D	E	F
A	-5	0	-	-	-	-
B	0	-5	0	0	-	-
C	-	0	-5	0	-	100
D	-	0	0	-5	0	-
E	-	-	-	0	-5	100
F	-	-	0	-	0	-

Of course, as a reinforcement learner you don't actually know the reward matrix. That's pretty much what you are trying to discover, but it doesn't make for a very good example. We'll assume that you have now reached a stage of tiredness where you can't even read what is on your paper properly. Having got this set up we've reached the stage where we need to do some learning, but for now we'll leave you stranded in that foreign city and flesh out a few of the things that we've talked about so far.

Copyrighted material

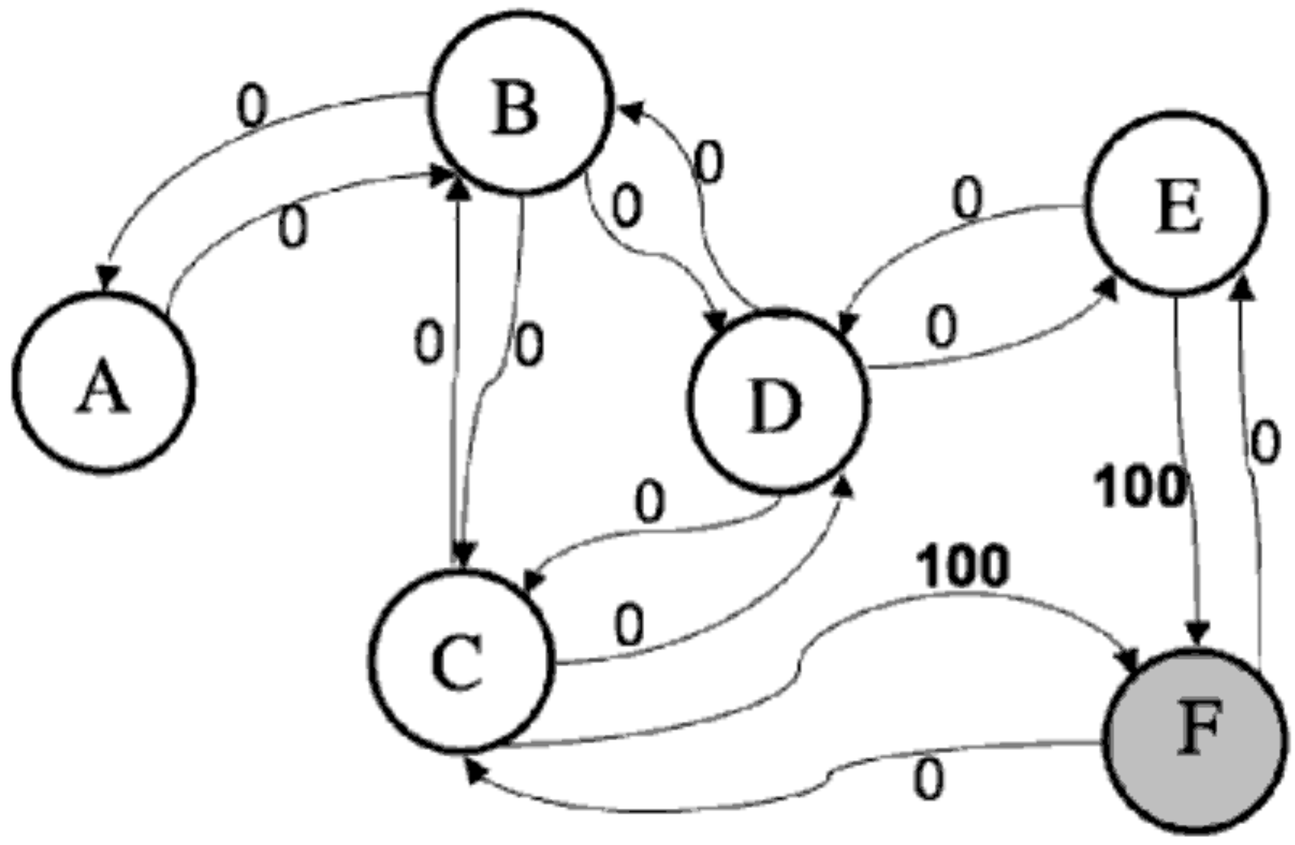


FIGURE 13.4: The *state diagram* if you are correct and the backpacker's is in square (state) F. The connections from each state back into itself (meaning that you don't move) are not shown, to avoid the figure getting too complicated. They are each worth -5 (except for staying in state F, which means that you are in the backpacker's).

13.2.1 State and Action Spaces

Our reinforcement learner is basically a search algorithm, and obviously the larger the number of states that the algorithm has to search through, the longer it will take to find a good solution. The set of all states that are possible for the learner to experience is known as the **state space**. There is a corresponding **action space** that contains all of the possible actions. If we can reduce the size of the state space and action space, then it is almost always a good idea, providing that it does not oversimplify the problem. In the example there are only six states, but still, look at Figure 13.4 and imagine wandering through all of the squares over and over again while we search: it seems like this learning is going to take a long time. And it generally does.

Computing the size of the state space (and the corresponding action space) is relatively simple. For example, suppose that there are five inputs, each an integer between 0 and 100. The size of the state space is then $100 \times 100 \times 100 \times 100 \times 100 = 100^5$, which is incredibly large, so the curse of dimensionality is really kicking in here. However, if we decide that we can **quantize** the data so that instead of 100 numbers there are only two for each input (for example, by assigning every number less than 50 to class 1, and every number 50 and above to class 2), then the size of the state space is a more manageable $2^5 = 32$. Choosing the state space and action space carefully is therefore a crucial part of making a successful reinforcement learner. You want them to be as small as possible without losing accuracy in the results—by reducing the scale of each input from 100 to 2, we have obviously thrown away a lot of information that might have made the quality of the answer better. As is usually the case, there is some element of compromise between the two.

Copyrighted material

13.2.2 Carrots and Sticks: the Reward Function

The basic idea of the learner is that it will choose the action that gets the maximum expected reward. In the example, we worked out what the rewards would be in a fairly ad hoc way, by saying what we wanted and then thinking about how to get it. That's pretty much the way that it works in practice, too: in Chapter 12 where we looked at genetic algorithms, we had to carefully craft the fitness function to solve the problem that we wanted, and the same thing is true of the reward function. In fact, they can be seen as the same thing.

The reward function takes the current state and the chosen action and produces a numerical reward based on them. So in the example, if we are in state A, and choose the action of doing nothing, so that we remain in state A, we get a reward of -5. Note that the reward can be positive or negative, with the latter corresponding to 'punishment,' showing that particular actions should be avoided. The reward is generated by the environment around the learner, it is not internal to the learner itself (this is what makes it difficult to describe in our example: the environment doesn't give you rewards in the real world, only when there is a computer (or brain) as part of the environment to help out). In effect, the reward function makes the goal of the learner explicit—the learner is trying to maximise the reward, which means behaving in exactly the way that the reward function expects. The reward tells the learner what the goal is, not how the goal should be achieved, which would be supervised learning. It is therefore usually a bad idea to include sub-goals (extra things that the learner should achieve along the way, which are meant to speed up learning), because the learner can find methods of achieving the sub-goals without actually achieving the real goal.

The choice of a suitable reward function is absolutely crucial, with very different behaviours resulting from varying the reward function. For example, consider the difference between these two reward functions for a maze-traversing robot (try to work out the difference before reading the paragraph that follows them):

- receive a reward of 50 when you find the centre of the maze
- receive a reward of -1 for each move and a reward of +50 when you find the centre of the maze

In the first version, the robot will learn to get to the centre of the maze, just as it will in the second version, but the second reward function is biased towards shorter routes through the maze, which is probably a good thing. The maze problem is episodic: learning is split into episodes that have a definite endpoint when the robot reaches the centre of the maze. This means that the rewards can be given at the end and then propagated back through all the actions that were performed to update the learner. However, there are plenty of other examples that are not episodic (continual tasks), and there is no cut

Copyrighted material

off when the task stops. An example is the child learning to walk that was mentioned at the start of the chapter. A child can walk successfully when it doesn't fall over at all, not when it doesn't fall over for 10 minutes.

Now that the reward has been broken into two parts—an immediate part and a pay-off in the end—we need to think about the learning algorithm a bit more. The thing that is driving the learning is the total reward, which is the expected reward from now until the end of the task (when the learner reaches the **terminal state** or **accepting state**—the backpacker's in our example). At that point there is generally a large pay-off that signals the successful completion of the task. However, the same thing does not work for continual tasks, because there is no terminal state, so we want to predict the reward forever into the infinite future, which is clearly impossible.

13.2.3 Discounting

The solution to this problem is known as **discounting**, and means that we take into account how certain we can be about things that happen in the future: there is lots of uncertainty in the learning anyway, so we should discount our predictions of rewards in the future according to how much chance there is that they are wrong. The rewards that we expect to get very soon are probably going to be more accurate predictions than those a long time in the future, because lots of other things might change. So we add an additional parameter $0 \leq \gamma \leq 1$, and then discount future rewards by multiplying them by γ^t, where t is the number of timesteps in the future this reward is from. As γ is less than 1, so γ^2 is smaller again, and $\gamma^k \to 0$ as $k \to \infty$ (i.e., γ gets smaller and smaller as k gets larger and larger), so that we can ignore most of the future predictions. This means that our prediction of the total future reward is:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots + \gamma^{k-1} r_k + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (13.1)$$

Obviously, the closer γ is to zero, the less distance we look into the future, while with $\gamma = 1$ there is no discounting, as in the episodic case above (in fact, discounting is sometimes used for episodic learning as well, since the eventual reward could be a very long way off and we have to deal with that uncertainty in learning somehow). We can apply discounting to the example of learning to walk. When you fall over you give yourself a reward of -1, and otherwise there are no rewards. The -1 reward is discounted into the future, so that a reward k steps into the future has reward $-\gamma^k$. The learner will therefore try to make k as large as possible, resulting in proper walking.

The point of the reward function is that it gives us a way to choose what to do next—our **predictions** of the reward let us exploit our current knowledge and try to maximise the reward we get. Alternatively, we can carry on exploring

Copyrighted material

Hidden page

13.2.5 Policy

We have just considered different action selection methods, such as ϵ-greedy and soft-max. The aim of the action selection is to trade off exploration and exploitation in such a way as to maximise the expected reward into the future. Instead, we can make an explicit decision that we are going to always take the optimal choice at each stage, and not do exploration any more. This choice of which action to take in each state in order to get optimal results is known as the policy, π. The hope is that we can learn a better policy that is specific to the current state s_t. This is the crux of the learning part of reinforcement learning—learn a policy π from states to actions. There is at least one optimal policy that gives the maximum reward, and that is what we want to find. In order to find a policy, there are a few things that we need to worry about. The first is how much information we need to know regarding how we got to the current state, and the second is how we ascribe a value to the current state. The first one is important enough both for here and for Chapter 15 that we are going to go into some detail now.

13.3 Markov Decision Processes

13.3.1 The Markov Property

Let's go back to the example. Standing in the square labelled D you need to make a choice of what action to take next. There are four possible options (see Figure 13.4): standing still, or moving to one of B, C, or E. The question is whether or not that is enough information for you to predict the reward accurately and so to choose the best possible action. Or do you also need to know where you have been in the past? Let's say that you know that you came to D from B. In that case, maybe it does not make sense to move back to B, since your reward won't change. However, if you came to D from E then it does actually make sense to go back there, since it moves you closer to F. So in this case, it appears that knowing your previous action doesn't actually help very much, because you don't have enough information to work out what was useful.

Another example where this is usually true is in a game of chess, where the current situation of all the pieces on the board (the state) is enough to predict whether or not the next move is a good one—it does not depend on precisely how each piece got to the current location. Thus, the current state provides enough information. A state that has this property, which is that the current state provides enough information for the reward to be computed without looking at previous states, is known as a Markov state. The importance of this can be seen from the following two equations, the first of which is what

Copyrighted material

Hidden page

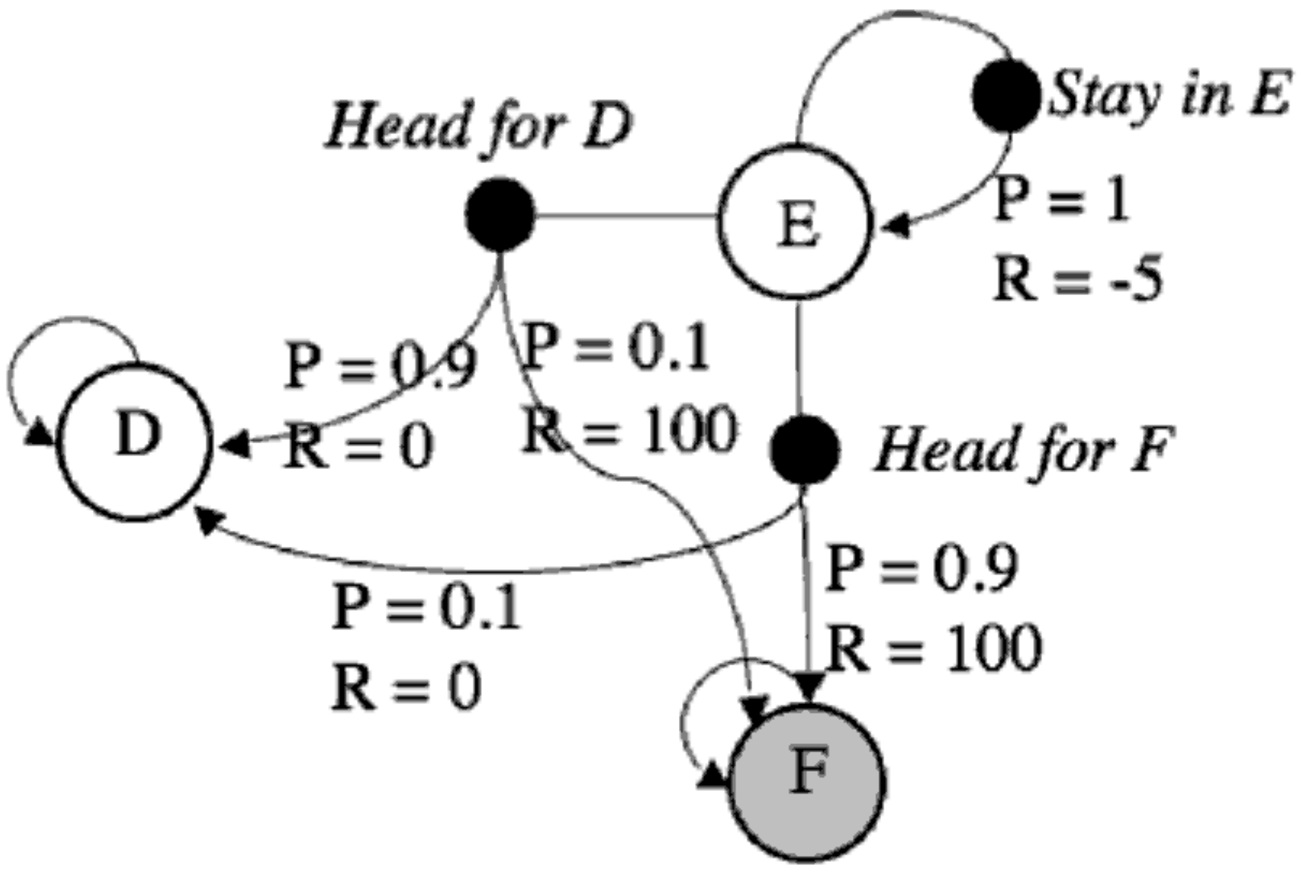


FIGURE 13.6: A small part of the transition diagram for the example. From state E there are three possible actions, and the states in which they end up, together with the rewards, are shown here.

of a finite Markov Decision Process and usually includes information about the rewards.

We can make a transition diagram for our example, based on Figure 13.4. We'll make the situation a little bit more complicated now by adding in the assumption that you are so tired that even though you are in state B and trying to get to state A, there is a small probability that you will actually take the wrong street and end up in either C or D. We'll make those probabilities be 0.1 for each extra exit that there is from each state, and we'll assume that you can stand in one place without fear of ending up elsewhere. A very tiny bit of the transition diagram, centred on state E, is shown in Figure 13.6. There are three actions that can be taken in state E (shown by the black circles), with associated probabilities and expected rewards. Learning and using this transition diagram can be seen as the aim of any reinforcement learner.

The Markov Decision Process formalism is a powerful one that can deal with additional uncertainties. For example, it can be extended to deal with the case where the true states are not known, only an observation of the state can be made, which is probabilistically related to the state, and possibly the action. These are known as partially observable Markov Decision Processes (POMDPs), and they are related to the Hidden Markov Models that we will see in Section 15.3. POMDPs are commonly used for robotics, where the sensors of the robots are usually far too inexact and noisy for places the robot visits to be identified with any degree of success. Methods to deal with these problems maintain an estimate of belief of their current state and use that in the reinforcement learning calculations. It is now time to get back to the reinforcement learner and the concept of values.

Copyrighted material

13.4 Values

The reinforcement learner is trying to decide on what action to take in order to maximise the expected reward into the future. This expected reward is known as the value. There are two ways that we can compute a value. We can consider the current state, and average across all of the actions that can be taken, leaving the policy to sort this out for itself (the state-value function, $V(s)$), or we can consider the current state and each possible action that can be taken separately, the action-value function, $Q(s, a)$. In either case we are thinking about what the expected reward would be if we started in state s (where $E(\cdot)$ is the statistical expectation):

$$V(s) = E(r_t|s_t = s) = E\left\{\sum_{i=0}^{\infty} \gamma^i r_{t+i+1}|s_t = s\right\}, \tag{13.5}$$

$$Q(s, a) = E(r_t|s_t = s, a_t = a) = E\left\{\sum_{i=0}^{\infty} \gamma^i r_{t+i+1}|s_t = s, a_t = a\right\}. \tag{13.6}$$

It should be fairly obvious that the second estimate is more accurate in the long run, because we have more information: we know which action we are going to take. However, because of that we need to collect a lot more data, and so it will take a long time to learn. In other words, the action-value function is even more susceptible to the curse of dimensionality than the state-value function. In situations where there are lots of states it will not be possible to store either, and some other method, such as using a parameterised solution space (i.e., having a set of parameters that are controlled by the learner, rather than explicit solutions), will be needed. This is more complicated than we will consider here.

There are now two problems that we need to solve, predicting the value function, and then selecting the optimal policy. We'll think about the second one first. The optimal policy is the one in which the value function is the greatest over all possible states. We label this (not necessarily unique) policy with a star: π^*. The optimal state-value function is then $V^*(s) = \max_\pi V^\pi(s)$ for all possible states s, and the optimal action-value function is $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ for all possible states s and actions a. We can link these two value functions, because the first considers taking the optimal action in each case (since the policy π^* is optimal), while the second considers taking action a this time, and then following the optimal policy from then on. Hence we only need to worry about the current reward and the (discounted) estimate of the future rewards:

Copyrighted material

$$Q^*(s,a) = E(r_{t+1}) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$$
$$= E(r_{t+1}) + \gamma V^*(s_{t+1}|s_t = s, a_t = a). \quad (13.7)$$

Of course, there is no guarantee that we will ever manage to learn the optimal policy. There is bound to be noise and other inaccuracies in the system, and the curse of dimensionality is liable to limit the amount of exploration that we do. However, it will be enough to learn good approximations to the optimal policy. One thing that will work to our advantage is that reinforcement learning operates on-line, in that the learner is exploring the different states as it learns, which means that it is getting more chances to learn about the states that are seen more often, and so will have a better chance of finding the optimal policy for those states.

The question is how you actually update the value function ($V(s)$ or $Q(s,a)$). The idea is to make a look-up table of all the possible states or state-action pairs, and set them all to zero to start with. Then we will use experience to fill them in. Returning to your foreign trip, you wander around until eventually you stumble upon the backpacker's. Gorging yourself on the chips you remember that at the last timestep you were in square E (this remembering is known as a backup). Now you can update the value for E (the reward is $\gamma \times 100$). That is all we do, since your memory is so shot that you can't remember anything else. And there we stop until the next night, when you wake up again and the same thing happens. Except now, you have information about E, although not about any other state. However, when you reach E now, say from D, then you can update the value for D to have reward $\gamma^2 \times 100$. And so it continues, until all of the states (and possibly actions) have values attached.

The obvious problem with this approach is that we have to wait until we reach the goal before we can update the values. Instead, we can use the same trick that we used in Equation (13.7) and use the current reward and the discounted prediction instead, so that the update equation looks like (where μ is the learning rate as usual):

$$V(s_t) \leftarrow V(s_t) + \mu(r_{t+1} + \gamma V(s_{t+1}) - V(s_t)). \quad (13.8)$$

The $Q(s,a)$ version looks very similar, except that we have to include the action information. In both cases we are using the difference between the current and previous estimates, which is why these methods have the name of temporal difference (TD) methods. Suppose that we knew rather more about where we had been. In that case, we could have updated more states when we got to the backpacker's, which would have been rather more efficient. The trouble is that we don't know if those states were useful or not—it might have been chance that we visited them. The answer to this is similar to discounting: we introduce another parameter $0 \leq \lambda \leq 1$ that we apply to

Copyrighted material

Hidden page

Hidden page

Hidden page

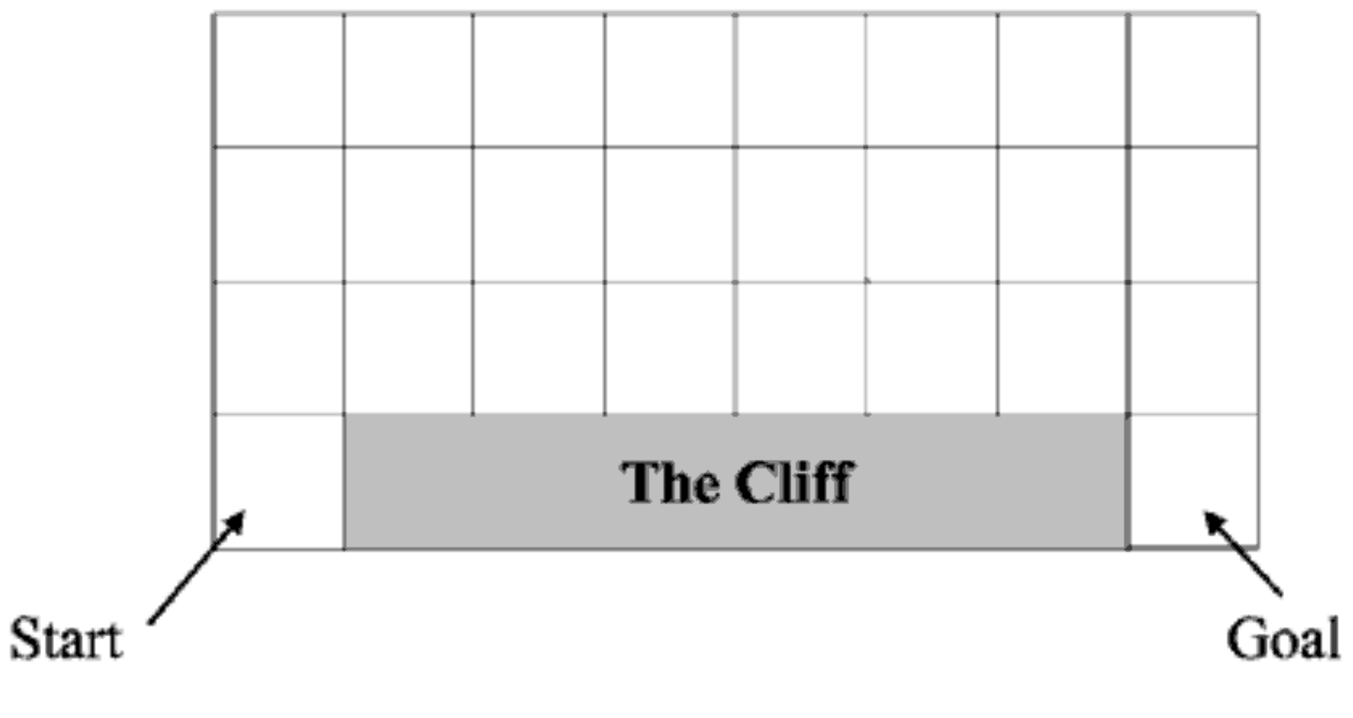


FIGURE 13.7: The example environment.

13.6 The Difference between Sarsa and Q-Learning

It might not be clear what the difference is between the two algorithms in practice. We're going to consider the little environment that is shown in Figure 13.7, where the agent has to learn a route from the start location on the left to the final location on the right (the example comes from Section 6.5 of Sutton and Barto's book, which is in the readings at the end of the chapter). The reward structure is that every move gets a reward of -1, except for moves that end up on the cliff. These get a reward of -100, and the agent gets put back at the start location. This is clearly an episodic problem, since there is a clear end state.

Both algorithms will start out with no information about the environment, and will therefore explore randomly, using the ϵ-greedy policy. However, over time, the strategies that the two algorithms produce are quite different. The main reason for the difference is that Q-learning always attempts to follow the optimal path, which is the shortest one. This takes it close to the cliff, and the ϵ-greedy part means that inevitably it will sometimes fall over. By way of contrast, the sarsa algorithm will converge to a much safer route that keeps it well away from the cliff, even though it takes longer. The two solutions are shown in Figures 13.8 and 13.9. The sarsa algorithm produces the safe route because it includes information about action selection in its estimates of Q, while Q-learning produces the riskier, but shorter, route. The choice of which is better is up to you, and it depends on how serious the effects of falling off the cliff are.

The reason for the difference between the algorithms is because Q-learning always assumes that the policy will pick the optimal action, and while this is true most of the time, the ϵ-greedy policy does occasionally choose a different action, which can cause problems here. However, the algorithm ignores these dangers because it only focuses on the optimal solution. Sarsa does not take this maximum, and so it will be biased against solutions that take it close to

Copyrighted material

Hidden page

A famous example of reinforcement learning was TD-Gammon, which was produced by Gerald Tesauro. His idea was that reinforcement learning should be very good at learning to play games, because games were clearly episodic—you played until somebody won—and there was a clear reward structure, with a positive reward for winning. There was another benefit, which was that you could set the learner to play against itself. This is actually very important, since the version of TD-Gammon that was actually bundled with the IBM operating system OS/2 Warp had played 1,500,000 games against itself before it stopped improving.

Further Reading

A detailed book on reinforcement learning is:

- R.S. Sutton and A.G. Barto. *Reinforcement Learning: An Introduction.* MIT Press, Cambridge, MA, USA, 1998.

An interesting article concerning the use of reinforcement learning is:

- G. Tesauro. Temporal difference learning and TD-gammon. *Communications of the ACM*, 38(3):58–68, 1995.

Alternative treatments are:

- Chapter 13 of T. Mitchell. *Machine Learning.* McGraw-Hill, New York, USA, 1997.
- Chapter 16 of E. Alpaydin. *Introduction to Machine Learning.* MIT Press, Cambridge, MA, USA, 2004.

Practice Questions

Problem 13.1 Work through the first few steps of the hill by hand for both sarsa and Q-learning. Then modify the code to run on this example and ensure that they match.

Problem 13.2 Design a Q-learner for playing noughts-and-crosses (also known as Tic-Tac-Toe). Run the algorithm by hand, describing the states, transitions, rewards, and Q-values. Assume that the opponent picks a random (but valid) square for each move. How would your learner change if the opponent played optimally? Would a TD learner behave differently?

Copyrighted material

Hidden page

in each row, and 7 again for 2 counters being on the board. However, from there the number of states mushrooms. In the case where the game is a draw, so that all of the squares are full, there is something less than $2^{7\times6} = 2^{42}$ states. I say something less because this counts all the cases that include a line of 4, and also ignores the fact that there are only 21 counters of each colour. The fact remains that the state space is immense, so it is probably going to take a long time to learn.

However, programming the game is relatively simple. There are two absorbing states: when the board is full, and when somebody wins. In either of these cases a reward is given. So you will have to decide on rewards, and write some code that detects when one or the other state has happened. The choice that is made at each turn is simply which column to add the new counter in, so there are only seven possible actions. You need to represent the board, for which I'd recommend a 2D array with 0 meaning empty, 1 meaning contains a red counter, and 2 meaning contains a yellow counter. This should make it easy to detect the absorbing states.

Having set up that lot, you need to make a number of modifications to the Q-learning code. Firstly, you are not going to pass in transition and reward matrices, since making them would be crazy. You are probably going to give a reward of 0 to every move except a win and a loss, so change the code to present those rewards. You then need to change the ϵ-greedy search strategy to simply pick a random (but not full) column, rather than look at the transition matrix. Then that's it, set it running (and be prepared to wait for a very long time. I trained the algorithm for 20,000 games against a purely random player, and at the end of that the Q-learner was winning about 80% of the games).

Copyrighted material

Chapter 14

Markov Chain Monte Carlo (MCMC) Methods

In this chapter we are going to look at a method that has revolutionised statistical computing and statistical physics over the past 20 years. The principal algorithm has been around since 1953, but only when computers became fast enough to be able to perform the computations on real world examples in hours instead of weeks did the methods become really well known. However, this algorithm has now been cited as one of the most influential ever created.

There are two basic problems that can be solved using these methods, and they are the two that we have been wrestling with for pretty much the entire book: we may want to compute the optimum solution to some objective function, or compute the posterior distribution of a statistical learning problem. In either case the state space may well be very large, and we are only interested in finding the best possible answer—the steps that we go through along the way are not important. We've seen several methods of solving these types of problems during the book, and here we are going to look at one more. We will see a place where MCMC methods are very useful in Section 15.1.

The idea behind everything that we are going to talk about in this chapter is that as we explore the state space, we can also construct samples as we go along in such a way that the samples are likely to come from the most probable parts of the state space. In order to see what this means, we will discuss what Monte Carlo sampling is, and look at Markov chains.

14.1 Sampling

We have produced samples from probability distributions in almost all of the algorithms we have looked at, for example, for initialisation of weights. In many cases, the probability distribution we have used has been the uniform one on $[0, 1)$, and we have done it using the `random.rand()` function in NumPy, although we have also seen sampling from Gaussian distributions using `random.normal()`.

Copyrighted material

14.1.1 Random Numbers

The basis of all of these sampling methods is in the generation of random numbers, and this is something that computers are not really capable of doing. However, there are plenty of algorithms that produce pseudo-random numbers, the simplest of which is the linear congruential generator. This is a very simple function that is defined by a recurrence relation (i.e., you put one number in to get the second number, and then feed that back in to get the third, and then repeat the cycle):

$$x_{n+1} = (ax_n + c) \mod m, \tag{14.1}$$

where a, c, and m are parameters that have to be chosen carefully. All of them, and the initial input x_0 (which is known as the seed), are integers, and so are all of the outputs. The modulus function means that the largest number that can be produced is m, and so there are at most m numbers that can be produced by the algorithm. Once one number appears a second time, the whole pattern will repeat again since the equation only uses the current output as input. The length of the sequence between repeats is the period, and it should obviously be as long as possible, since it is the most obvious non-randomness in the algorithm. There has been a lot of investigation of choices of the parameters so that the period is length m, so that every integer between 0 and m is produced before the pattern cycles. There are various choices of the parameters that have been selected to work well, including $m = 2^{32}$; $a = 1,664,525$; and $c = 1,013,904,223$. Clearly, just picking numbers at random isn't going to be that useful.

There has been a lot of effort put into different random number generators, since they are important not just for statistical computing, but also cryptography and security. The industry-standard algorithm for generating random samples is the Mersenne Twister, which is based on Mersenne prime numbers. It is the random number generator used in NumPy. No matter what algorithm generates the numbers, though, it is important to remember that they are not genuinely random, and as John von Neumann, one of the fathers of modern computing, stated:

> Anyone who considers arithmetic methods of producing random digits is, of course, in a state of sin.

The other troublesome thing about random numbers is that it is not actually possible to prove that a sequence of numbers are truly random. There are several tests that can be made of a sequence of numbers to see if they seem to be random. Examples include calculating the entropy of the sequence (entropy was described in Section 6.2.1), using a compression algorithm on the sequence (since compression algorithms exploit redundancy, i.e., predictability, in the input, if the compression algorithms fail to make the input smaller, then it might be because they are random), and just checking how many numbers

Copyrighted material

are odd compared to even. However, you can never guarantee that a sequence is random, just that it hasn't failed most of the tests yet (but just because it fails one or two of them at some point in the sequence doesn't mean that the sequence isn't random; truly random numbers can look deterministic for a long time... this is part of the joy of randomness!). I'll leave the last word on this to von Neumann again:

> In my experience it was more trouble to test random sequences than to manufacture them.

14.1.2 Gaussian Random Numbers

The Mersenne twistor produces uniform random numbers. However, often we might want to produce samples from other distributions, e.g., Gaussian. The usual method of doing this is the Box-Muller scheme, which uses a pair of uniformly randomly distributed numbers in order to make two independent Gaussian-distributed numbers with zero mean and unit variance. There are a few ways to implement this; we will look at the more efficient polar form of the method, which works as follows:

The Box-Muller Scheme

- Pick two uniformly distributed random numbers between -1 and 1 (x_1, x_2) (for example, use `random.rand(2)*2-1`)
- If $x_1^2 + x_2^2 > 1$, discard them and pick two more (this means that $p(x_1, x_2) = \frac{1}{\pi}$, since they are inside the unit circle)
- Compute $w = x_1^2 + x_2^2$
- Compute $y_1 = x_1 \left(\frac{-2 \ln w}{w}\right)^{\frac{1}{2}}$ and similarly for y_2
- These y_i have probability:

$$p(y_1, y_2) = p(x_1, x_2) \left(\frac{\partial(x_1, x_2)}{\partial(y_1, y_2)}\right) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2} y_1^2\right) \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2} y_2^2\right) \tag{14.2}$$

  which describes two independent variables with zero mean, unit variance Gaussian distribution

---

A plot of 1,000 samples created by the Box-Muller scheme along with the zero mean, unit variance Gaussian line is shown in Figure 14.1. There is a more efficient algorithm for computing Gaussian-distributed random numbers known as the Ziggurat algorithm.

There may well be many other distributions that we want to sample from. For common statistical distributions people have worked out schemes like the

Copyrighted material

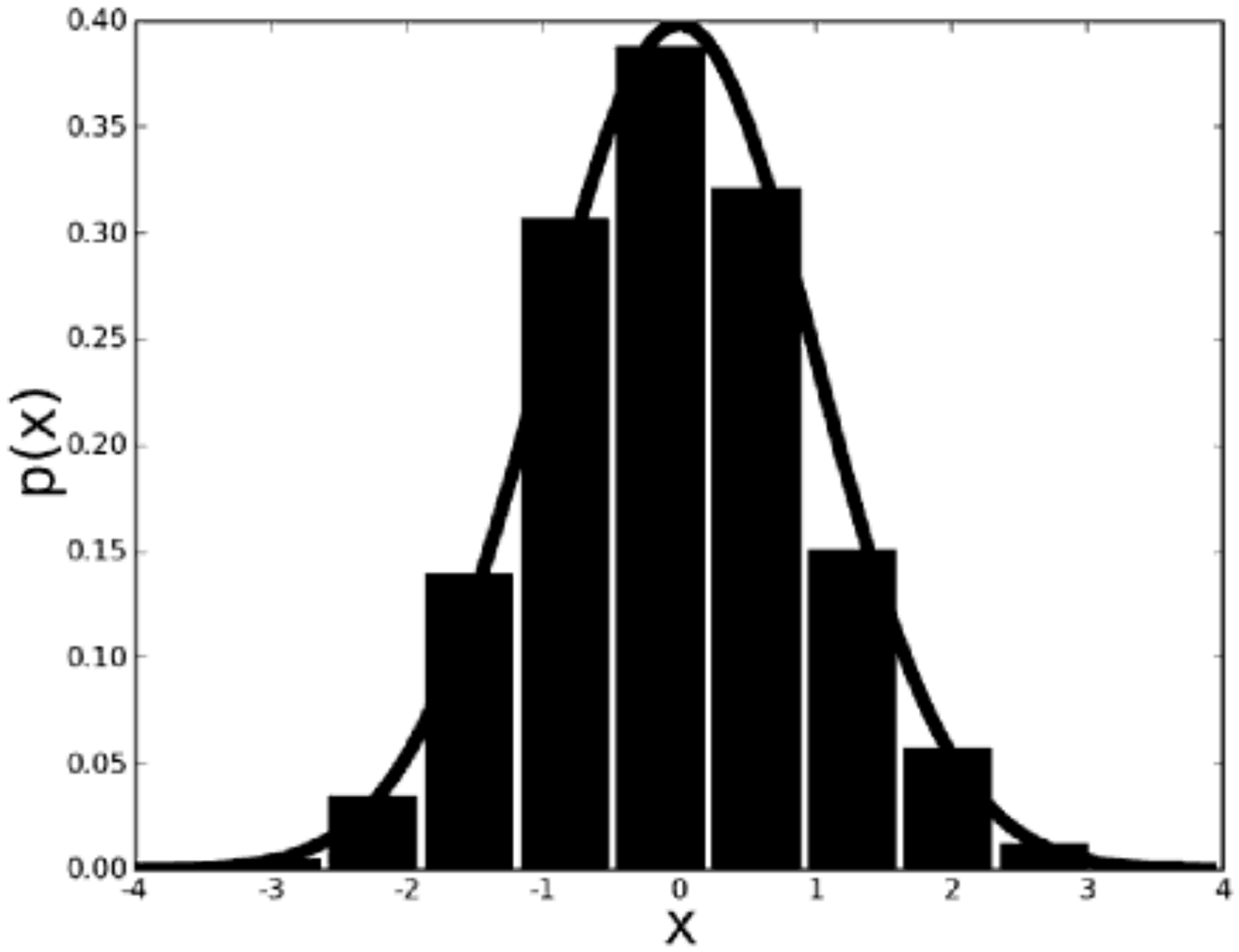


FIGURE 14.1: Histogram of 1,000 Gaussian samples created by the Box-Muller scheme. The line gives the Gaussian distribution with zero mean and unit variance.

Box-Muller scheme, but we might want to sample from distributions that we can't describe in those terms. We will see examples of this in Chapter 15. We would like a method of sampling from a distribution that doesn't have to be tailored to the distribution. There is one important concept that can be seen in the Box-Muller scheme, and that is the idea of rejection. When the original samples were not inside the unit circle they were rejected and another one computed to replace them. This is a bit like simulated annealing as we saw it in Section 11.6: we constructed a possible solution and then decided whether or not to use it. Rejection adds computational cost to the procedure, since if we were unlucky this algorithm could run for a long time before it found a pair of numbers that satisfied the criteria. However, it also means that we find samples that satisfy our requirements without having to design any tricky code, and it is generally faster as well, since the computational cost of generating some random numbers is rather less than the cost of doing the complicated transform. We are going to see rejection used a lot more in this chapter, but before we get there, we should set the idea of sampling on to a proper theoretical footing.

Copyrighted material

Hidden page

$$\hat{\mathbf{x}} = \arg\max_{\mathbf{x}^{(i)}} p\left(\mathbf{x}^{(i)}\right). \tag{14.5}$$

Allegedly, the idea of Monte Carlo sampling (and the reason that it got its name) first came about when Stan Ulam was considering the probabilities of particular hands of cards. In fact, the whole of probability theory was originally developed by some of the great French mathematicians, such as Fermat, in order to reason about games of chance, so Monte Carlo sampling is in pretty good company. Suppose that you want to do something relatively simple, such as to predict how many times you should expect to win at the patience game that came free with your computer. All you need to do is work out the rules for when you win based on the initial setup, and then look at how many of these setups there are. In a standard deck there are 52 cards, so there are 52! ($\approx 8 \times 10^{67}$) different ways in which the cards can be distributed. So even before we start thinking about the specific rules for the game, we know that the number of different layouts is so large it is basically impossible to think about. Despairing of working it out, you might decide to play a couple of hands of patience and see how well you do. In fact, the Monte Carlo principle tells you that that is exactly what you should be doing. Suppose that you play ten games of patience and six of them come out. You might be able to argue that approximately 60% of the patience games will do well. To believe this, you will have to play far more than ten games with the same success rate, or course; and even then it assumes that you are a good patience player, and don't cheat.

14.3 The Proposal Distribution

We now have everything that we need if the distribution $p(\mathbf{x})$ that we are sampling from is easy (that is, not computationally expensive) to sample from. Unfortunately, this is very rarely the case, but fortunately there is a way to get around this problem, which is to cheat by inventing a simpler distribution $q(\mathbf{x})$ that we can sample from easily, and picking samples from there. Obviously we can't just pick any distribution $q(\mathbf{x})$, there has to be some relation between them. So we assume that even though we don't know $p(\mathbf{x})$, we can evaluate $\tilde{p}(\mathbf{x})$, for a given $\mathbf{x}$ where:

$$p(\mathbf{x}) = \frac{1}{Z_p}\tilde{p}(\mathbf{x}), \tag{14.6}$$

where Z_p is some normalisation constant that we don't know. This is not usually an unreasonable assumption; we are not saying that we do not know $p(\mathbf{x})$, just that we can't sample from it easily. Now we can pick a number M so that $\tilde{p}(\mathbf{x}) \leq Mq(\mathbf{x})$ for all values of $\mathbf{x}$. We generate a random number $\mathbf{x}^*$

Copyrighted material

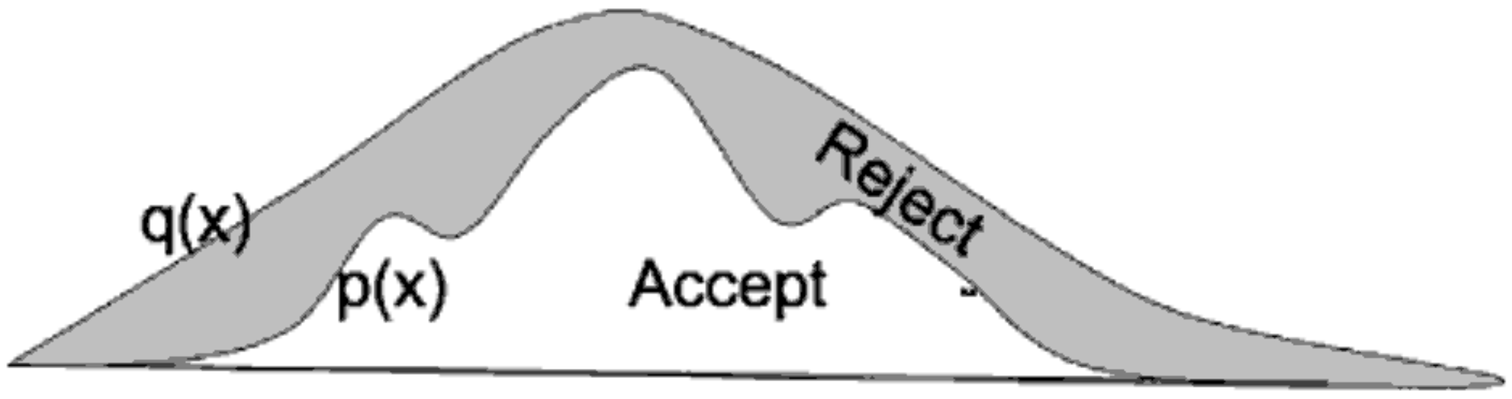


FIGURE 14.2: The proposal distribution method.

from $q(\mathbf{x})$, and we want this to look like a sample from $p(\mathbf{x})$. We therefore turn to the idea of rejection again, looking at how likely it is that the sample comes from $p(\mathbf{x})$, and discarding it if it turns out to be unlikely.

We make the decision of whether or not to accept the sample by picking a uniformly distributed random number u between 0 and $Mq(\mathbf{x}^*)$. If this random number is less than $\tilde{p}(\mathbf{x}^*)$, then we reject $\mathbf{x}^*$, otherwise we keep it. The reason why this works is known as the envelope principle: the pair $(\mathbf{x}^*, u)$ is uniformly distributed under $Mq(\mathbf{x}^*)$, and the rejection part throws away samples that don't match the uniform distribution on $p(\mathbf{x}^*)$, so $Mq(\mathbf{x})$ forms an envelope on $p(\mathbf{x})$. Figure 14.2 shows the idea. We sample from $Mq(\mathbf{x})$ and reject any sample that lies in the grey area. The smaller M is, the more samples we get to keep, but we need to ensure that $\tilde{p}(\mathbf{x}) \leq Mq(\mathbf{x})$. This method is known as rejection sampling, and the algorithm can be written as:

The Rejection Sampling Algorithm

- Sample $\mathbf{x}^*$ from $q(\mathbf{x})$ (e.g., using the Box-Muller scheme if $q(\mathbf{x})$ is Gaussian)
- Sample u from uniform$(0, \mathbf{x}^*)$
- if $u < p(\mathbf{x}^*)/Mq(\mathbf{x}^*)$:
  - add $\mathbf{x}^*$ to the set of samples
- else:
  - reject $\mathbf{x}$ and pick another sample

As an example of using rejection sampling, Figure 14.3 shows the results of using it to sample from the mixture of two Gaussians by using the uniform distribution shown by the dotted line. Using $M = 0.8$, as shown in the figure, the algorithm rejects about half of the samples. Using $M = 2$ the algorithm rejects about 85% of samples. So with rejection sampling, you have to throw away samples, and if you don't pick M properly, you will have to reject a lot of them. The curse of dimensionality makes the problem even worse. There are

Copyrighted material

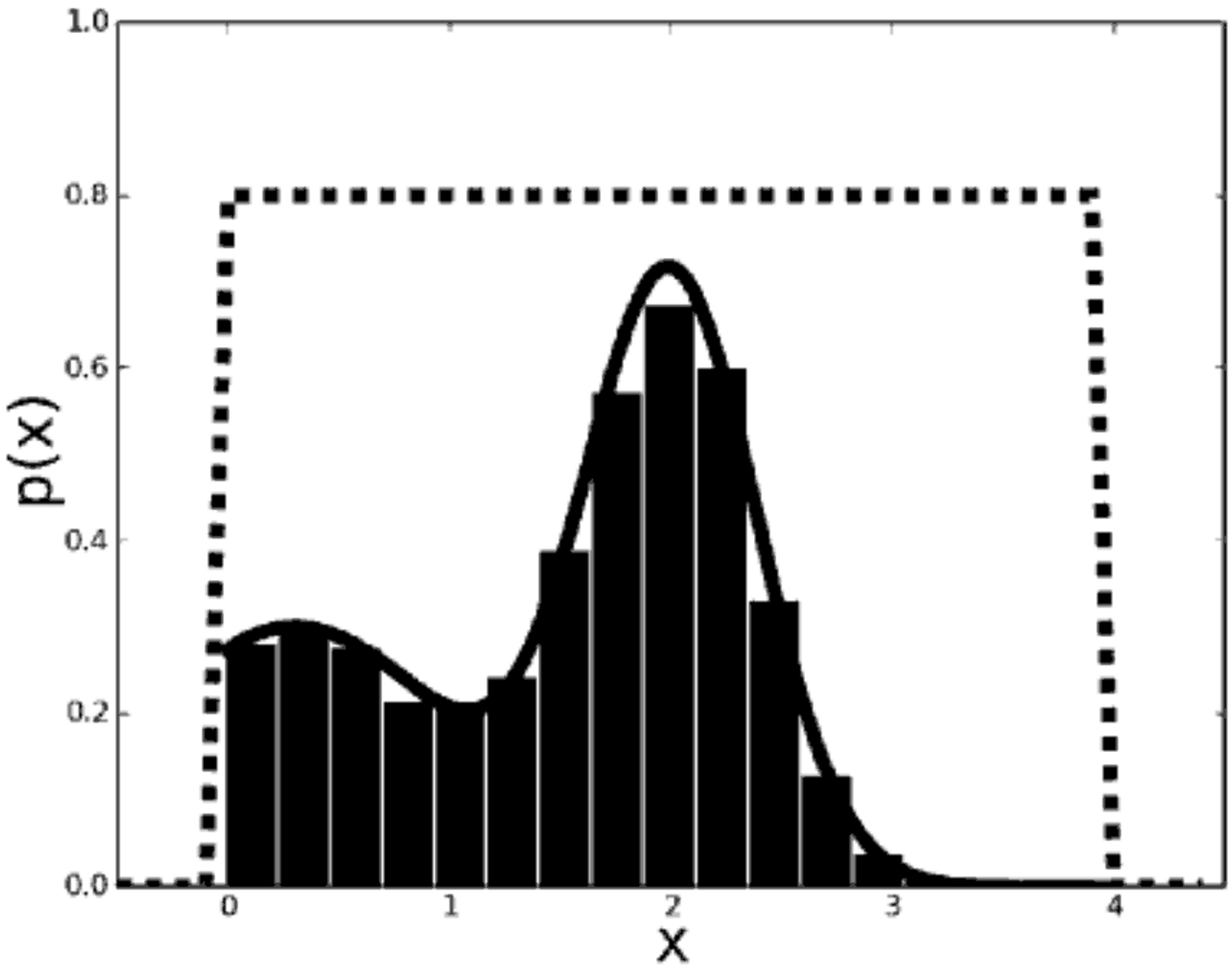


FIGURE 14.3: The histogram shows samples of a mixture of two Gaussians (given by the solid line) as sampled from the uniform box shown as a dotted line by using rejection sampling.

two things that we can do to get over this problem. One is to develop some more sophisticated methods of understanding the space that we are sampling, and the other is to try to ensure that the samples are taken from areas of the space that have high probability.

The reason why we are using these methods at all is that we can't sample from the actual distribution we want, since that is too difficult and/or expensive, but it might be possible to understand it in other ways. In Section 14.4.1 we will look at methods that allow us to travel around within the space by using simple local moves. Before we get to that we will look at a method that tries to ensure that the samples come from regions of high probability. The method is known as importance sampling, because it attaches a weight that says how important each sample is.

Suppose that we want to compute the expectation of a function $f(\mathbf{x})$ for a continuous random variable $\mathbf{x}$ distributed according to unknown distribution $p(\mathbf{x})$. Starting from the expression of the expectation that we wrote out earlier, we can introduce another distribution $q(\mathbf{x})$:

Copyrighted material

$$
\begin{aligned}
E(f) &= \int p\left(\mathbf{x}\right) f\left(x\right) d\mathbf{x} \\
&= \int p(\mathbf{x}) f(\mathbf{x}) \frac{q(\mathbf{x})}{q(\mathbf{x})} d\mathbf{x} \\
&\approx \frac{1}{N} \sum_{i=1}^{N} \frac{p\left(\mathbf{x}^{(i)}\right)}{q\left(\mathbf{x}^{(i)}\right)} f\left(x^{(i)}\right), \qquad (14.7)
\end{aligned}
$$

where we have used the fact that $q(\mathbf{x})$ is the density of a random variable, and so if we perform $\int q(\mathbf{x}) d\mathbf{x}$ over all values of $\mathbf{x}$ then it must equal 1. The ratio $w(\mathbf{x}^{(i)}) = p(\mathbf{x}^{(\mathbf{i})})/q(\mathbf{x}^{(\mathbf{i})})$ is called the importance weight, and it corrects for sampling from the grey region in Figure 14.2 without having to reject samples. While this can be used to estimate the expectation directly, the real benefit of computing the importance weights is that they can be used in order to resample the data. This leads to an algorithm known descriptively as Sampling-Importance-Resampling. In the words of the advert, it 'does exactly what it says on the tin':

The Sampling-Importance-Resampling Algorithm

- Produce N samples $\mathbf{x}^{(i)}$, $i = 1 \ldots N$ from $q(\mathbf{x})$
- Compute normalised importance weights

$$
w^{(i)} = \frac{p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})}{\sum_j p(\mathbf{x}^{(j)})/q(\mathbf{x}^{(j)})} \qquad (14.8)
$$

- Resample from the distribution $\{\mathbf{x}^{(i)}\}$ with probabilities given by the weights $w^{(i)}$

An implementation of this in Python is shown next, and the results of using sampling-importance-resampling on the example in Figure 14.3 is given in Figure 14.4. Note that this method does not reject any samples, but it does involve two separate sampling steps and a relatively expensive loop. Like the other algorithms we have seen, it is sensitive to the quality of the match between the proposal distribution $q(\mathbf{x})$ and the actual distribution $p(\mathbf{x})$.

```
# Sample from q
sample1 = random.rand(n)*4

# Compute weights
w = p(sample1)/q(sample1)
w /= sum(w)
```

Copyrighted material

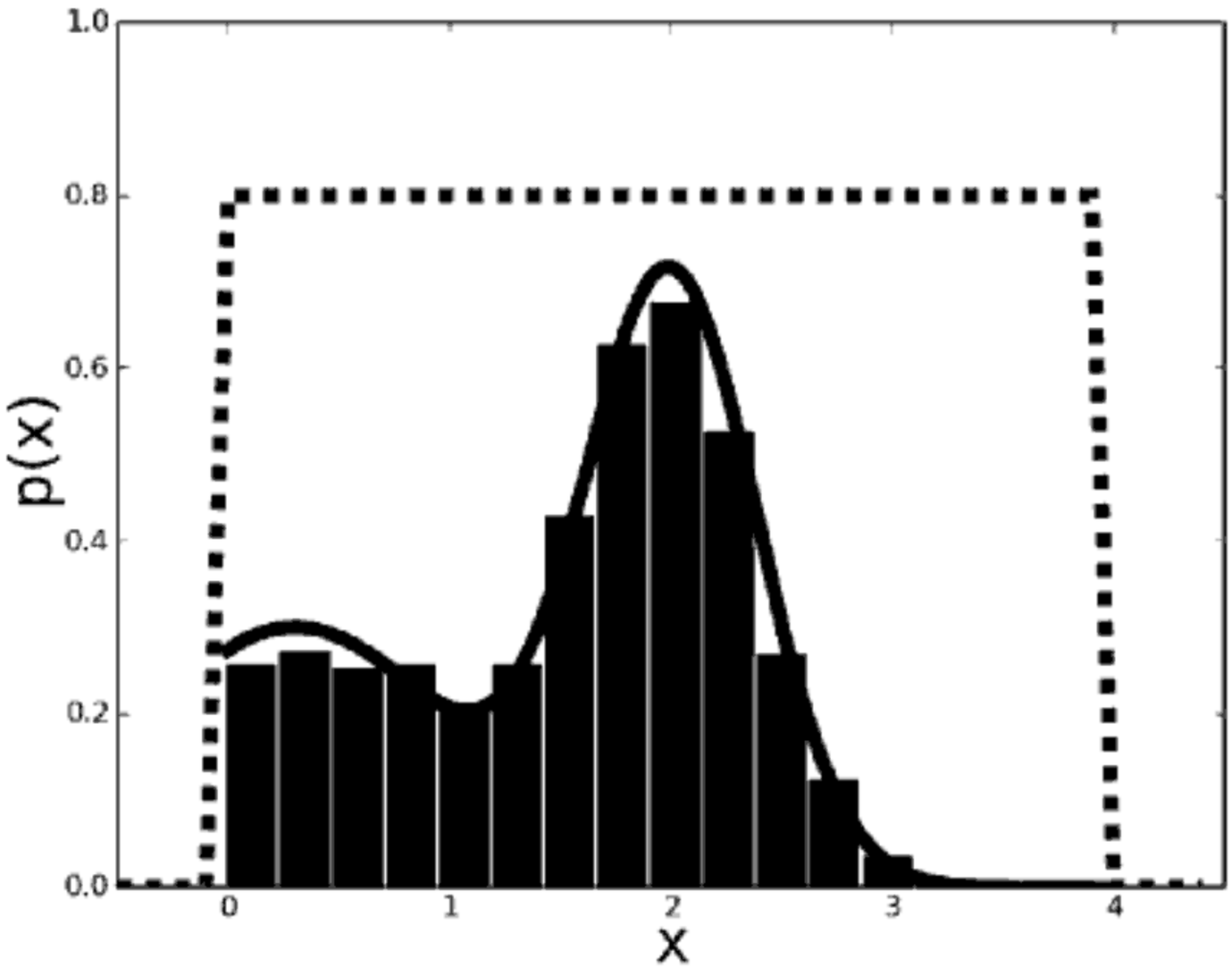


FIGURE 14.4: The histogram shows samples created using sampling-importance-resampling from a mixture of two Gaussians (given by the solid line) as sampled from the uniform box shown as a dotted line.

```
# Sample from sample1 according to w
cumw = zeros(n)
cumw[0] = w[0]
for i in range(1,n):
    cumw[i] = cumw[i-1]+w[i]

u = random.rand(n)

index = 0
for i in range(n):
    indices = where(u<cumw[i])
    sample2[index:index+size(indices)] = sample1[i]
    index += size(indices)
    u[indices]=2
```

In Section 15.4.2 we will see a method that uses sampling-importance-resampling in an on-line application, known as a particle filter or sequential Monte Carlo method. However, we will first turn our attention to how we can find out more about the sample space. The basic idea is to keep track of the sequence of samples and modify the proposal distribution to take advantage of this, for whic we will have to use some more complicated machinery.

Copyrighted material

14.4 Markov Chain Monte Carlo

14.4.1 Markov Chains

In probabilistic terms a chain is a sequence of possible states, where the probability of being in state s at time t is a function of the previous state. A Markov chain is a chain with the Markov property, i.e., the probability at time t depends only on the state at $t-1$, as discussed in Section 13.3. The set of possible states are linked together by transition probabilities that say how likely it is that you move from the current state to each of the others, and they are generally written as a matrix T. They might be constant, or functions of some other variables, but here we will assume that they are constant. Note that, unlike the Markov Decision Processes that we saw in Section 13.3, there is no action here that affects the probability of moving into a particular state.

Given a chain, we can perform a random walk on the chain by choosing a start state and randomly choosing each successive state according to the transition probabilities. The link to sampling that we need is that if the transition probabilities reflect the distribution that we wish to sample from, then a random walk will explore that distribution. One problem with this is that random walks are very inefficient at exploring space, since they move back towards the start as often as they move away, which means the distance they move from the start scales as $\sqrt{t}$, where t is the number of samples. We therefore want to explore more efficiently than just using a random walk.

We do this by setting up our Markov chain so that it reflects the distribution we wish to sample from, and we want the distribution $p(\mathbf{x}^{(i)})$ to converge to the actual distribution $p(\mathbf{x})$ no matter what state we start from. Since we can start from any state, this tells us that every state is reachable from every other state (so that the transition matrix can't be cut up into smaller matrices); this means that the chain is irreducible, a property that is known as ergodicity, and aperiodic so that there are no cycles in the chain.

We also want the distribution $p(\mathbf{x})$ to be invariant to the Markov chain, which means that the transition probabilities don't change the distribution:

$$p(\mathbf{x}) = \sum_{\mathbf{y}} T(\mathbf{y}, \mathbf{x}) p(\mathbf{x}). \tag{14.9}$$

Finding the transition probabilities to make this true requires that we can move backwards and forwards along the chain with equal probability, so that the chain is reversible. This says that the probability of being in an unlikely state s (sampling datapoint $\mathbf{x}$), but heading for a likely state s' (datapoint $\mathbf{x}'$) should be the same as being in the likely state s' and heading for the unlikely state s, so that:

$$p(\mathbf{x})T(\mathbf{x}, \mathbf{x}') = p(\mathbf{x}')T(\mathbf{x}', \mathbf{x}). \tag{14.10}$$

Copyrighted material

This is known as the detailed balance condition and the fact that it leaves the distribution $p(\mathbf{x})$ alone is fairly obvious with a little calculation. If the chain satisfies the detailed balance condition, then it must be ergodic, since $\sum_{\mathbf{y}} T(\mathbf{x}, \mathbf{y}) = 1$, since you must have come from some state, and so:

$$\sum_{\mathbf{y}} p(\mathbf{y}) T(\mathbf{y}, \mathbf{x}) = p(\mathbf{x}), \tag{14.11}$$

which means that $p(\mathbf{x})$ must be an invariant distribution of T. So if we can work out how to construct a Markov chain with detailed balance we can sample from it in order to sample from our distribution. This is known as Markov Chain Monte Carlo (MCMC) sampling, and the most popular algorithm that is used for MCMC is the Metropolis-Hastings algorithm after the two people who were directly involved in its creation.

14.4.2 The Metropolis-Hastings Algorithm

We assume that we have a proposal distribution of the form $q(\mathbf{x}^{(i)}|\mathbf{x}^{(i-1)})$ that we can sample from. The idea of Metropolis-Hastings is similar to that of rejection sampling: we take a sample $\mathbf{x}^*$ and choose whether or not to keep it. Except, unlike rejection sampling, rather than picking another sample if we reject the current one, instead we add another copy of the previous accepted sample. Here, the probability of keeping the sample is $u(\mathbf{x}^*|\mathbf{x}^{(i-1)})$:

$$u(\mathbf{x}^*|\mathbf{x}^{(i-1)}) = \min\left(1, \frac{\tilde{p}(\mathbf{x}^*) q(\mathbf{x}^{(i)}|\mathbf{x}^*)}{\tilde{p}(\mathbf{x}^{(i)}) q(\mathbf{x}^*|\mathbf{x}^{(i)})}\right). \tag{14.12}$$

The Metropolis-Hastings Algorithm

- Given an initial value x_0
- repeat
  - sample $\mathbf{x}^*$ from $q(\mathbf{x}_i|\mathbf{x}_{i-1})$
  - sample u from the uniform distribution
  - if $u <$ Equation (14.12):
    - set $\mathbf{x}[i+1] = \mathbf{x}^*$
  - otherwise:
    - set $\mathbf{x}[i+1] = \mathbf{x}[i]$
- until you have enough samples

So why does this algorithm work? Each step involves using the current value to sample from the proposal distribution. These values are accepted if they move the Markov chain towards more likely states, and because the

Copyrighted material

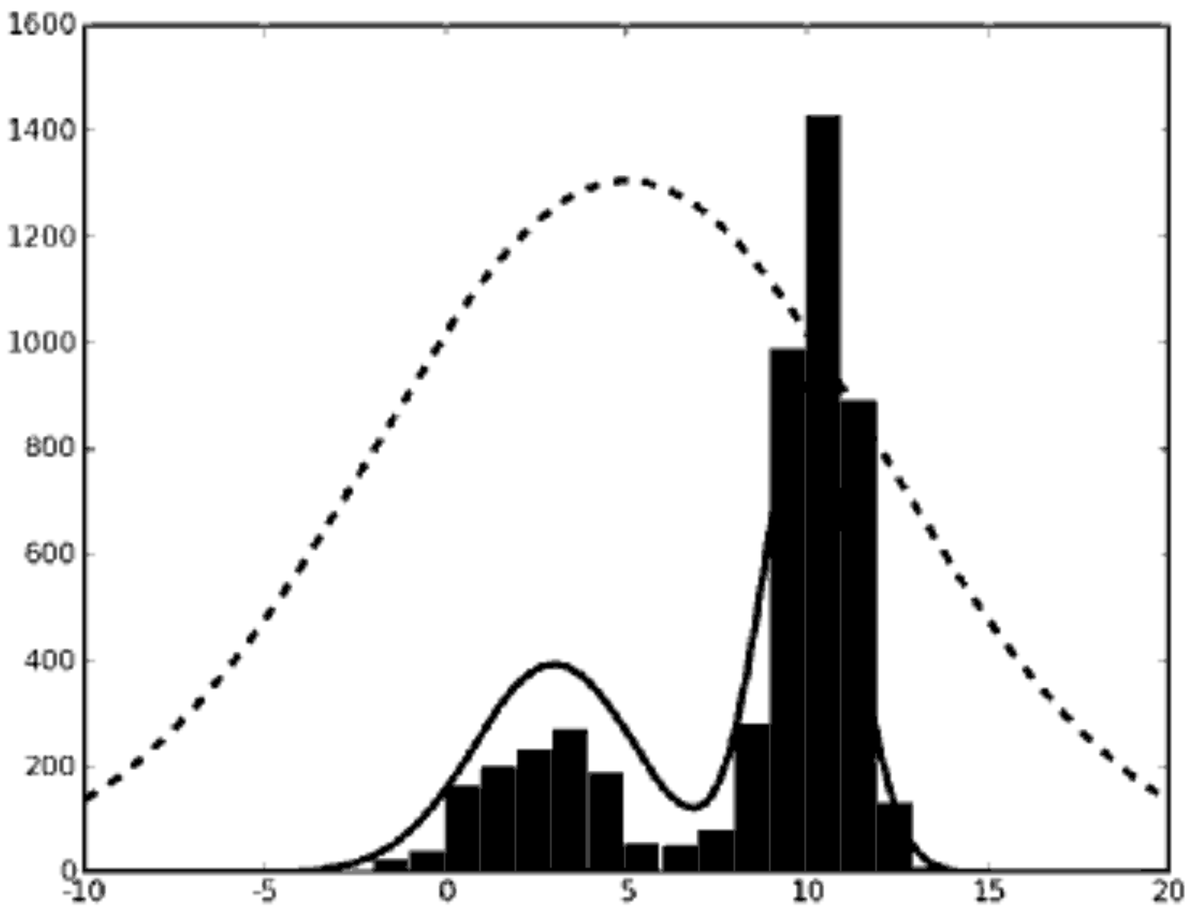


FIGURE 14.5: The results of the Metropolis-Hastings algorithm when the true distribution is a mixture of two Gaussians (shown by the solid line) and the proposal distribution is a single Gaussian (the dotted line).

Markov chain is reversible (since it satisfies the detailed balance condition) the algorithm explores states that are proportional to the difficult distribution $p(x)$.

The Metropolis-Hastings (and variants of it) are by far the most commonly used MCMC methods, and it is also the most general. It requires that you choose the proposal distribution $q(x^*|x)$ carefully, but it is a very simple algorithm to use. Figure 14.5 shows 5,000 samples computed using the algorithm on a mixture of two Gaussians based on a proposal distribution that is a single Gaussian.

Note that if the proposal distribution is symmetric, then it drops out of the test in Equation (14.12). This is the original Metropolis algorithm, and it is much closer to the pure random walk. The results of using this algorithm on the same data can be seen in Figure 14.6.

There are other choices of proposal distribution, and they lead to variants on the Metropolis-Hastings algorithm. We will consider the two most common choices next.

14.4.3 Simulated Annealing (Again)

There are lots of times when we might just want to find the maximum of a distribution rather than approximate the distribution itself. We can do this in calculating $\arg\max_{\mathbf{x}^{(i)}} p(\mathbf{x}^{(i)})$ (that is, the $\mathbf{x}^{(i)}$ with the largest probability), but while doing this we will have computed samples from many parts of the space, not just around the maximal region. A possible solution is to use simulated annealing as we did in Section 11.6. This changes the Markov chain

Copyrighted material

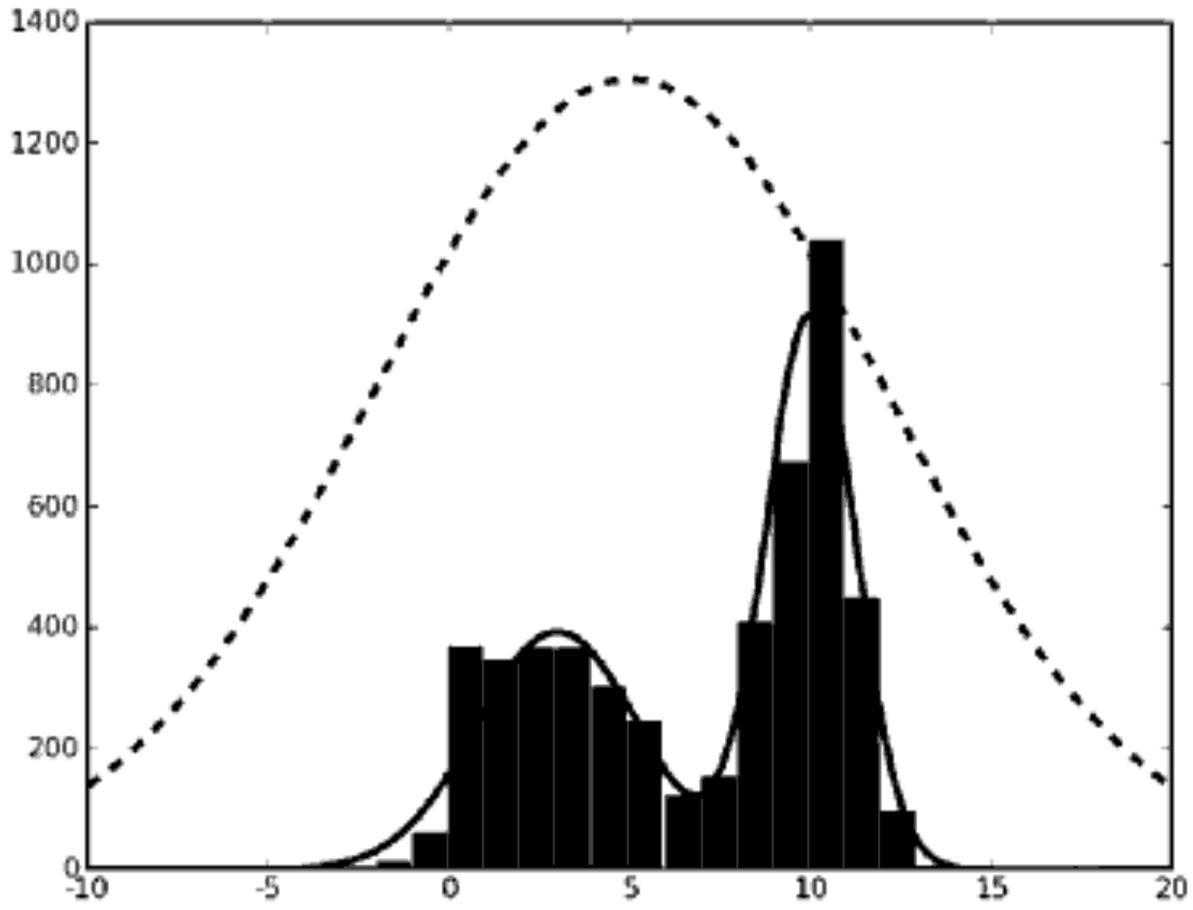


FIGURE 14.6: The results of the Metropolis algorithm when the true distribution is a mixture of two Gaussians (shown by the solid line) and the proposal distribution is a single Gaussian (the dotted line).

so that its invariant distribution is not $p(\mathbf{x})$, but rather $p^{1/T_i}(\mathbf{x})$, where $T_i \to 0$ as $i \to \infty$. We need an annealing schedule that cools the system down over time so that we are progressively less likely to accept solutions that are worse over time.

There are only two modifications needed to the Metropolis-Hastings algorithm, and both are trivial: we extend the acceptance criterion to include the temperature and add a line into the loop to include the annealing schedule. The results of using simulated annealing on the example where the true distribution is a mixture of two Gaussians and the proposal distribution is just one is shown in Figure 14.7.

14.4.4 Gibbs Sampling

Another variation on the Metropolis-Hastings algorithm comes when we already know the full conditional probability $p(x_j|x_1, \ldots x_{j-1}, x_{j+1}, \ldots x_n)$ (which is often written as $p(x_j|x_{-j})$ for convenience). We are going to see some examples of this in the next chapter: Bayesian networks. In Section 15.1.2 we will deal with a set of probabilities from a network that looks like:

$$p(\mathbf{x}) = \prod_j p(_j|x_{\alpha j}). \tag{14.13}$$

Given that we know $p(x_j|x_{\alpha j}) \prod_{k \in \beta(j)} p(x_k|x_{\alpha(k)})$ (which is $p(x_j|x_{-j})$), maybe we should try using it as the proposal distribution, giving:

Copyrighted material

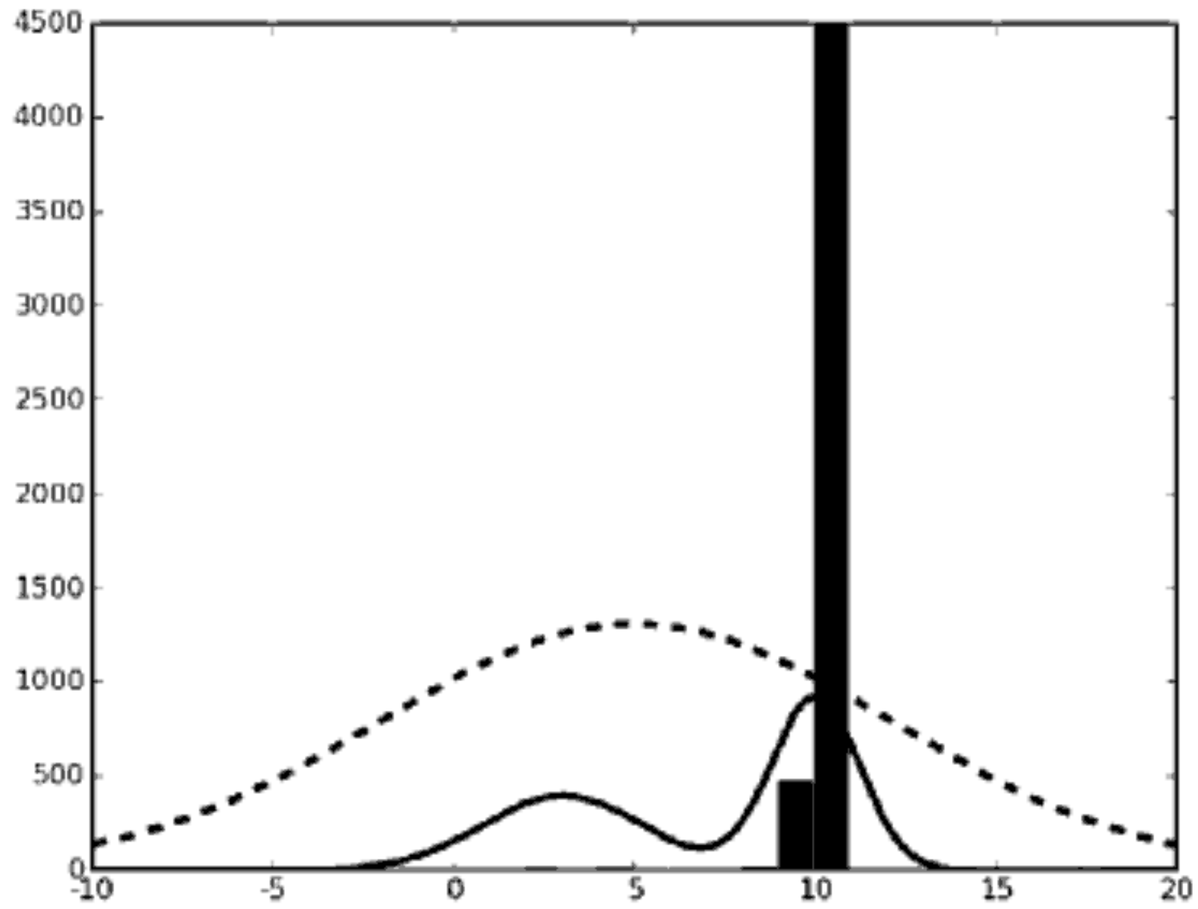


FIGURE 14.7: Using simulated annealing gives the maximum rather than an approximation to the distribution, as is shown here for the same example as in Figures 14.5 and 14.6.

$$q(x^*|x^{(i)}) = \begin{cases} p\left(x_j^*, x_{-j}^{(i)}\right) & \text{if } x^*_{-j} = x^{(i)}_{-j} \\ 0 & \text{otherwise.} \end{cases} \tag{14.14}$$

If we then use Metropolis-Hastings, we find that the acceptance probability P_a is:

$$P_a = \min\left\{1, \frac{p(x^*)p(x_j^{(i)}|x^{(i)}_{-j})}{p(x^{(i)})p(x_j^*|x^*_{-j})}\right\}, \tag{14.15}$$

and looking carefully at this and expanding out the conditional probabilities we get:

$$P_a = \min\left\{1, \frac{p(x^*)p(x_j^{(i)}, x^{(i)}_{-j})p(x^{(i)}_{-j})}{p(x^{(i)})p(x_j^*, x^*_{-j})p(x^*_{-j})}\right\}. \tag{14.16}$$

Since $p(x_j^*, x^*_{-j}) = p(x^*)$, and similarily for $p^{(i)}$, we only have to worry about $\frac{p(x^{(i)})}{p(x^*_{-j})}$. From the definition of the proposal distribution we know that $x*_{-j} = x^{(i)}_{-j}$, and so the computation is actually $\min 1, 1 = 1$. So we always accept the proposal, which makes things much simpler.

The total algorithm is given by choosing each variable and sampling from its conditional distribution. That's it! The only option that you have is whether to go through the variables in order, or whether to update them in a random order. Rather than running up to some maximum value N, it is not

Copyrighted material

uncommon to run until the joint distribution stops changing. This algorithm is known as the Gibbs sampler and it forms the basis of the software package BUGS (Bayesian Updating With Gibbs Sampling) that is commonly used in statistics. It is also a very useful algorithm for Bayesian networks, as we shall see in the next chapter.

The Gibbs Sampler

- for each variable x_j:
  - initialise $x_j^{(0)}$
- repeat
  - for each variable x_j:
    - sample $x_1^{(i+1)}$ from $p(x_1|x_2^{(i)}, \ldots x_n^{(i)})$
    - sample $x_2^{(i+1)}$ from $p(x_2|x_1^{(i+1)}, x_3^{(i)}, \ldots x_n^{(i)})$
    - ...
    - sample $x_n^{(i+1)}$ from $p(x_n|x_1^{(i+1)}, \ldots x_{n-1}^{(i+1)})$
- until you have enough samples

As an example, suppose that we have a distribution that is made up of two different distributions, a binomial one in x and a beta in y. If you don't know what these distributions are, the combined distribution can be written as:

$$p(x, y, n) = \left(\frac{n!}{x!(n-x)!}\right) y^{x+\alpha-1} + (1-y)^{n-x+\beta-1}. \tag{14.17}$$

The important point is that the overall distribution is a product of two separate ones that can be sampled from separately. Figure 14.8 shows the output of the sampling using the Gibbs sampler, with the line being the correct distribution as usual. There is another example of Gibbs sampling in Section 15.1.2.

Copyrighted material

Hidden page

Hidden page

Chapter 15

Graphical Models

Throughout this book we have seen that machine learning brings together computer science and statistics. Nowhere is this more clearly shown than in one of the most popular areas of current research in machine learning: graphical models (or more completely, probabilistic graphical models), which use graph theory with all its underlying computational and mathematical machinery in order to explain probabilistic models.

The graphs used in graphical models are the exact ones that are taught in basic algorithms classes: a set of nodes, together with links between them, which can be either directed (i.e., have arrows on them so that you can only go one way along them) or not. There are two basic types of graphical models, depending upon whether or not the edges are directed. We will focus primarily on directed graphs, but the undirected kind (known as Markov Random Fields) are described in Section 15.2. For such a simple data structure, graphs have turned out to be incredibly powerful in many different parts of computer science, from constructing compilers to managing computer networks. For this reason, there are lots of readily available algorithms for finding shortest paths (Floyd's and Djiksta's algorithms, which we've already discussed briefly in Section 10.6), determining cycles, etc. Any good book on algorithms will give details of these and many other graph algorithms.

For our part, we are interested in using graphs to encode probability distributions and so we need to decide what nodes and links are in this context. The nodes are fairly obvious. We generate a node for each random variable, and label it accordingly. In this book, we will only consider discrete variables, so that there is a finite number of possible values that the random variable can take. Given a continuous variable we will discretise it into a finite set. While this loses information, it makes the problem much simpler. The alternative is to specify the variable by a probability density function, which can be done, but makes the whole thing harder to describe and understand.

The question is what to make the links represent. Perhaps the best way to think about this is to ask what it means if two nodes are not linked. In this case we are saying that there is no connection between those two variables, which is the same as saying that they are independent. Except it isn't quite as simple as that, because two nodes could be linked through a third node. Have a look at the right of Figure 15.1, where C is not directly linked to B, but there is a link through A. For this reason we have to be careful and talk

Copyrighted material

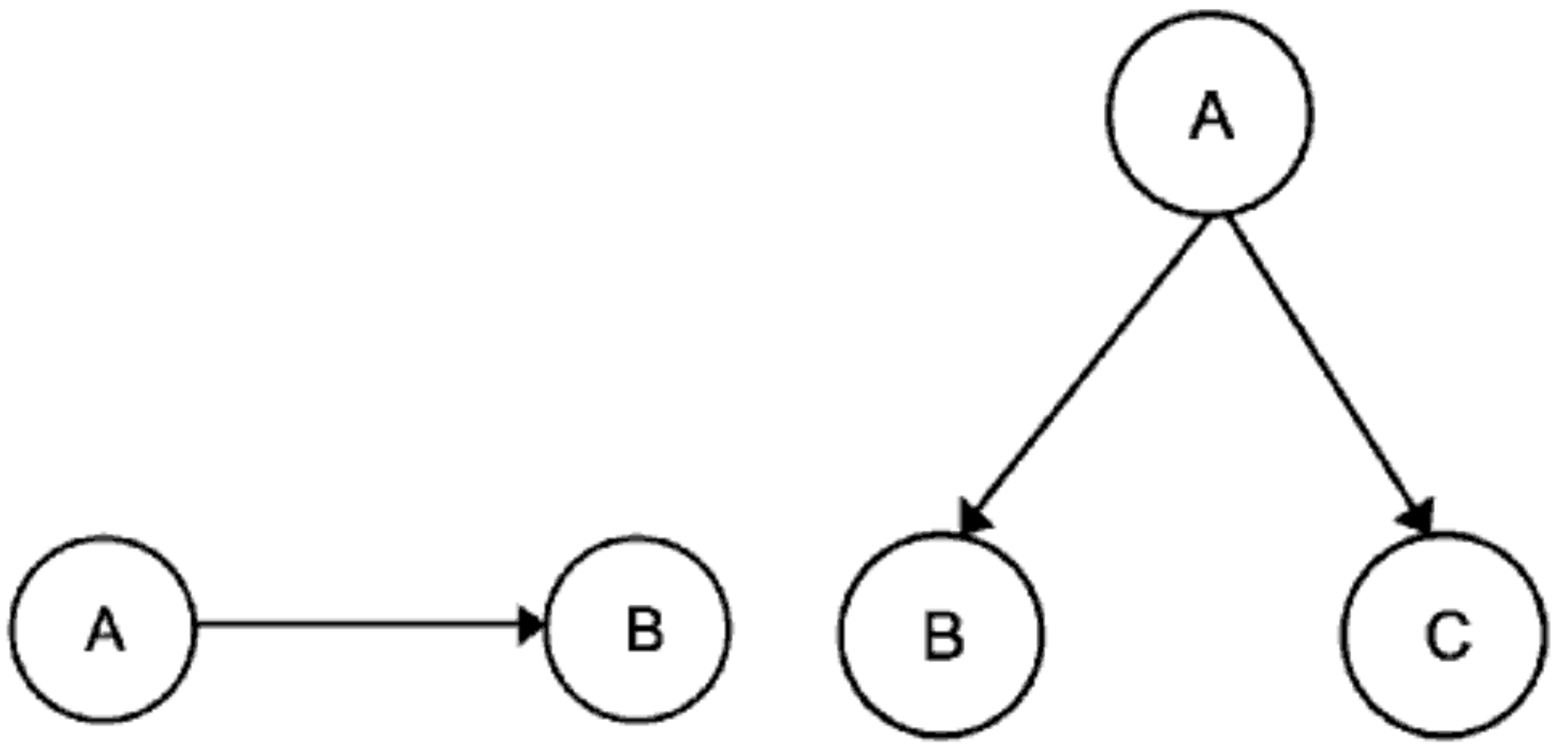


FIGURE 15.1: Two simple graphical models. The arrows denote causal relationships between nodes that represent features.

about conditional independence: C is conditionally independent of B, given A.

We use directed links because these relationships are not symmetrical (unless the variables are independent, in which case there is no link). What does the simplest connected graph that we can make, the one on the left of Figure 15.1, mean? There is a rather loose interpretation of the link, which is to say that 'A' causes 'B' (but note that this isn't quite the same semantic usage that we normally have for 'causes,' since there may be several variables that are all involved in causing B). This is a useful intuition to have, but it is not really correct. More properly, the graph tells us that the probability of A and B is the same as the probability of A times the probability of B conditioned on A: $P(a, b) = P(b|a)P(a)$.

There is a third thing that we need in order to specify the problem properly, which is the conditional probability table for each variable. This specifies what the probabilities are for each of the nodes, conditioned on any nodes that are its parents. If we wanted to work out a value for $P(a, b)$, then we would need a distribution table for $P(a)$ and one for $P(b|a)$. The nodes are separated into these where we can see their values directly—observed nodes—and hidden or latent nodes, whose values we hope to infer, and which may not have clear meanings in all cases.

The basic concept of the graphical model is very simple, which makes it all the more amazing that it produces a powerful set of tools for understanding and creating machine learning algorithms. We will start by looking at the most general model, the Bayesian Belief network or more simply, Bayesian network, and see how they are represented, and the difficulties involved in dealing with them. Following this, we will identify a few places where these difficulties can be overcome, resulting in some very important algorithms that solve a variety of different tasks. In particular, we will look at Markov Random Fields (MRFs), Hidden Markov Models (HMMs), the Kalman Filter, and particle filter.

Copyrighted material

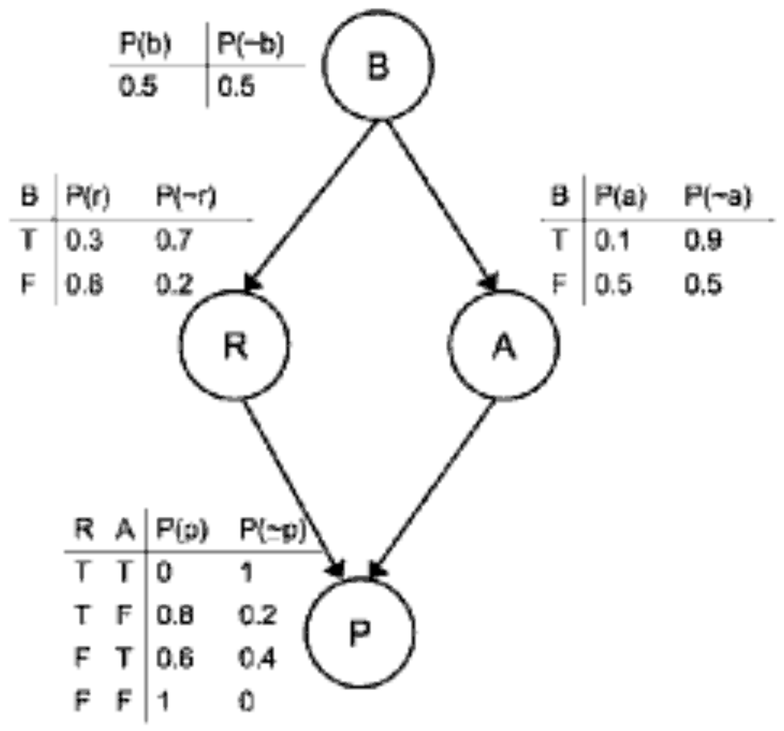


FIGURE 15.2: The sample graphical model. 'B' denotes a node stating whether the exam was boring, 'R' whether or not you revised, 'A' whether or not you attended lectures, and 'P' whether or not you will panic before the exam.

15.1 Bayesian Networks

To start with, we will consider directed graphs, and make one restriction to them, namely that they must not contain cycles, that is, there cannot be any loops in the graphs. These graphs go by the rather unlovely name of DAGs: directed, acyclic graphs, but for graphical models, when they are paired with the conditional probability tables, they are called Bayesian networks. In order to see what we can do with such a network, we need an example.

15.1.1 Example: Exam Panic

Figure 15.2 shows a graph with a full set of distribution tables specified. It is a handy guide to whether or not you will panic before an exam based on whether or not the course was boring ('B'), which was the key factor you used to decide whether or not to attend lectures ('A') and revise ('R'). We can use it to perform inference in order to decide the likelihood of you panicking ('P'). There are two kinds of inferences, depending on whether the observations that are made come from the top of the graph or the bottom. If we have a set of observations that can be used to predict an unknown outcome, then we are doing top-down inference or prediction, whereas if the outcome is known, but the causes are hidden, then we are doing bottom-up inference or diagnosis. Either way, we are working out the values of the hidden (unknown) nodes given information about the observed nodes. For the example in Figure 15.2 we will start by predicting whether or not you will panic before the exam, so it is the outcome that is hidden.

In order to compute the probability of panicking, we need to compute $P(b, r, a, p)$, where the lower-case letters indicate particular values that the upper-case variables can take. The wonderful thing about the graphical model

Copyrighted material

Hidden page

$$
\begin{aligned}
P(r|p) &= \frac{P(p|r)P(r)}{P(p)} \\
&= \frac{\sum_{b,a} P(b,a,r,p)}{P(p)} \\
&= \frac{0.5 \cdot (0.3 \cdot 0.1 \cdot 0 + 0.3 \cdot 0.9 \cdot 0.8) + 0.5 \cdot (0.8 \cdot 0.5 \cdot 0 + 0.8 \cdot 0.5 \cdot 0.8)}{P(p)} \\
&= \frac{0.268}{0.684} = 0.3918. \qquad (15.3)
\end{aligned}
$$

$$
\begin{aligned}
P(r|a) &= \frac{P(p|a)P(a)}{P(p)} \\
&= \frac{0.144}{0.684} = 0.2105. \qquad (15.4)
\end{aligned}
$$

This use of Bayes' rule is the reason why this type of graphical model is known as a Bayesian network. Even in this very simple example, the inference was not trivial, since there were a lot of calculations to do. However, the problem is actually rather worse than that. The computational cost of the simple algorithm we used (start at the root, and follow each link through the graph to perform the computation) is $\mathcal{O}(2^N)$ for a graph with N nodes where each node can be either true or false. In general the problem of exact inference on Bayesian networks is NP-hard (technically, it is actually #P-hard, which is even worse). However, for so-called polytrees where there is at most one path between any two nodes, the computational cost is much smaller—linear in the size of the network.

Unfortunately, it is rare to find such polytrees in real examples, so we can either try to turn other networks into polytrees, or consider only approximate inference, which is the most common solution to the problem, and the method that we'll consider next. We can speed things up a little by getting things into the form of Equation (15.1), where the summations were carefully placed as far to the right as possible, so that program loops can be minimised. By doing this the algorithm is as efficient as possible, but it is still NP-hard. This is sometimes known as the variable elimination algorithm, which is a variation on the bucket elimination algorithm. The idea is to convert the conditional probability tables into what are called λ tables, which simply list all of the possible values for all variables, and which initially contain the conditional probabilities. For example, the λ table for the 'P' variable in Figure 15.2 is:

Copyrighted material

R	A	P	λ
T	T	T	0
T	T	F	1
T	F	T	0.8
T	F	F	0.2
F	T	T	0.6
F	T	F	0.4
F	F	T	1
F	F	F	0

If I see you panicking outside the exam (so that P is true), then I can eliminate it from the graph by removing from each table all rows that have P false in them, and deleting the P column. This simplifies things a little, but I have to do rather more in order to compute the probability of you having attended lectures. I don't know whether you revised or not, and I don't know if you found the lectures boring, so I have to marginalise over these variables. The order in which we marginalise doesn't change the correctness (although more advanced algorithms can improve the speed by taking advantage of conditional independence) so we'll pick R first. To eliminate it from the graph, we have to find all of the λ tables that contain it (there will be two of them containing R: the one for R itself and the one that we have just modified to remove P). To remove R, we have to add together the products of the λ values that correspond to places where the other values match. So to complete the entry where B is true and A is false, we have to multiply together the values where B, A, R are respectively true, false, true in the two tables and then add to that the product of where B, A, R are respectively true, false, false. In other words:

$$\begin{pmatrix} B & R & \lambda \\ T & T & 0.3 \\ T & F & 0.7 \\ F & T & 0.8 \\ F & F & 0.2 \end{pmatrix} \times \begin{pmatrix} R & A & \lambda \\ T & T & 0 \\ T & F & 0.8 \\ F & T & 0.6 \\ F & F & 1 \end{pmatrix} \Rightarrow \begin{pmatrix} B & A & \lambda \\ T & T & 0.3 \cdot 0 + 0.7 \cdot 0.6 = 0.42 \\ T & F & 0.3 \cdot 0.8 + 0.7 \cdot 1 = 0.94 \\ F & T & 0.8 \cdot 0 + 0.2 \cdot 0.6 = 0.12 \\ F & F & 0.8 \cdot 0.8 + 0.2 \cdot 1 = 0.84 \end{pmatrix} \quad (15.5)$$

We can do the same thing in order to eliminate B, which involves all three of the tables, and this will enable the computation of the conditional probability of you attending lectures given that I saw you panicking before the exam. The benefit of doing things this way is that the whole thing can be written as a general algorithm:

The Variable Elimination Algorithm

- create the λ tables:
  - for each variable v:
    - make a new table

Copyrighted material

        * for all possible true assignments x of the parent variables:
            · add rows for $P(v|x)$ and $1 - P(v|x)$ to the table
        * add this table to the set of tables

- eliminate known variables v:
    - for each table:
        * remove rows where v is incorrect
        * remove columm for v from table

- eliminate other variables (where x is the variable to keep):
    - for each variable v to be eliminated:
        * create a new table t'
        * for each table t containing v:
            · $v_{\text{true},t} = v_{\text{true},t} \times P(v|x)$
            · $v_{\text{false},t} = v_{\text{false},t} \times P(\neg v|x)$
        * $v_{\text{true},t'} = \sum_t (v_{\text{true},t})$
        * $v_{\text{false},t'} = \sum_t (v_{\text{false},t})$
    - replace tables t with the new one t'

- calculate conditional probability:
    - for each table:
        * $x_{\text{true}} = x_{\text{true}} \times P(x)$
        * $x_{\text{false}} = x_{\text{false}} \times P(\neg x)$
        * probability is $x_{\text{true}}/(x_{\text{true}} + x_{\text{false}})$

To see that these algorithms do not scale well, consider Figure 15.3, which shows a very simple development of the example in Figure 15.2 by adding just one extra node to the network: whether or not this is your final year ('F'). This makes the network significantly more complicated, since we need another table and extra entries in two of the other tables, and therefore the variable elimination algorithm will take rather longer to run.

15.1.2 Approximate Inference

Since the variable elimination algorithm will only take you so far, for reasonably sized Bayesian networks there is no choice but to perform approximate inference. Fortunately, we have already seen a set of algorithms that are ideally suited to the problem: the Markov Chain Monte Carlo methods that we saw in Chapter 14. There are two other methods of doing approximate inference (loopy belief propagation and mean field approximation) as well, but

Copyrighted material

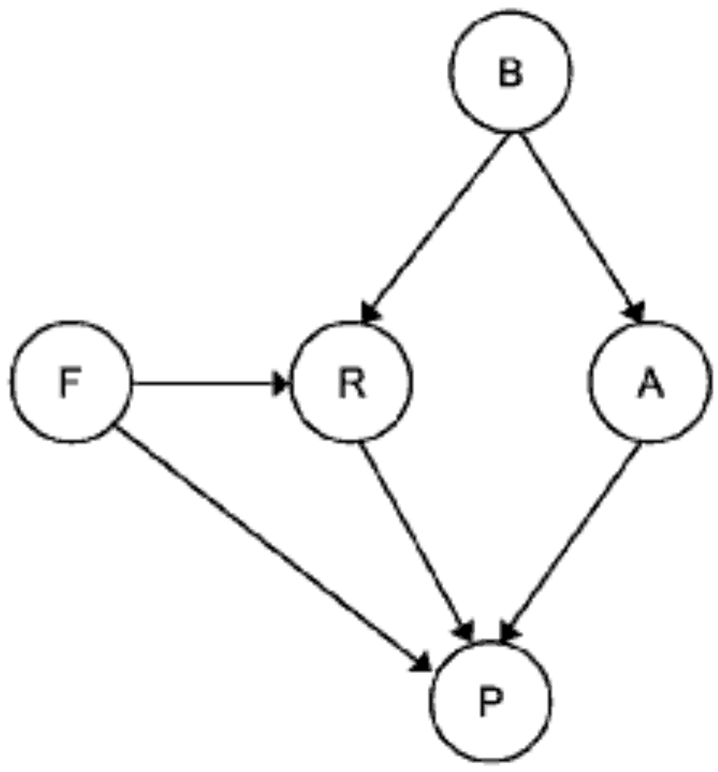


FIGURE 15.3: Adding just one extra node ('F', information about whether or not this is your final year) makes the conditional probability tables significantly more complicated.

we will not consider them further; there are references to descriptions of these methods at the end of the chapter.

The basic idea of using MCMC methods in Bayesian networks is to sample from the hidden variables, and then (depending upon the MCMC algorithm employed) weight the samples by their likelihoods. Creating the samples is very easy: for prediction, we start at the top of the graph and sample from each of the known probability distributions. Using Figure 15.2 again, we generate a sample from $P(b)$, and then use that value in the conditional probability tables for 'R' and 'A' to compute $P(r|b = \text{sample value})$ and $P(a|b = \text{sample value})$. These three values are then used to sample from $P(p|b, a, r)$. We can take as many samples as we like in this way, and expect that as the number of samples gets large, so the frequency of specific samples will converge to their expected values.

In this sampling method, we have to work through the graph from top to bottom and select rows from the conditional probability table that match the previous case. This is not what we would do if we were constructing the table by hand. Suppose that you wanted to know how many courses you did not attend the lectures for because the course was boring. You would simply look back through your courses and count the number of boring courses where you didn't go to lectures, ignoring all the interesting courses. We can use exactly this idea if we use rejection sampling (see Section 14.3). The method samples from the unconditional distribution and simply rejects any samples that don't have the correct prior probability. It means that we can sample from each distribution independently, and then throw away any samples that don't match the other variables. This is obviously computationally easier, but we might have to reject a lot of samples.

The solution to this problem is to work out what evidence we already have and use this evidence to assign likelihoods to the other variables that are

Copyrighted material

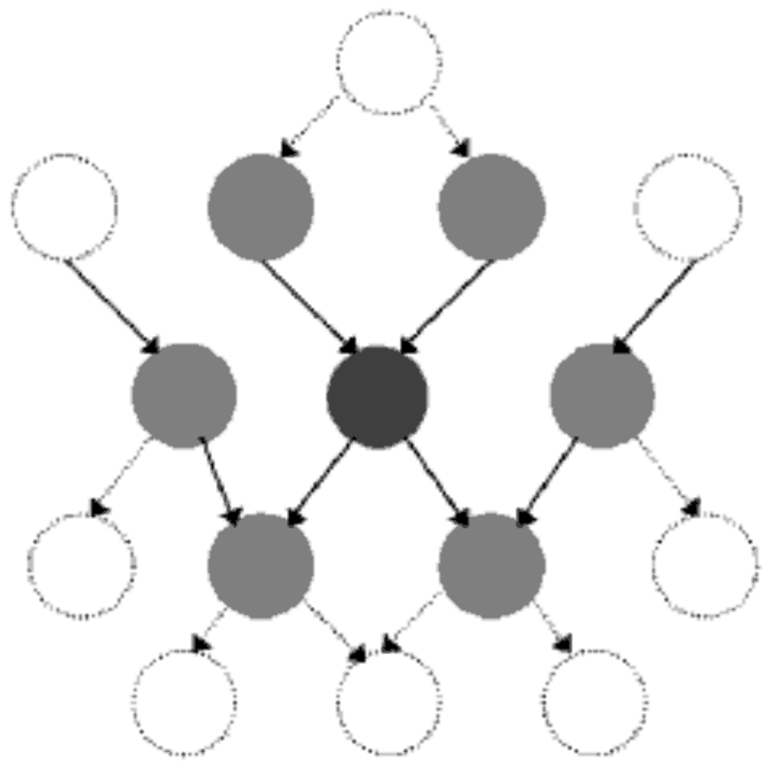


FIGURE 15.4: The Markov blanket of a node is the set of nodes (shaded light grey) that are either parents or children of the node, or other parents of its children (shaded dark grey).

sampled. Suppose that we sample from $P(b)$ and get value 'true'. If we already know that we did revise, then we weight the observation $P(r|b)$ by the appropriate probability, which is 0.3. We continue through the other variables, sampling where there is no evidence and using the tables to find the probability if we do have evidence. However, we can do rather better than this by using the full MCMC framework. We start by setting values for all of the possible probabilities, based on either evidence or random choices. This gives us an initial state for a Markov chain. Now Gibbs sampling (Section 14.4.4) will find us the maxima of our probability distribution given enough samples.

The probabilities in the network are:

$$p(x) = \prod_j p(x_j|x_{\alpha j}), \tag{15.6}$$

where $x_{\alpha j}$ are the parent nodes of x_j. In a Bayesian network, any given variable is independent of any node that is not their child, given their parents. So we can write:

$$p(x_j|x_{-j}) = p(x_j|x_{\alpha j}) \prod_{k \in \beta(j)} p(x_k|x_{\alpha(k)}), \tag{15.7}$$

where $\beta(j)$ is the set of children of node x_j and x_{-j} signifies all values of x_i except x_j. For any node we only need to consider its parents, its children, and the other parents of the children, as shown in Figure 15.4. This set is known as the Markov blanket of a node.

Given these calculations, computing the inference on any real Bayesian network generally consists of using Gibbs sampling in order to approximate the inference. For the exam panic example, the algorithm to perform the Gibbs sampling consists of computing the probability distributions (possibly

Copyrighted material

by using parts of the variable elimination algorithm) and then sampling from it:

```
for i in range(nsamples):
    # values contain current samples of b, r, a, p
    values = where(random.rand(4)<0.5,0,1)
    for j in range(nsteps):
        values=pb_rap(values)
        values=pr_bap(values)
        values=pa_brp(values)
        values=pp_bra(values)
    distribution[values[0]+2*values[1]+4*values[2]+8*values[3]]
    += 1
distribution /= nsamples
```

For the example, a sample distribution (based on 500 samples, with 10 iterations of each chain) is:

```
b r a p:   dist
1 1 1 1    0.0
1 1 1 0    0.086
1 1 0 1    0.038
1 1 0 0    0.052
1 0 1 1    0.048
1 0 1 0    0.116
1 0 0 1    0.274
1 0 0 0    0.0
0 1 1 1    0.0
0 1 1 0    0.088
0 1 0 1    0.068
0 1 0 0    0.114
0 0 1 1    0.03
0 0 1 0    0.076
0 0 0 1    0.01
0 0 0 0    0.0
```

15.1.3 Making Bayesian Networks

If we are given the structure and conditional probability tables of the Bayesian network, then we can perform inference on it by using Gibbs sampling or, if the network is simply enough, exactly. However, this raises the important question about where the Bayesian network itself comes from. Unfortunately, the news in this area isn't particularly good: the computational

Copyrighted material

Hidden page

and then you use data in order to compute the conditional probability tables. However, it is still difficult. The idea is to choose the probability distributions to maximise the likelihood of the training data. If there are no hidden nodes, then it is possible to compute the likelihood directly:

$$\begin{aligned} L &= \frac{1}{M} \log \prod_{m=1}^{N} P(D_m|G) \\ &= \frac{1}{M} \sum_{n=1}^{N} \sum_{m=1}^{M} \log P(X_n|\text{parents}(X_n), D_m), \end{aligned} \tag{15.8}$$

where M is the number of training data examples D_m, and X_n is one of the N nodes in graph G. Equation (15.8) has broken everything into sums over each node individually, which means that we can compute each separate conditional probability table. To compute the values of the table, you just need to count how often you have panicked in an exam given each of the possible values for having revised and attended lectures, and normalise it to make it into a probability. The danger with this is that with small amounts of data there could be examples that have not happened in training, and that will therefore have probability 0, although this can be dealt with by including prior probabilities and using Bayes' rule to update the estimates using the real data.

Obviously, this doesn't work if there are hidden nodes, since we don't know values for them in the data. Surprisingly, getting around this problem isn't as difficult as might be expected. The key is to see that if we did have values for them, then Equation (15.8) could be used. We can estimate values for them by inference, and then we can iterate these two steps: an estimation step using inference followed by a maximisation step, making this an EM algorithm (Section 8.3.1).

There is lots more work on Bayesian networks, and the references at the end of the chapter include entire books on the topic for anybody wishing to explore more in this area. We will now turn our attention to some other types of graphical models, starting with the variation where the edges are undirected.

15.2 Markov Random Fields

Bayesian networks are inherently asymmetric, since each edge had an arrow on it. If we remove this constraint, then there is no longer any idea of children and parent nodes. It also makes the idea of conditional independence that we saw for the Bayesian network easier: two nodes in a Markov Random Field

Copyrighted material

(MRF) are conditionally independent of each other, given a third node, if there is no path between the two nodes that doesn't pass through the third node. This is actually a variation on the Markov property, which is how the networks got their name: the state of a particular node is a function only of the states of its immediate neighbours, since all other nodes are conditionally independent given its neighbours. You might think that this fact would make inference on MRFs simpler, but unfortunately it doesn't; in general it is still a #P-hard problem. However, there are particular applications where MRF methods have turned out to be particularly useful, often for images.

The most well-known example is image denoising, something that we have already seen in Section 3.4.5 when we talked about auto-associative learning in the MLP. Suppose that we have a binary image I with pixel values $I_{x_i,x_j} \in \{-1, 1\}$. This image is a representation of an 'ideal' image I'_{x_i,x_j} that has no noise in it, which is what we want to recover. If we assume that the amount of noise is small, then there should be a good correlation between the values of each pixel in the two images, so I_{x_i,x_j} and I'_{x_i,x_j} should be correlated. We also assume that within a small 'patch' or region in an image, there is good correlation between pixels (so I_{x_i,x_j} should correlate well with I_{x_i+1,x_j} and its other neighbouring pixels (I_{x_i,x_j-1}, etc.). This assumption says that there are lots of places in the image where all of the pixels are of the same value, and this is (at least approximately) true for most images, and says that the pixels are correlated (and that other pixels in the image are conditionally independent of I_{x_i,x_j} given the neighbours of that pixel, which is the MRF bit).

The original theory of MRFs was worked out by physicists, initially by looking at the **Ising model**, which is a statistic description of a set of atoms connected in a chain, where each can spin up (+1) or down (-1) and whose spin affects those connected to it in the chain. Physicists tend to think of the energy of such systems, and argue that stable states are those with the lowest energy, since the system needs to get extra energy if it wants to move out of this state. For this reason, the jargon of MRFs is in terms of energies, and we therefore want the energy of our pair of images to be low when the pixels match, and higher when they do not. So we write the energy of the same pixel in two images as $-\eta I_{x_i,x_j} I'_{x_i,x_j}$, where η is a positive constant. Note that if the two pixels have the same sign then the energy is negative, while if they have opposite signs then the energy is positive and therefore larger. The energy of two neighbouring pixels is $-\zeta I_{x_i,x_j} I_{x_i+1,x_j}$, and we can just add these components together to get the total energy:

$$E(I, I') = -\zeta \sum_{i,j}^{N} I_{x_i,x_j} I_{x_i \pm 1, x_j \pm 1} - \eta \sum_{i,j=1}^{N} I_{x_i,x_j} I'_{x_i,x_j}, \qquad (15.9)$$

where the index of the pixels is assumed to run from 1 to N in both the x and y directions in both images and we are only interested in locally flat patches of the image we are changing, which is I.

Copyrighted material

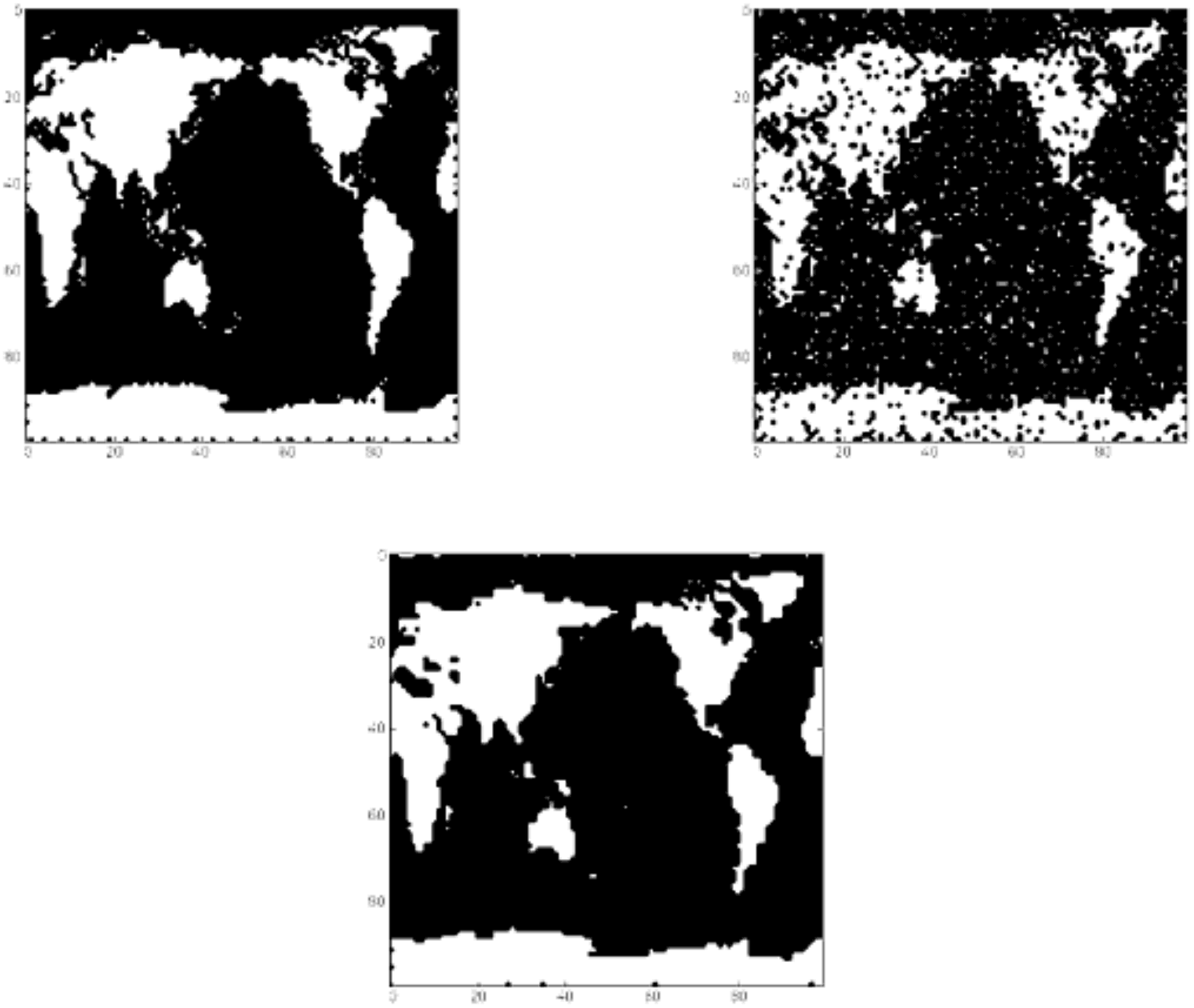


FIGURE 15.5: Using the MRF image denoising algorithm with $\eta = 2.1, \zeta = 1.5$ on a map of the world (*top left*) corrupted by 10% uniformly distributed random noise (*top right*) gives the image below which has about 1% error, although it has smoothed out the edges of all the continents.

There is now a simple iterative update algorithm, which is to start with noisy image I and ideal I', and update I so that at each step the energy calculation is lower. So you pick one pixel I_{x_i,x_j} for some values of x_i, x_j at a time, and compute the energies with this pixel being set to -1 and 1, picking the lower one. In probabilistic terms, we are making the probability $p(I, I')$ higher. The algorithm then moves on to another pixel, either choosing a random pixel at each step or moving through them in some pre-determined order, running through the set of pixels until their values stop changing. Figure 15.5 shows an original black and white image, a version corrupted with 10% noise, and the MRF-reconstructed version using parameters $\eta = 2.0, \zeta = 1.5$. This reduces the error from 10% to less than 1%, although it also removes my home country of New Zealand from the map!

The Markov Random Field Image Denoising Algorithm

- given a noisy image I and an original image I', together with parameters η, ζ:
- loop over the pixels of image I:

Copyrighted material

Hidden page

scared, or fine. I want to use these observations to try to work out what you did last night. The problem is that I don't know why you look the way you do, but I can guess by assigning probabilities to those things. So if you look hungover, then I might give probability 0.5 to the guess that you went to the pub last night, 0.25 to the guess that you went to a party, 0.2 to watching TV, and 0.05 to studying. In fact, we will use these the other way round, using the probability that you look hungover given what you did last night. These are known as observation or emission probabilities.

I don't have access to the other information that was used in Chapter 6, such as what parties are on and what other assignments you have (one of the worst things about stopping being a student is that the number of parties you get invited to drops off), but based on my own experience of being a student I can guess how likely parties are, etc., and knowing what student finances are, I can guess things like the probability of you going to the pub tonight if you went to the pub last night. So now it is just a question of putting these things into a form where I can work with them, and I can prepare my lectures according to how well you are working.

Each day that I see you in lectures I make an observation of your appearance, $o(t)$, and I want to use that observation to guess the state $\omega(t)$. This requires me to build up some kind of probabilities $P(o_k(t)|\omega_j(t))$, which is the probability that I see observation o_k (e.g., you are tired) given that you were in state ω_j (e.g., you went to a party) last night. These are usually labelled as b_{jk}. The other information that I have, or think I have, is the transition probability, which tells me how likely you are to be in state ω_j tonight given that you were in state ω_i last night. So if I think you were at the pub last night I will probably guess that the probability of you being there again tonight is small because your student loan won't be able to handle it. This is written as $P(\omega_j(t+1)|\omega_i(t))$ and is usually labelled as a_{ij}.

I can add one more constraint to each of the probability distributions a_{ij} and b_{ij}. I know that you did something last night, so $\sum_j a_{ij} = 1$ and I know that I will make some observation (since if you aren't in the lecture I'll assume you were too tired), so $\sum_k b_{jk} = 1$. There is one other thing that is generally assumed, which is that the Markov chain is ergodic, something that we saw in Section 14.4.1: it means that there is a non-zero probability of reaching every state eventually, no matter what the starting state.

After a couple of weeks of the course I have made observations about you, and I am ready to sort out my HMM. There are three things that I might want to do with the data:

- see how well the sequence of observations that I've made match my current HMM (Section 15.3.1)
- work out the most probable sequence of states that you've been in based on my observations (Section 15.3.2)

Copyrighted material

- given several sets of observations (for example, by watching several students) generate a good HMM for the data (Section 15.3.3)

We will start by assuming that I invent a model and want to see how good it is. So I use my own knowledge of being a student to work out the probability distributions and then I can test the observations I make of you against my model. At this point I will probably find out that my student life was different to yours, or things have changed since I was a student, and I will have to generate a new model to match current data. I can then use this improved model to work out what you've been doing each evening. These problems are dealt with in the next three sections.

The HMM itself is made up of the transition probabilities a_{ij} and the observation probabilities b_{jk}. So these are the things that I need to specify for myself, starting with the transition probabilities (which are also shown in Figure 15.7):

	Previous night			
	TV	Pub	Party	Study
TV	0.4	0.6	0.7	0.3
Pub	0.3	0.05	0.05	0.4
Party	0.1	0.1	0.05	0.25
Study	0.2	0.25	0.2	0.05

and then the observation probabilities:

	TV	Pub	Party	Study
Tired	0.2	0.4	0.3	0.3
Hungover	0.1	0.2	0.4	0.05
Scared	0.2	0.1	0.2	0.3
Fine	0.5	0.3	0.1	0.35

15.3.1 The Forward Algorithm

Suppose that I see the following observations: O = (tired, tired, fine, hungover, hungover, scared, hungover, fine) and I want to work out the likely run of states that generated it. The probability that my observations $O = \{o(1), \ldots, o(T)\}$ come from the model can be computed using simple conditional probability. I know you were doing something last night, so for an observation $o(t)$ =tired (say) I just need to compute the probability that I made that observation given you were in a particular state (say watching TV) and multiply it by the probability that you were in that state given the state I thought you were in the night before (say partying). So for the example, I compute the probability that you were tired given that you were watching TV, which is 0.2, and then multiply it by the probability that you spent last night watching TV given that I thought you were partying the night before, which is 0.1. So this yields probability 0.02 for this particular state change.

Copyrighted material

Hidden page

$$P(O) = \sum_{r=1}^{R} \prod_{t=1}^{T} P(o_k(t)|\omega_j(t)) P(\omega_j(t)|\omega_i(t-1))$$
$$= \sum_{r=1}^{R} \prod_{t=1}^{T} b_{jk} a_{ij}. \tag{15.13}$$

This looks fairly easy now. The only problem is in that sum over r, which runs over all possible sequences of hidden states. If there are N hidden states then there are N^T possible sequences, and for each one we have to compute a product of T probabilities. Not only will these probabilities be incredibly small, but the computational cost of getting them will be astronomical: $\mathcal{O}(N^T T)$.

Fortunately, the Markov property comes to our rescue again. Since the probability of each state only depends on the data at the current and previous timestep $(o(t), \omega(t), \omega(t-1))$ we can build up our computation of $P(O)$ one timestep at a time. This is known as the forward trellis by some people, since it looks like a garden trellis in Figure 15.8. To construct the trellis we introduce a new variable $\alpha_i(t)$ that describes the probability that at time t the state is ω_i and that the first $(t-1)$ steps all matched the observations $o(t)$:

$$\alpha_j(t) = \begin{cases} 0 & t = 0, j \neq \text{initial state} \\ 1 & t = 0, j = \text{initial state} \\ \sum_i \alpha_i(t-1) a_{ij} b_{j(o_t)} & \text{otherwise.} \end{cases} \tag{15.14}$$

where $b_{j(o_t)}$ means the particular transition probability for output o_t. This ensures that only the observation probability that has the index that matches the observation o_t contributes to the sum. Computing $P(O)$ now requires only $\mathcal{O}(N^2 T)$, which is a substantial improvement, in a very simple algorithm:

The HMM Forward Algorithm

- for each observation in order o_t, $t = 1, \ldots, T$
  - for each possible state s
    - $\alpha_{s,t} = b_{s,o_t} * \sum_x (\alpha_{x,t} * a_{x,s})$

Let's look at the first two states of our example HMM. In both, the observation is that you were tired, so we need to compute $\alpha_i(t)$ and so construct the trellis. Figure 15.8 shows the idea, with the initial $\alpha_1(\cdot)$ coming from my guesses about how likely each state is, and we just need to run through the set of computations to compute $\alpha_2(\cdot)$ and so on, getting the numbers that are shown in the figure. We then repeat this for the next step to get the $\alpha_3(\cdot)$ and so on until we reach the final state. At this stage we can sum up all

Copyrighted material

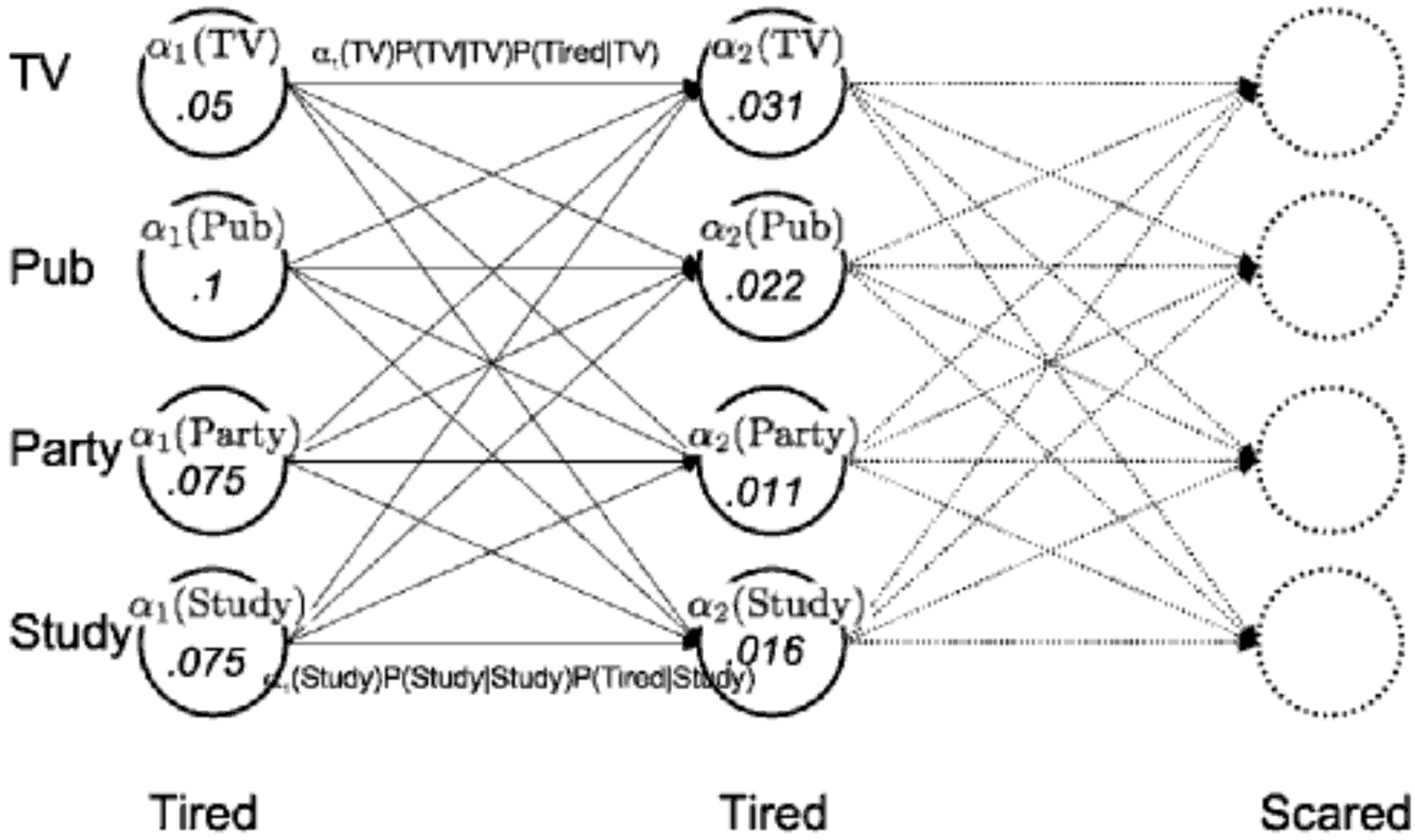


FIGURE 15.8: The forward trellis for the first two observations of the example HMM.

of the possible probabilities, which tells me in this case that you were most likely watching TV last night. We will also need to be able to go backwards through the trellis, which is a very similar algorithm that works backwards to compute β values, also based on the transmission probability and observation probability matrices.

15.3.2 The Viterbi Algorithm

The next problem that we want to solve is the decoding problem: I can use my model of how students are expected to behave and match them with my observations to guess what you have been doing each evening. The algorithm is known as the Viterbi algorithm after its creator, although he actually derived it for error correction, a completely different application! It works on very similar principles to the forward algorithm, except that we act in a greedy way, since for each timestep we pick the state that is most likely as the next step in the path, rather than maintaining probabilities of all possible paths.

The HMM Viterbi Algorithm

- for each observation in order o_t, $t = 1 \ldots T$
  - for each possible state s
    - $v_{s,t} = \max_x(v_{x,t-1} * a_{x,s} * b_{s,o_t})$
  - $\text{path}_t = \arg\max_t(v_{x,t})$

Figure 15.9 shows the path for the first three states of the example.

Copyrighted material

Hidden page

ples. We can then use these forwards and backwards estimates to compute transition probabilities. It works like this. Suppose we want to compute the probability of a transition between state ω_i at time t and ω_j at time $t+1$. We first use our current model to run forwards via α to get to state ω_i at time t, and run backwards to get to state ω_j at time $t+1$ via β. Then we use the current estimates of a_{ij} and b_{jk}. The only other thing that we need is to realise that we might have got there by some other path through the states, so we need to normalise this calculation by how likely this particular training sequence is according to the current model, which is $P(O|a_{ij}, b_{jk})$. This value is usually called γ_{ij} and written out in all its glory looks like:

$$\gamma_{ij}(t) = \frac{\alpha_i(t-1)a_{ij}b_{jk}\beta_j(t)}{P(O|a_{ij}, b_{jk})}. \tag{15.16}$$

What is the meaning of $\sum_{t=1}^{T} \gamma_{ij}(t)$? It tells us how many times we can expect to transition from state ω_i to state ω_j at any time in the sequence. This is the expectation that we are using within this EM algorithm. We just need to work out how to do the maximisation bit. We know how many times we can expect to do the ω_i to ω_j transition, but a_{ij} is concerned with the probability of this event. So we need to divide this number by the number of times we expect to transition out of state ω_i, regardless of where we end up. This is $\sum_{t=1}^{T} \sum_m \gamma_{im}(t)$. The update rule for a_{ij} is simply the ratio of these:

$$a_{ij} = \frac{\sum_{t=1}^{T} \gamma_{ij}(t)}{\sum_{t=1}^{T} \sum_m \gamma_{im}(t)}. \tag{15.17}$$

The update rule for b_{jk} is similar, except that we need to think about the frequency that an observation o_k is made in state j compared to any other symbol:

$$b_{jk} = \frac{\sum_{t=1, o(t)=o_k}^{T} \sum_m \gamma_{km}(t)}{\sum_{t=1}^{T} \sum_m \gamma_{jm}(t)}. \tag{15.18}$$

This leads to the complete Baum-Welch algorithm:

The HMM Baum-Welch (Forward-Backward) Algorithm

- while updates have not converged:
  - **E-step:**
  - Compute forwards and backwards steps (α and β)
  - for each observation in order $o_t, t = 1 \ldots T$

Copyrighted material

    * for each possible pair of states s and σ:
        · $\gamma_{\sigma,s,t} = \alpha_{\sigma,t} * a_{\sigma,s} * \beta_{s,t+1} * b_{s,o(t+1)} / \max_x(\alpha_{x,T-1})$
- **M-step:**
- for each possible pair of states s and σ:
    * $a_{s,\sigma} = \sum_x \gamma_{s,\sigma,x} / \sum_y \sum_z \gamma_{s,y,z}$
- for each observation o:
    * for each state s:
        · tally $= \sum_x \sum_y \gamma_{s,x,y}$
        · $b_{s,o}$ =sum(tally where observation o was seen)/total tally

---

Since this is the most important algorithm, here is a Python implementation as well:

```
def BaumWelch(obs,nStates):

    T = shape(obs)[0]
    a = random.rand(nStates,nStates)
    b = random.rand(nStates,T)
    olda = zeros((nStates,nStates))
    oldb = zeros((nStates,T))
    maxCount = 50
    tolerance = 1e-5

    count = 0
    while (abs(a-olda)).max() > tolerance and
     (abs(b-oldb)).max() > tolerance and count < maxCount:
        # E-step

# Compute the forward and backward steps
        alpha = HMMfwd(a,b,obs)
        beta = HMMbwd(a,b,obs)
        gamma = zeros((nStates,nStates,T))

        for t in range(T-1):
          for s in range(nStates):
            gamma[:,s,t] = alpha[:,t] * a[:,s]
             ... * b[s,obs[t+1]] * beta[s,t+1]
             / max(alpha[:,T-1])

    # M-step
    olda = a.copy()
    oldb = b.copy()
```

Copyrighted material

```
            for i in range(nStates):
                for j in range(nStates):
                    a[i,j]=sum(gamma[i,j,:])/sum(sum(gamma[i,:,:]))

            for o in range(max(obs)):
                for j in range(nStates):
                    places = (obs==o).nonzero()
                    tally = sum(gamma[j,:,:],axis=0)
                    b[j,o]=sum(tally[places])/sum(sum(gamma[j,:,:
                    ]))

                count += 1
        return a,b
```

We can't really use this algorithm very well on the simple example, since we would need rather more data to do justice to the training. However, if we do apply it and then compute the Viterbi path, then it gives the same answer as with the invented data. That pretty much sums it up for the HMM. It is worth mentioning two limitations of it, which are that the probability distributions are not time dependent, and that the probabilities can get very small. The second of these problems is an implementation detail that needs careful monitoring, while the first can be dealt with by using more general graphical models, although with the additional computational costs that come with that.

15.4 Tracking Methods

We will now look at two methods of performing tracking. You perform tracking fairly easily, keeping tabs on where something is and how it is moving. This has an obvious evolutionary benefit, since keeping track of where predators were and whether they were coming towards you could keep you alive. It is also useful for a machine to be able to do this, both for similar reasons to a human or animal (watching something moving and predicting what path it will follow, for example in radar or other imaging method) and to keep track of a changing probability distribution. We will look at two methods of doing it, the Kalman filter and the particle filter.

Copyrighted material

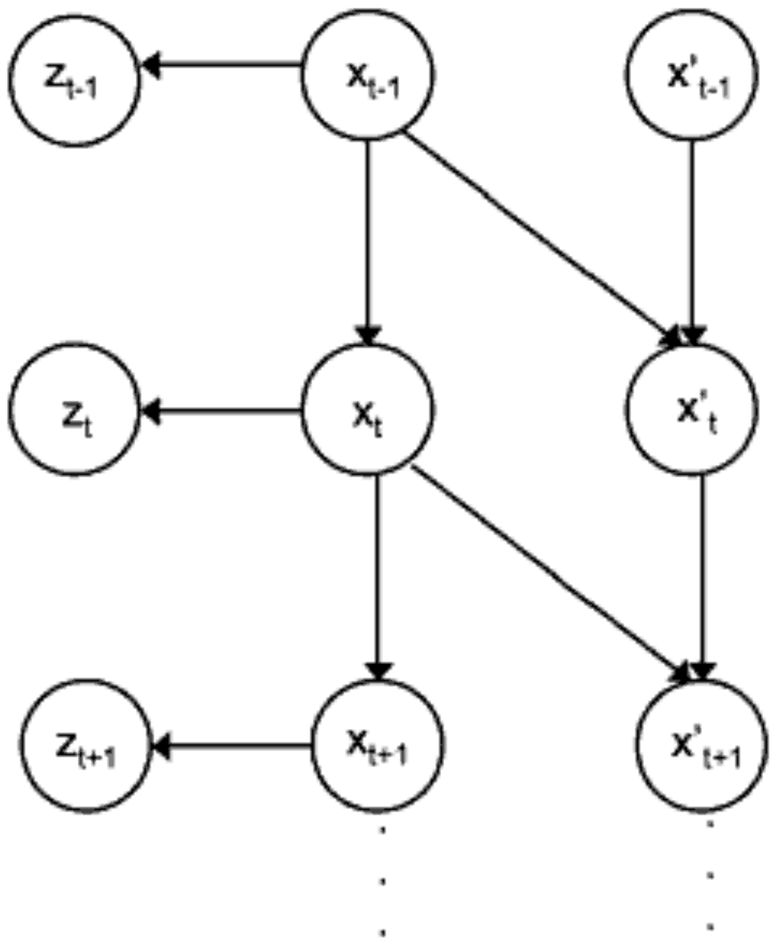


FIGURE 15.10: A representation of the Kalman filter as a graphical model.

15.4.1 The Kalman Filter

The Kalman filter (named for E. Kalman; although he was not the original inventor he did do quite a lot of work on it) is a recursive estimator. It makes an estimate of the next step, then computes an error term based on the value that was actually produced in the next step, and tries to correct it. It then uses both of those to make the next prediction, and iterates this procedure. It can be seen as a simple cycle of predict-correct behaviour, where the error at each step is used to improve the estimate at the next iteration. The Kalman filter can be represented by the graphical model shown in Figure 15.10.

Much of the jargon that is associated with the Kalman filter is familiar to us: the state, which is hidden, consists of the variables that we want to know, which we see through noisy observations over time. There is a transition model that tells us how states change from one to another, and an observation model (also called the sensor model here) that tells us how states lead to observations.

The principal simplifying assumptions of the Kalman filter are that the process is linear and that all of the distributions are Gaussian with constant covariance. Since the convolution of Gaussians is also Gaussian, this means that we can put them together to form new Gaussians, and so the model stays well behaved. This was a significant advantage over previous methods of tracking, which tended to stop working fairly quickly, since the estimates broke down because the probability distribution stopped being well-defined. We assume that both the transition model and the observation model are Gaussians with means based on the previous observations, and fixed covariances $\mathbf{Q}$ and $\mathbf{R}$:

$$P(\mathbf{x}_{t+1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t+1}|\mathbf{A}\mathbf{x}_t, \mathbf{Q}) \tag{15.19}$$

$$P(\mathbf{z}_{t+1}|\mathbf{x}_{t+1}) = \mathcal{N}(\mathbf{z}_{t+1}|\mathbf{H}\mathbf{x}_{t+1}, \mathbf{R}), \tag{15.20}$$

Copyrighted material

where $\mathbf{A}$ and $\mathbf{H}$ are matrices that provide a (constant) represention of the probability models, just as we had for the HMM above.

The idea of the Kalman filter is to make a prediction and then correct it when the next observation is available, i.e., at the next timestep. The predicted observation is $\hat{\mathbf{z}}_{t+1} = \mathbf{HAx}_{t+1}$ (by following the equations above) and so the error is $\mathbf{z}_{t+1} - \mathbf{HAx}_{t+1}$. The predicted covariance matrix that goes with it is $\hat{\mathbf{\Sigma}}_{t+1} = \mathbf{A\Sigma}_t\mathbf{A}^T + \mathbf{Q}$. The Kalman filter weights these error computations by how much trust the filter currently has in its predictions; these weights are known as the Kalman gain and are computed by:

$$\mathbf{K}_{t+1} = \hat{\mathbf{\Sigma}}_{t+1}\mathbf{H}^T \left(\mathbf{H}\hat{\mathbf{\Sigma}}_{t+1}\mathbf{H}^T + \mathbf{R}\right)^{-1}. \tag{15.21}$$

So the update for the estimate is:

$$\mathbf{x}_{t+1} = \hat{\mathbf{x}}_{t+1} + \mathbf{K}_{t+1}\left(\mathbf{z}_{t+1} - \mathbf{H}\hat{\mathbf{x}}_{t+1}\right). \tag{15.22}$$

All that is then required is to update the covariance estimate:

$$\mathbf{\Sigma}_{t+1} = (\mathbf{I} - \mathbf{K}_{t+1}\mathbf{H})\hat{\mathbf{\Sigma}}_{t+1}, \tag{15.23}$$

where $\mathbf{I}$ is the identity matrix of the relevant size. Putting these equations together leads to a simple algorithm.

The Kalman Filter Algorithm

- given an initial estimate $\mathbf{x}(0)$
- for each timestep:
  - **predict the next step**
    - predict state as $\hat{\mathbf{x}}_{t+1} = \mathbf{Ax}_t$
    - predict covariance as $\hat{\mathbf{\Sigma}}_{t+1} = \mathbf{A\Sigma}_t\mathbf{A}^T + \mathbf{Q}$
  - **update the estimate**
    - compute the error in the estimate, $\boldsymbol{\epsilon} = \mathbf{z}_{t+1} - \mathbf{HAx}_{t+1}$
    - compute the Kalman gain using Equation (15.21)
    - update the state using Equation (15.22)
    - update the covariance using Equation (15.23)

Figure 15.11 shows a simple 1D example of using the Kalman filter. The dots are noisy estimates of a process, and the line is the Kalman filter estimate, with the dashed lines being the possible error. It can be seen that the initial estimate was not very good, but the algorithm quickly converges to a good estimate of the mean of the data, and the error drops accordingly.

Now that we have seen the Kalman filter in action, we need to work out how to use it for tracking. If we write the position of the object at time t

Copyrighted material

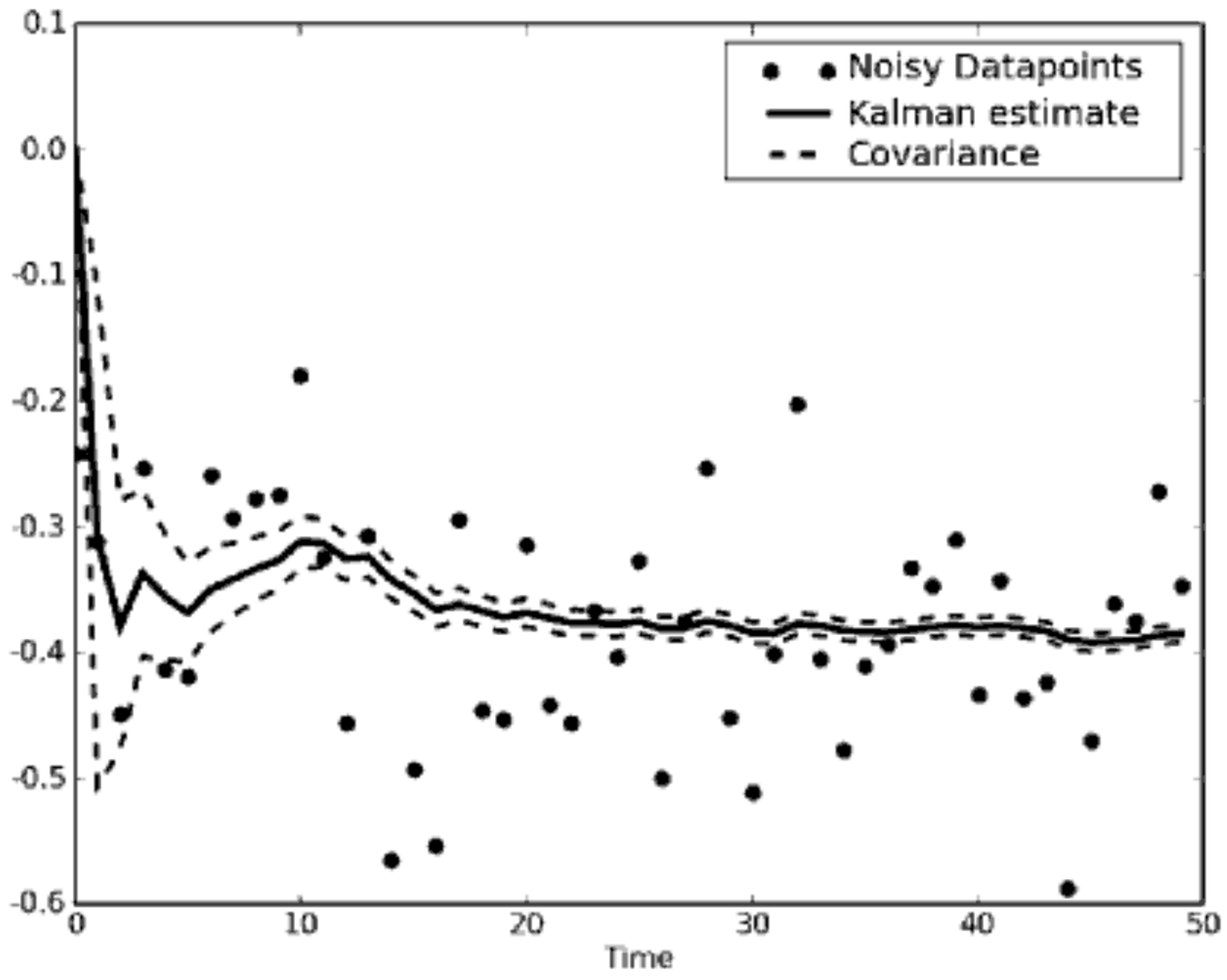


FIGURE 15.11: Estimates of a 1D noisy process using a Kalman filter.

as $\mathbf{y}_t$, then its velocity is the time derivative of its position, $\dot{\mathbf{y}}_t$. Now, the position at time $t + \Delta t$ is $\mathbf{y}_t + \dot{\mathbf{y}}\Delta t$. We are using the Kalman filter to keep track of both the position and the velocity of the object, so we write the state as:

$$\mathbf{x}_t = \begin{pmatrix} \mathbf{y} \\ \dot{\mathbf{y}} \end{pmatrix}. \tag{15.24}$$

The update equations will therefore be:

$$\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{a}_{t+1}, \tag{15.25}$$

where the acceleration $\mathbf{a}_t$ is a Gaussian random variable with mean $\mathbf{0}$ and variance $\boldsymbol{\sigma}$, and:

$$\mathbf{A} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \quad \mathbf{B} = \begin{pmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{pmatrix}. \tag{15.26}$$

The same Kalman filter algorithm can then be used to perform the tracking. A 1D example of such tracking is shown in Figure 15.12, where an object is moving in one spatial dimension and time. The Kalman filter does a good job of following the object.

One of the main assumptions of the Kalman filter was that the process was linear. Where this is not true it is possible to linearise about the current estimate $(\mathbf{x}_t, \boldsymbol{\Sigma}_t)$ and this leads to the Extended Kalman Filter, where the main change is that you have to compute the Jacobian of the covariance matrix and use that in the update rule. This is not an optimal estimator, and there have

Copyrighted material

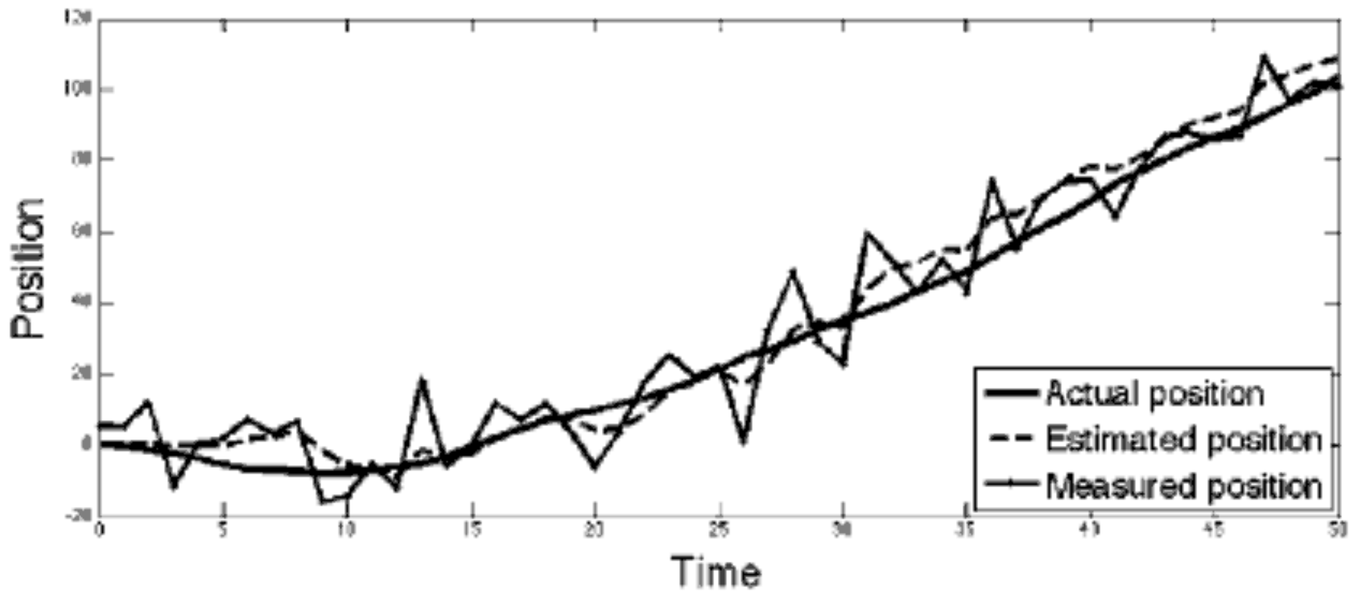


FIGURE 15.12: Demonstration of the Kalman filter tracking an object moving in one spatial dimension.

been various attempts to improve upon it; the most successful have been to use sampling, and the leads to the Unscented Kalman Filter, which uses a particular sampling technique known as the unscented transform. For more information on this, see the Further Reading section; instead we will look at a common MCMC algorithm for performing tracking, the particle filter.

15.4.2 The Particle Filter

There was one assumption that the Kalman filter made that we did not challenge at all, which was that the distributions were Gaussian. If this is not the case then we cannot use an algorithm like the Kalman filter, since the distributions would stop behaving as probabilities very quickly. In order to get around this problem, we return to the methods that have underpinned many of the algorithms in this chapter: sampling. The particular sampling technique that we will use is the sampling-importance-resampling algorithm of Section 14.3, which forms the basis of the particle filter, or condensation method. This is a relatively recent development, and has been finding many successful applications in tracking, including in image and signal analysis. The idea is to use sampling to keep track of the state of the probability distribution. This is actually known as sequential sampling, since we are using a set of samples for time t to estimate the process at time $t + 1$, and then resampling from there.

One benefit of sampling methods is that we don't have to hold on to the Markov assumption. In tracking, prior history can be useful, which means that the Markov assumption is a bad one. The proposal distribution is generally written as $q(\mathbf{x}_{t+1}|\mathbf{x}_{0:t}, \mathbf{z}_{0:t})$ to make this dependence clear, and the proposal distribution that is generally used in the estimated transition probabilities $p(\hat{\mathbf{x}}_{t+1}|\mathbf{x}_{0:t}, \mathbf{z}_{01:t})$, since it is a simple distribution that is related to the process. With this decided, there is nothing else to the particle filter. Figure 15.13 shows it keeping track of a distribution based on a noisy sinusoidal function. The plot on the right of the figure shows the particles that were created at each iteration.

Copyrighted material

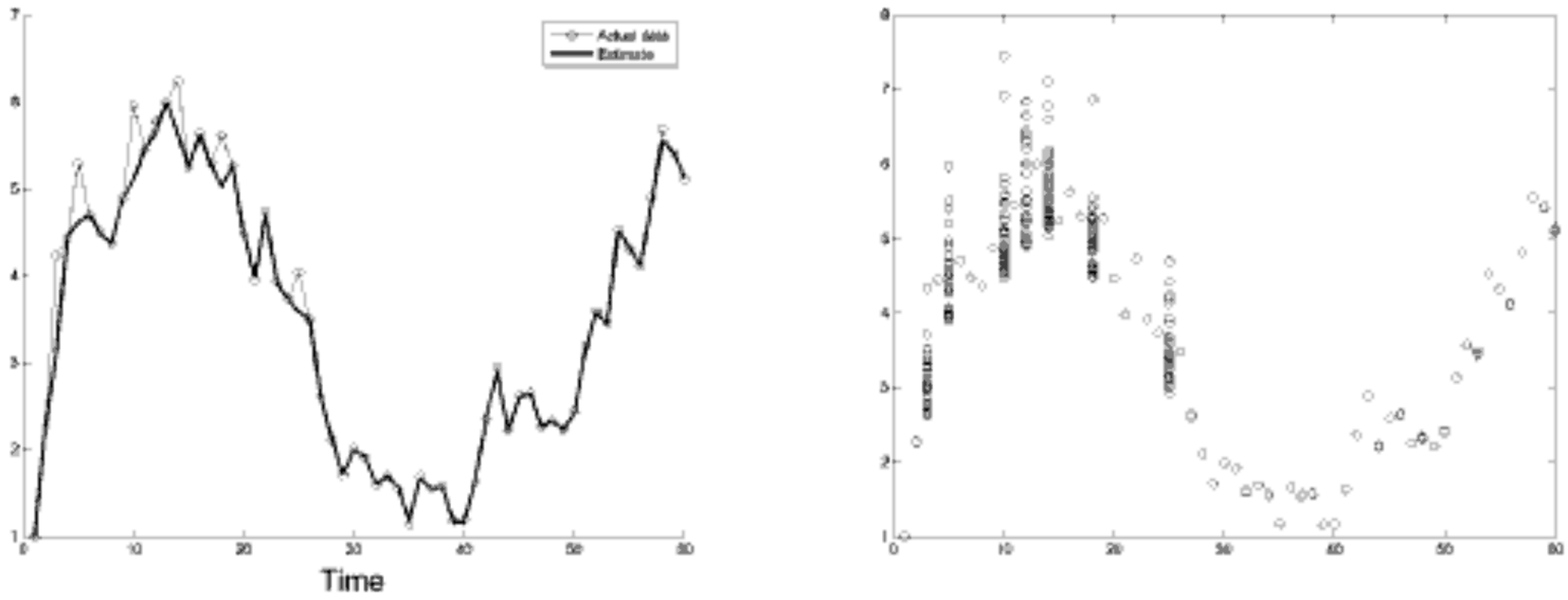


FIGURE 15.13: *Left:* A set of (noisy) data from a noisy distribution based on a sinusoid, and the results of a particle filter tracking the distribution. *Right:* The particles that were sampled at each iteration.

Further Reading

Graphical models are a growth area at the moment, with lots of interesting research being done in the area. The original work in the area, and the motivations for it, are described in:

- J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference.* Morgan Kaufmann, San Mateo, CA, USA, 1988.

Alternative overviews of Bayesian networks can be found in the following papers and books, the last of which is a collection of papers that provides a good overview of the area:

- W.L. Buntine. Operations for learning with graphical models. *Journal of Artificial Intelligence Research*, 2: 159–225, 1994.

- D. Husmeier. Introduction to learning Bayesian networks from data. In D. Husmeier, R. Dybowski, and S. Roberts, editors, *Probabilistic Modelling in Bioinformatics and Medical Informatics*, Springer, Berlin, Germany, 2005.

- Chapters 8 and 13 of C.M. Bishop. *Pattern Recognition and Machine Learning.* Springer, Berlin, Germany, 2006.

- M.I. Jordan, editor. *Learning in Graphical Models.* MIT Press, Cambridge, MA, USA, 1999.

In the area of Markov Random Fields, the image denoising example comes from:

Copyrighted material

- S. Geman and D. Geman. Stochastic relaxation, Gibbs distributions and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6:721–741, 1984.

Markov Random Fields are most commonly used in imaging. There are good overviews in:

- P. Pèrez. Markov random fields and images. *CWI Quarterly*, 11(4): 413–437, 1998.
- R. Kindermann and J.L. Snell. *Markov Random Fields and Their Applications.* American Mathematical Society, Providence, RI, USA, 1980.

For more details on the Hidden Markov Model, the Kalman filter and the particle filter, you might want to look at:

- L.R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–268, 1989.
- Z. Ghahramani. An introduction to Hidden Markov Models and Bayesian networks. *International Journal of Pattern Recognition and Artificial Intelligence*, 15:9–42, 2001.
- G. Welch and G. Bishop. An introduction to the Kalman filter, 1995. URL `http://www.cs.unc.edu/~welch/kalman/`. Technical Report TR 95-041, Department of Computer Science, University of North Carolina at Chapel Hill, USA.
- M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002.

A more rigorous treatment is given in:

- Chapters 8 and 13 of C.M. Bishop. *Pattern Recognition and Machine Learning.* Springer, Berlin, Germany, 2006.

Practice Questions

Problem 15.1 Compute the probability of taking notes (N) in the Bayesian network shown in Figure 15.14. The problem describes the chance of you taking notes in the lecture or sleeping (S) according to whether or not the course was boring (B) based on whether or not the professor is

Copyrighted material

Hidden page

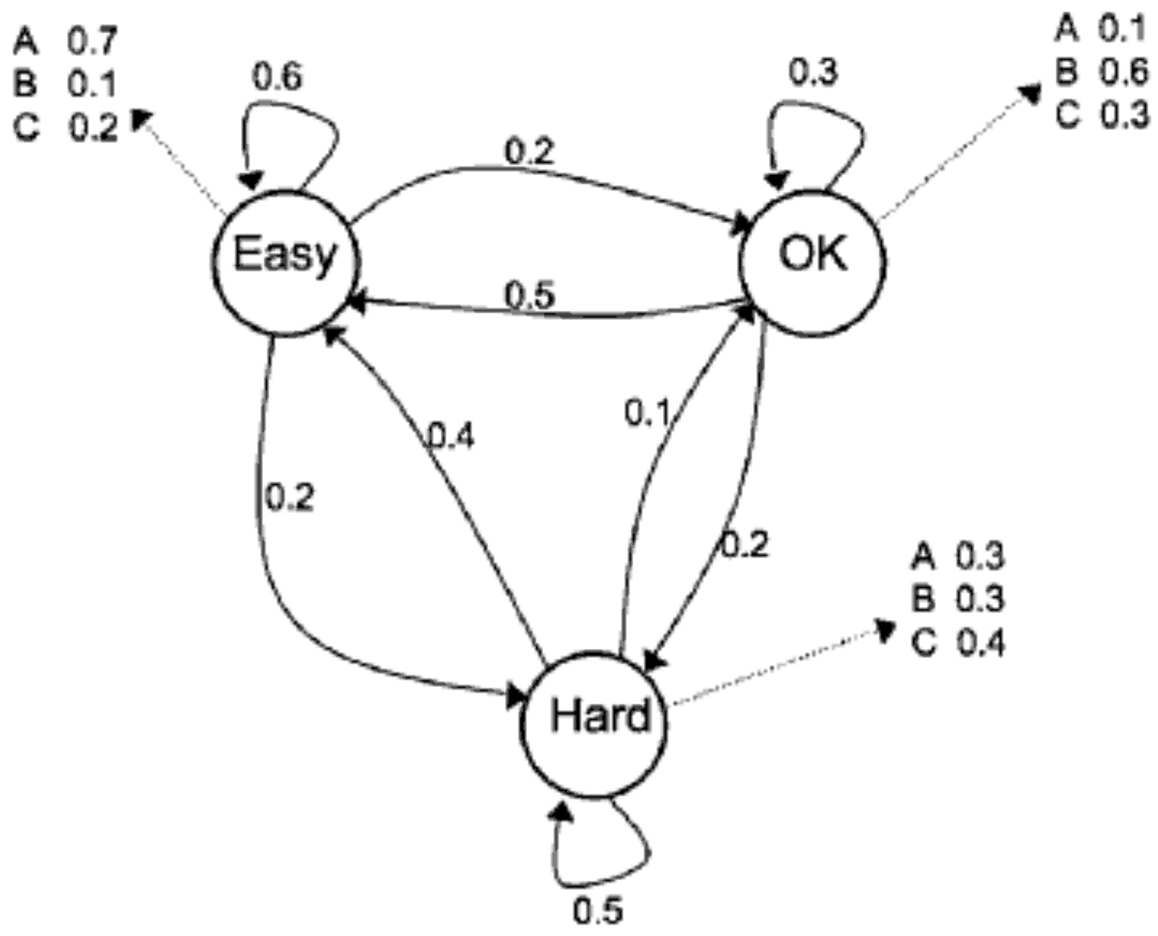


FIGURE 15.15: The Hidden Markov Model example for Problem 15.3.

Copyrighted material

Hidden page

Hidden page

same object. This might not seem important, but Python works by reference, which means that the command `>>> a = b` does not put a copy of the value of `b` into `a`, but rather assigns to `a` a reference to the variable `b`. This can be a trap for the unwary, as will be discussed more shortly. The normal logical operators are slightly unusual in Python, with the normal logical operators using the words `and, or` and `not`; the symbols `&, |` perform bit-wise and/or. These bit-wise operators are actually quite useful, as we'll see later.

In addition to integer and floating point representations of numbers, Python also deals with strings, which are described by using single or double quotes (' or ") to surround them: `>>> b = 'hello'`. For strings, the `+` operator is overloaded (given a new meaning), which is concatenation: merging the strings. So `>>> 'a' + 'd'` returns the new string `'ad'`.

Having made the basic data types, Python then allows you to combine them into three different basic data structures:

Lists A list is a combination of basic data types, surrounded by square brackets. So `>>> mylist = [0, 3, 2, 'hi']` is a perfectly good list that contains integers and a string. This ability to store different types inside a list gives you a hint that Python handles lists differently to the way other languages handle arrays. This comes about because Python is inherently object-oriented, so that every variable that you make is simply an object, and so a list is just a collection of objects. This is why the type of the object does not matter. It also means that you can have lists of lists without a problem: `>>> newlist = [3, 2, [5, 4, 3], [2, 3, 2]]`.

Accessing particular elements of a list simply requires giving it an index. Like C, but unlike MATLAB, Python indices start at 0, so `>>> newlist[0]` returns the first element (3). You can also index from the end using a minus sign, so `>>> newlist[-1]` returns the last element, `>>> newlist[-2]` the last-but-one, etc. The length of a list is given by `len`, so `>>> len(newlist)` returns 4. Note that `>>> newlist[3]` returns the list in the 4th location of `newlist` (i.e., `[2, 3, 2]`). To access an element of that list you need an extra index: `>>> newlist[3][1]` returns 3.

A useful feature of Python is the slice operator. This is written as a colon (`:`) and enables you to access sections of a list easily, such as `>>> newlist[2:4]` which returns the elements of `newlist` in positions 2 and 3 (the arguments you use in a slice are inclusive at the start and exclusive at the end, so the second parameter is the first index that is excluded). In fact, the slice can take three operators, which are [start:stop:step], the third element saying what stepsize to use. So `>>> newlist[0:4:2]` returns the elements in locations 0 and 2, and you can use this to reverse the order of a list: `>>> newlist[::-1]`. This last example shows a couple of other refinements of the slice operator: if

Copyrighted material

you don't put a value in for the first number (so it looks like `[:3]`) then the value is taken as 0, and if you don't put a value for the second operator (`[1:]`) then it is taken as running to the end of the list. These can be very useful, especially the second one, since it avoids having to calculate the length of the list every time you want to run through it. `>>> newlist[:]` returns the whole string.

This last use of the slice operator, returning the whole string, might seem useless. However, because Python is object-oriented, all variable names are simply **references** to objects. This means that copying a variable of type `list` isn't as obvious as it could be. Consider the following command: `>>> alist = mylist`. You might expect that this has made a copy of `mylist`, but it hasn't. To see this, use the following command `>>> alist[3] = 100` and then have a look at the contents of `mylist`. You will see that the 3rd element is now 100. So if you want to copy things you need to be careful. The slice operator lets you make actual copies using: `>>> alist = mylist[:]`. Unfortunately, there is an extra wrinkle in this if you have lists of lists. Remember that lists work as references to objects. We've just used the slice operator to return the values of the objects, but this only works for one level. In location 2 of `newlist` is another list, and the slice operator just copied the reference to that embedded list. To see this, perform `>>> blist = newlist[:]` and then `>>> blist[2][2] = 100` and have a look at `newlist` again. What we've done is called a **shallow copy**, to copy everything (known as a **deep copy**) requires a bit more effort. There is a `deepcopy` command, but to get to it we need to `import` the `copy` module using `>>> import copy` (we will see more about importing in Section 16.3.1). Now we can call `>>> clist = copy.deepcopy(newlist)` and we finally have a copy of a complete list.

There are a variety of functions that can be applied to lists, but there is another interesting feature of the fact that they are objects. The functions (methods) that can be used are part of the object class, so they modify the list itself and do not return a new list (this is known as working **in place**). To see this, make a new list `>>> list = [3, 2, 4, 1]` and suppose that you want to print out a list of the numbers sorted into order. There is a function `sort()` for this, but the obvious `>>> print list.sort()` produces the output `None`, meaning that no value was returned. However, the two commands `>>> list.sort()` followed by `>>> print list` do exactly what is required. So functions on lists modify the list, and any future operations will be applied to this modified list.

Some other functions that are available to operate on lists are:

`append(x)` adds `x` to the end of the list

`count(x)` counts how many times `x` appears in the list

Copyrighted material

`extend(L)` adds the elements in list `L` to the end of the original list

`index(x)` returns the index of the first element of the list to match `x`

`insert(i, x)` inserts element `x` at location `i` in the list, moving everything else along

`pop(i)` removes the item at index `i`

`remove(x)` deletes the first element that matches `x`

`reverse()` reverses the order of the list

`sort()` we've already seen

You can compare lists using `>>> a==b`, which works elementwise through the list, comparing each element against the matching one in the second list, returning True if the test is true for each pair (and the two lists are the same length), and False otherwise.

Tuples A tuple is an immutable list, meaning that it is read-only and doesn't change. Tuples are defined using round brackets, e.g., `>>> mytuple = (0, 3, 2, 'h')`. It might seem odd to have them in the language, but they are useful if you want to create lists that cannot be modified, especially by mistake.

Dictionaries In the list that we saw above we indexed elements by their position within the list. In a dictionary you assign a key to each entry that you can use to access it. So suppose you want to make a list of the number of days in each month. You could use a dictionary (shown by the curly braces): `>>> months = {'Jan': 31, 'Feb': 28, 'Mar': 31}` and then you access elements of the dictionary using their key, so `>>> months['Jan']` returns 31. Giving an incorrect key results in an exception error.

The function `months.keys()` returns a list of all the keys in the dictionary, which is useful for looping over all elements in a dictionary. The `months.values()` function returns a list of values instead, while `months.items()` gives a list of tuples containing everything. There are lots of other things you can do with dictionaries, and we shall see some of them when we use the dictionary in Chapter 6.

There is one more data type that is built directly into Python, and that is the `file`. This makes reading from and writing to files very simple in Python: files are opened using `>>> input = open('filename')`, closed using `>>> input.close()` and reading and writing are performed using `readlines()` (and `read()`, and `writelines()` and `write()`). There are also `readline()` and `writeline()` functions, that read and write one line at a time.

Copyrighted material

Hidden page

Hidden page

The most common loop is the `for` loop, which differs slightly from other languages in that it iterates over a list of values:

```
for var in set:
    commands
else:
    commands
```

There is one very useful command that goes with this `for` loop, which is the `range` command, which produces a list output. Its most basic form is simply `>>> range(4)`, which produces the list `[0, 1, 2, 3]`. However, it can also take 2 or 3 arguments, and works in the same way as in the slice command, but with commas between them instead of colons: `>>> range(start,stop,step)`. This can include going down instead of up a list, so `>>> range(5,-3,-2)` produces `[5, 3, 1, -1]` as output.

Finally, there is a `while` loop:

```
while condition:
    commands
else:
    commands
```

16.3.3 Functions

Functions are defined by:

```
def name(args):
    commands
    return value
```

The `return value` line is optional, but enables you to return values from the function (otherwise it returns `None`). You can list several things to return in the line with commas between them, and they will all be returned. Once you have defined a function you can call it from the command line and from within other functions. Python is case sensitive, so with both function names and variable names, `Name` is different to `name`.

As an example, here is a function that computes the hypotenuse of a triangle given the other two distances (`x` and `y`). Note the use of '#' to denote a comment:

Copyrighted material

```
def pythagorus(x,y):
    """ Computes the hypotenuse of two arguments"""
    h = pow(x**2+y**2,0.5)
    # pow(x,0.5) is the square root
    return h
```

Now calling `pythagorus(3,4)` gets the expected answer of `5.0`. You can also call the function with the parameters in any order provided that you specify which is which, so `pythagorus(y=4,x=3)` is perfectly valid. When you make functions you can allow for default values, which means that if fewer arguments are presented the default values are given. To do this, modify the function definition line: `def pythagorus(x=3,y=4):`

16.3.4 The `doc` String

The help facilities within Python are accessed by using `help()`. For help on a particular module, use `help('modulename')`. (So using `help(pythagorus)` in the previous example would return the description of the function that is given there). A useful resource for most code is the `doc` string, which is the first thing defined within the function, and is a text string enclosed in three sets of double quotes (`"""`). It is intended to act as the documentation for the function or class. It can be accessed using `>>> print functionname.__doc__`. The Python documentation generator `pydoc` uses these strings to automatically generate documentation for functions, in the same way that `javadoc` does.

16.3.5 `map` and `lambda`

Python has a special way of performing repeated function calls. If you want to apply the same function to every element of a list you don't need to loop over the elements of the list, but can instead use the `map` command, which looks like `map(function,list)`. This applies the function to every element of the list. There is one extra tweak, which is the fact that the function can be anonymous (created just for this job without needing a name) by using the `lambda` command, which looks like `lambda args : command`. A `lambda` function can only execute one command, but it enables you to write very short code to do relatively complicated things. As an example, the following instruction takes a list and cubes each number in it and adds 7:

```
map(lambda x:pow(x,3)+7,list)
```

Another way that `lambda` can be used is in conjunction with the `filter` command. This returns elements of a list that evaluate to `True`, so:

Copyrighted material

```
filter(lambda x:x>=2,list)
```

returns those elements of the list that are greater than or equal to 2. NumPy provides simpler ways to do these things for arrays of numbers, as we shall see.

16.3.6 Exceptions

Like other modern languages, Python allows for the trapping of exceptions. This is done through the `try ... except ... else` and `try... finally` constructions. This example shows the use of the most common version. For more details, including the types of exceptions that are defined, see a Python programming book.

```
try:
    x/y
except ZeroDivisonError:
    print "Divisor must not be 0"
except TypeError:
    print "They must be numbers"
except:
    print "Something unspecified went wrong"
else:
    print "Everything worked"
```

16.3.7 Classes

For those that wish to use it in this way, Python is fully object-oriented, and classes are defined (with their constructor) by:

```
class myclass(superclass):

    def __init__(self,args):

    def functionname(self,args):
```

If a superclass is not specified then the class does not inherit from elsewhere. The `__init__(self,args)` function is the constructor for the class. There can also be a destructor `__del__(self)`, although they are rarely used. Accessing methods from the class uses the `classname.functionname()` syntax. The `self` argument can be ignored in all function calls, since Python fills it in for you, but it does need to be specified in the function definition. Many of the

Copyrighted material

examples in the book are based on classes provided on the book website. You need to be aware that you have to create an instance of the class before you can run it. So to import and run the class you need to use:

```
import myclass
var = myclass.myclass()
var.function()
```

16.4 Using NumPy and Matplotlib

Most of the commands that are used in this book actually come from the NumPy and Matplotlib packages, rather than the basic Python language. More specialised commands are described thoughout the book in the places where they become relevant. There are lots of examples of performing tasks using the various functions within NumPy on its website. Getting information about functions within NumPy is easy, because there is a special command: `info()`; for example, to find out about the sum command, use `info(sum)`.

NumPy has a base collection of functions and then additional packages that have to be imported as well if you want to use them. To import the NumPy base library and get started you use:

```
>>> from numpy import *
```

16.4.1 Arrays

The basic data structure that is used for numerical work, and by far the most important one for the programming in this book, is the array. This is exactly like multi-dimensional arrays (or matrices) in any other language; it consists of one or more dimensions of numbers or chars. Unlike Python lists, the elements of the array all have the same type, which can be Boolean, integer, real, or complex numbers.

Arrays are made using a function call, and the values are passed in as a list, or set of lists for higher dimensions. Here are one-dimensional and two-dimensional arrays (which are effectively arrays of arrays) being made. Arrays can have as many dimensions as you like up to a language limit of 40 dimensions, which is more than enough for this book.

```
>>> myarray = array([4,3,2])
>>> mybigarray = array([[3, 2, 4], [3, 3, 2], [4, 5, 2]])
```

Copyrighted material

Hidden page

`r_[]` **and** `c_[]` Perform row and column concatenation, including the use of the slice operator: `r_[1:4,0,4] = array([1, 2, 3, 0, 4])`. There is also a variation on `linspace()` using a `j` in the last entry: `r_[2,1:7:3j] = array([ 2.  , 1.  , 4.  , 7.])`. This is another nice feature of NumPy that can be used with `arange()` and `mgrid()` as well. The `j` on the last value specifies that you want 3 equally spaced points starting at 0 and running up to (and including) 7, and the function works out the locations of these points for you. The column version is similar.

The array `a` used in the next set of examples was made using `>>> a = arange(6).reshape(3,2)`, which produces:

```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

Indexing elements of an array is performed using square brackets '[' and ']', remembering that indices start from 0. So `a[2,1]` returns `5` and `a[:,1]` returns `array([1, 3, 5])`. We can also get various pieces of information about an array and change it in a variety of different ways, as follows.

Getting information about arrays, changing their shape, copying them

---

`ndim(a)` Returns the number of dimensions (here 2).

`size(a)` Returns the number of elements (here 6).

`shape(a)` Returns the size of the array in each dimension (here `(3, 2)`). You can access the first element of the result using `shape(a)[0]`.

`reshape(a,(2,3))` Reshapes the array as specified. Note that the new dimensions are in brackets. One nice thing about `reshape()` is that you can use '-1' for 1 dimension within the reshape command to mean 'as many as is required.' This saves you doing the multiplication yourself. For this example, you could use `reshape(a,(2,-1))` or `reshape(a,(-1,2))`.

`ravel(a)` Makes the array one-dimensional (here `array([0, 1, 2, 3, 4, 5])`).

`transpose(a)` Compute the matrix transpose. For the example:

```
[[0 2 4]
 [1 3 5]]
```

`a[::-1]` Reverse the elements of each dimension.

`a.min()`, `a.max(a)`, `a.sum(a)` Returns the smallest or largest element of the matrix, or the sum of the elements. Often used to sum the rows or columns using the `axis` option: `a.sum(axis=0)` for columns and `a.sum(axis=1)` for rows.

Copyrighted material

Hidden page

Hidden page

`linalg.pinv(a)` Compute the pseudo-inverse, which is defined even if `a` is not square

`linalg.det(a)` Compute the determinant of `a`

`linalg.eig(a)` Compute the eigenvalues and eigenvectors of `a`

16.4.4 Plotting

The plotting functions that we will be using are in the Matplotlib package. These are designed to look exactly like the MATLAB plotting functions. The entire set of functions, with examples, are given on the Matplotlib webpage, but the two most important ones that we will need are `plot` and `hist`. In order to use Matplotlib, you have to import it. For some reason it is called `pylab`, so the relevant command is `>>> from pylab import *`, which gives you access to the plotting commands. Matplotlib also provides some MATLAB functionality, and sometimes this can overwrite NumPy functions. The best way to avoid this is to ensure that you always import PyLab before NumPy. When producing plots they sometimes do not appear. This is usually because you need to specify the command `>>> ion()` which turns interactive plotting on. If you are using Matplotlib within Eclipse it has a nasty habit of closing all of the display windows when the program finishes. To get around this, issue a `show()` command at the end of your function.

The basic plotting commands of Matplotlib are demonstrated here, for more advanced plotting facilities see the package webpage.

The following code (best typed into a file and executed as a script) computes a Gaussian function for values -2 to 2.5 in steps of 0.01 and plots it, then labels the axes and gives the figure a title:

```
from pylab import *
from numpy import *

gaussian = lambda x: exp(-(0.5-x)**2/1.5)
x = arange(-2,2.5,0.01)
y = gaussian(x)
plot(x,y)
xlabel('x values')
ylabel('exp(-(0.5-x)**2/1.5')
title('Gaussian Function')
show()
```

The output of running this piece of code is shown in Figure 16.1.

Copyrighted material

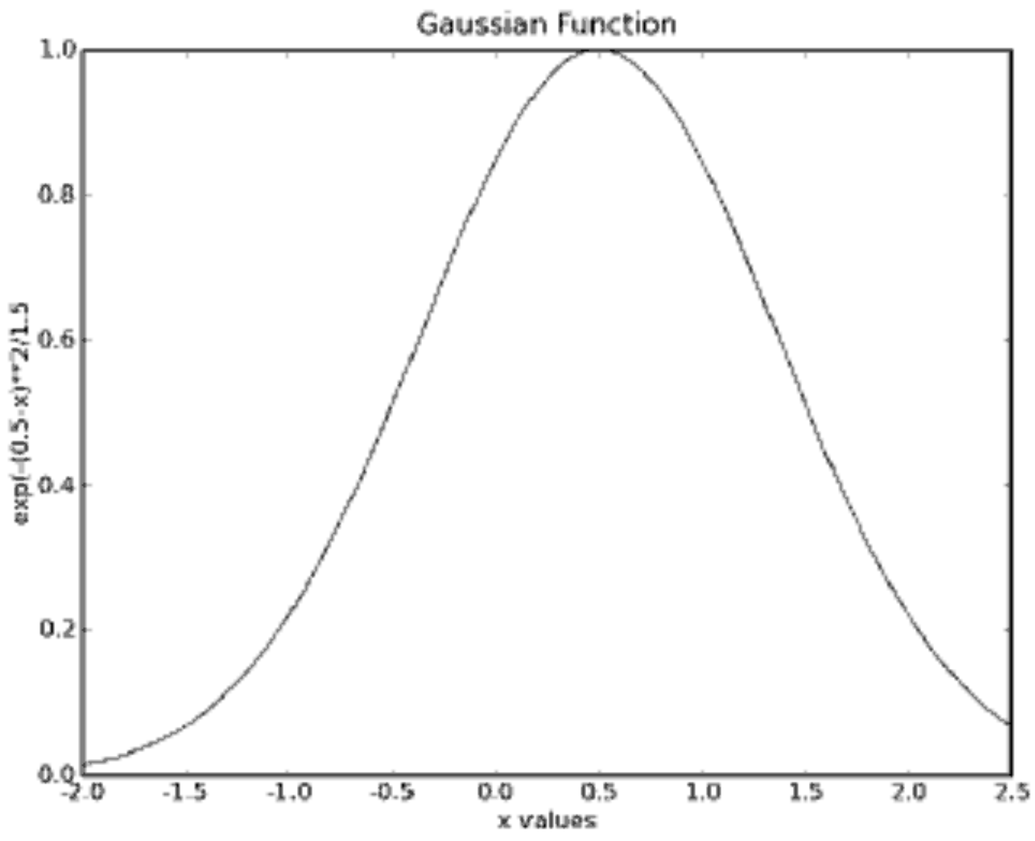


FIGURE 16.1: The Matplotlib package produces useful graphical output, such as this plot of the Gaussian function.

Further Reading

Python has become incredibly popular for both general computing and scientific computing. Because writing extension packages for Python is simple (it does not require any special programming commands: any Python module can be imported as a package, as can packages written in C), many people have done so, and made their code available on the Internet. Any search engine will find many of these, but a good place to start is the Python Cookbook website.

If you are looking for more complete introductions to Python, some of the following may be useful:

- M.L. Hetland. *Beginning Python: From Novice to Professional.* Apress Inc., Berkeley, CA, USA, 2nd edition, 2008.
- G. van Rossum and F.L. Drake Jr., editors. *An Introduction to Python.* Network Theory Ltd, Bristol, UK, 2006.
- W.J. Chun. *Core Python Programming.* Prentice-Hall, New Jersey, USA, 2006.
- B. Eckel. *Thinking in Python.* Mindview, La Mesa, CA, USA, 2001.
- T. Oliphant. Guide to NumPy, e-book, 2006. The official guide to NumPy by its creator.

Copyrighted material

Practice Questions

- Make an array `a` of size 6×4 where every element is a 2.
- Make an array `b` of size 6×4 that has 3 on the leading diagonal and 1 everywhere else. (You can do this without loops.)
- Can you multiply these two matrices together? Why does `a * b` work, but not `dot(a,b)`?
- Compute `dot(a.transpose(),b)` and `dot(a,b.transpose())`. Why are the results different shapes?
- Write a function that prints some output on the screen and make sure you can run it in Eclipse or whichever environment you are using.
- Now write one that makes some random arrays and prints out their sums, the mean value, etc.
- Write a function that consists of a set of loops that run through an array and count the number of ones in it. Do the same thing using the `where()` function (use `info(where)` to find out how to use it).

Copyrighted material

Index

k-Means Algorithm, 196
    algorithm, 197
    implementation, 198
    neural network, 200–202
        algorithm, 204
        example, 205
1-of-N encoding, 58, 74, 76

Absorbing state, 297
Action selection, 301
Action space, 298
Activation function, 13, 14, 19, 45, 52–54, 58, 59, 63, 74, 85, 87, 88
    Gaussian, 99
AdaBoost, 155, 157
    algorithm, 156
    example, 157, 158
    implementation, 156
    regression, 160
Approximate inference, 337, 339, 340, 343
Arcing, 158
Artificial Intelligence, ix, 5, 289, 361
Auto-associative network, 80–82
Auto-mpg dataset, 43
Autoencoder, *see* Auto-associative network
Average, 174

B-spline, 114, 115
Back-propagation, 50, 84, 88, 103
Backup, 306
Bagging, 160–162
    algorithm, 161
    bragging, 163
    example, 161
    subagging, 162
Basis expansion, 108
Batch training, 59
Baum-Welch algorithm, 353
    algorithm, 354
    implementation, 355
Bayes classifier, 194
Bayes' Optimal Classification, 171
Bayes' rule, 169, 170
Bayesian Belief network, *see* Bayesian network
Bayesian network, 334, 337, 341–343, 345, 363
Bayesian Optimisation Algorithm, 288
Belief propagation, 340
Bias input, 22, 29, 50
Bias-variance tradeoff, 177
Binary threshold device, 14
Binary tree, 133, 187
Black hole, 2, 3
Blind source separation, 237
Boltzman distribution, 265
Boltzman selection, 274
Boosting, 146, 154–156, 160, 162, 165
Bootstraps, 160, 161, 308
Box-Muller Scheme, 317, 318
Brain, 5, 11–13, 15, 18, 207
Breastcancer dataset, 165
Building block hypothesis, 276, 292
Bump function, 64

C4.5, 135, 142, 143
Camel, 153
CART, *see* Classification and Regression Trees
Central limit theorem, 176, 238

Copyrighted material

Chess, 302
City-block distance, 191, 192
Classical conditioning, 12
Classification, 9, 63, 74, 75, 170
Classification and Regression Trees, 145, 147
Clustering, 196, 200, 205, 222
Cocktail party problem, 237
Codebook, 207
Coins, 10
Committee, 153
Competitive learning, 201, 203, 205–207, 209, 210, 215
Compression, 63, 70
Computational complexity, 5, 24
Conditional independence, 334, 336, 338, 345
Conditional probability table, 334, 340, 344
Confusion matrix, 32, 39, 69, 171
Conjugate gradients, 257, 268
  algorithm, 260
  example, 260, 261
Connect-4, 313
Covariance, 174
Covariance matrix, 174–178, 223, 228, 229, 235
CPU dataset, 152
Critic, 7
Cross-validation, 67, 68, 114, 154
  leave-one-out, 68, 83
Crossover, 275, 276, 280, 281, 286, 287
Cubic spline, 108, 112, 115
Curse of dimensionality, 10, 106, 107, 115, 171, 172, 184, 202, 221, 298, 305, 306, 322

DAG, *see* Directed acyclic graph
Data compression, 80, 82, 206, 317
  lossy, 206
Data mining, 5, 130
Data Preprocessing, 63
Decision boundary, 11, 31–33, 36, 64, 122, 123, 125, 153, 154, 157, 183
Decision tree, 133–136, 139, 140, 142–144, 147, 149, 151–153, 157, 167, 194, 222
  C4.5, *see* C4.5
  Classification and Regression Tree, *see* Classification and Regression Tree
  classification example, 147
  computational complexity, 143
  construction, 134
  ID3, *see* ID3
  implementation, 140
Delaunay triangulation, 207
Density estimation, 7
Determinant, 105
Dijkstra's algorithm, 244
Dimensionality reduction, 81, 209, 222–224, 227, 231, 235, 239, 245
Directed acyclic graph, 335
Discounting, 300, 307
Discriminant function, *see* Decision boundary, 45
Distance measures, 190
DNA, 2
Dynamic Bayesian network, 347
Dynamic programming, 261

E-coli dataset, 218, 219
Early stopping, 69, 72
EDA, *see* Estimation of Distribution Algorithms
Eigenvalue, 228–230, 240, 246, 380
Eigenvector, 225, 229, 230, 240, 380
Elitism, 277, 278
EM, *see* Expectation-Maximisation algorithm
Ensemble, 155
Ensemble learning, 153
Entropy, 135–137, 146, 148, 151, 238, 317
  implementation, 135

Copyrighted material

Epanechnikov quadratic kernel, 185
Episodic learning, 300
Error function, 19, 21, 42, 48, 51, 53, 59, 60, 85–87, 247
  ϵ-insensitive, 129
  external, 195
  sum-of-squares, 44, 51, 53, 59, 67, 69, 80, 85, 112, 129, 147, 178, 199
Estimation of Distribution Algorithms, 286, 290
Euclidean distance, 97, 98, 106, 176, 190–192, 197
Evolution, 7, 269, 270, 272, 281, 284
Evolutionary learning, 7, 269, 286
Excitatory connection, 15
Exhaustive search, 261, 282
  computational complexity, 261
Expectation, 174, 183
Expectation-Maximisation algorithm, 179, 180, 235, 332, 344, 353
  algorithm, 182
Exploitation, 264, 265, 270, 273, 277, 296, 301, 302, 311
Exploration, 264, 265, 270, 273, 278, 296, 301, 302, 306, 311
Extended Kalman filter, 360

Factor analysis, 234, 235, 237
  example, 236
  implementation, 236
Factorised Distribution Algorithm, 288
False positive, 70
Feature derivation, 222
Feature mapping, 208, 214
Feature selection, 41, 44, 222
Fitness function, 272, 278, 280, 283–285
Fitness landscape, 269, 272, 284
Fletcher-Reeves formula, 258
Floyd's algorithm, 244
Forward algorithm, 351
Forward-backward algorithm, *see* Baum-Welch algorithm
Four peaks fitness function, 282
Function approximation, 8, 108, 207, 251

GA, *see* Genetic Algorithm
Gaussian Mixture Model, 178, 180, 182, 332
  algorithm, 181
  implementation, 181
Generalisation, 4–7, 17, 66, 69, 84, 200
Genetic Algorithm, 269–273, 276–280, 288, 289, 291, 343
  algorithm, 279
  example, 279, 282
  implementation, 273, 274, 278
  limitations, 284
Genetic operators, 275, 276, 280
Genetic Programming, 270, 285–287, 290
Geodesic, 239
Gibbs sampling, 328, 330, 331, 341, 343
  algorithm, 330
  example, 342
Gini impurity, 146, 152, 157, 165
GMM, *see* Gaussian Mixture Model
GP, *see* Genetic Programming
Gradient descent, 50, 51, 60, 85–87, 117, 124, 247, 248, 261, 265, 285, 291
Gram matrix, 127
Gram-Schmidt process, 258
Grandmother cell, 201
Graphical model, 333, 334, 336, 337, 344, 357
Greedy search, 262, 263, 266

Hard-max function, 74
Hebb's rule, 12
Heisenberg Uncertainty Principle, 178
Hessian, 252, 253

Copyrighted material

Hidden layer, 48–50, 53, 81, 85, 95, 103
Hidden Markov Model, 304, 334, 347, 349, 352–354, 356, 362–364
Hill climbing, 262, 263
  implementation, 263
HMM, *see* Hidden Markov Model
Horse, 153

ICA, *see* Independent Components Analysis
ID3, 135, 136, 139, 142, 143, 147, 151
  algorithm, 139
Identity matrix, 42
Image denoising, 345, 346, 361
  Auto-associative network, 81
  Markov Random Field
    algorithm, 346
Importance sampling, 322
Impurity, 135
  Gini, *see* Gini impurity
Independent Components Analysis, 237, 238, 245
Indicator function, 99
Indicator variable, 41
Inference, 150, 335–337, 340, 342, 344, 361
Infinite, *see* Loop
Information gain, 136, 137, 139, 142, 143, 145, 146, 148, 149
  implementation, 137
Information theory, 135, 136, 150, 206
Inhibitory connection, 15
Interpolation, 8, 108
Intrinsic dimensionality, 214
Invariant metrics, 192
Iris dataset, 75, 92, 194, 204, 205, 218, 226, 234, 236, 241, 244, 246
Irreducible error, 178
Isomap, 221, 242, 244, 245
  algorithm, 243

Jacobian, 252, 253, 256

Kalman filter, 334, 357, 362
  algorithm, 358
  example, 358, 359
  tracking, 359
Kalman gain, 358
Karush-Kuhn-Tucker conditions, 125
KD-Tree, 186, 187, 190, 194, 197
  example, 187, 189
  implementation, 187
Kernel classifier, 37
Kernel function, 119, 125, 130
Kernel Principal Components Analysis, 232
  algorithm, 233
  example, 234
  implementation, 233
Kernel trick, 127, 128, 232
KISS, *see* Minimum Description Length
Knapsack problem, 271–273, 282

Lagrange multipliers, 124, 130
Law of Effect, 293
LDA, *see* Linear Discriminant Analysis
Learning
  definition, 6
Learning rate, 21, 22, 26, 39, 62, 86, 211, 212, 306
Learning Vector Quantisation, 207
Least-squares, 42, 112–114, 239, 248, 251, 252
  example, 255, 256
Levenberg-Marquardt algorithm, 252, 254
  algorithm, 255
  example, 257
  implementation, 256
Likelihood, 179, 182, 335, 344
Line search, 249, 258
  implementation, 250
Linear Congruential Generator, 316
Linear discriminant, 17

Copyrighted material

Linear Discriminant Analysis, 44, 218, 223, 224
  example, 226
  implementation, 225
Linear regression, 37, 41, 42
  example, 43
  implementation, 42
Linear separability, 32, 34, 47, 125, 232
LLE, *see* Locally Linear Embedding
Local minimum, 51, 59–61, 69, 265
Locally Linear Embedding, 221, 239, 245
  algorithm, 240
  example, 241
  implementation, 241
Logical satisfiability, 268
Long-Term Potentiation, 13
Loop, *see* Infinite
Loss function
  exponential, 158
  sum-of-squares, 157
Loss matrix, 171

Mahalanobis distance, 175, 176
Majority voting, 154, 162
Manhattan distance, *see* City-block distance
Manifold, 243, 245
MAP, *see* Maximum a posteriori
Map colouring, 279, 291
Margin, 121–124, 128, 131
Markov blanket, 341
Markov chain, 304, 315, 325, 327, 328, 348
  aperiodic, 325
  detailed balance, 326
  ergodic, 325
  irreducible, 325
Markov Chain Monte Carlo, 161, 315, 325, 326, 331, 340, 341
Markov Decision Process, 302–304, 325
Markov model, 364
Markov property, 302, 303
Markov Random Field, 333, 334, 344–346
Maximum a posteriori, 170–172
Maximum likelihood, 179, 180, 235
Maximum Margin Classifier, 121
McCulloch and Pitts neuron, 13–15, 17, 19, 20, 100
MCMC, *see* Markov Chain Monte Carlo
MDL, *see* Minimum Description Length
MDP, *see* Markov Decision Process
MDS, *see* Multi-Dimensional Scaling
Mercer's Theorem, 127
Mersenne Twister, 316
Metropolis algorithm, 327, 328
Metropolis-Hastings algorithm, 326
  algorithm, 326
Mexican hat, 209, 210
MIMIC, 288
Minimum Description Length, 142, 343
Minkowski metric, 191
Misclassification impurity, 152
Mixture of experts, 163, 164
  algorithm, 164
MLP, *see* Multi-layer Perceptron
MNIST dataset, 45, 117, 131, 194
Momentum, 61, 62
Monte Carlo principle, 319
Morphometrics, 115, 116
Mount Ruapehu dataset, 185
MRF, *see* Markov Random Field
MRI, 2
Multi-Dimensional Scaling, 242, 243
  algorithm, 243
  example, 244
  Kruskal-Shephard scaling, 242
  Sammon mapping, 242
Multi-layer Perceptron, 47–50, 54, 56, 58–60, 62, 63, 67, 68, 70, 73, 80, 83, 84, 91, 92, 95, 231, 291

Copyrighted material

algorithm, 54
example
classification, 74–77
data compression, 80, 81
regression, 70–72
time-series prediction, 77–80
implementation, 56
learning capability, 64
training by genetic algorithm, 285
Multivariate tree, 143
Music Genome Project, 220
Mutation, 271, 280, 281, 286
Mutual information, 238, 288

N-armed bandit, 264
Naïve Bayes' Classifier, 171, 172
example, 172
Nearest Neighbour Smoothing, 185
Nearest Neighbours, 183
Neighbourhood, 210–213, 220, 239, 241, 244
Neuron, 11–15, 18, 19, 21, 53, 65, 74, 87, 88, 98, 202
Newton-Raphson iteration, 258–260
Niching, 277, 278
No Free Lunch theorem, 247, 261, 289
Normalisation, 40, 63, 71, 76, 83, 99, 156, 202, 321
Normalised Gaussians, 105
Novelty detection, 10, 75, 91
NP, 261, 268, 271, 337, 343

Observation probability, 348, 351, 352
Occam's Razor, 142, 343
On-line learning, 213
Optimal separation, 120
Optimisation, 42, 60, 61, 112, 114, 125, 179, 247, 248, 251, 290
discrete, 261, 279
OR logic function, 24, 32
Outliers, 63, 158, 163, 200
Overfitting, 66, 67, 69, 84, 105, 108, 113, 128, 142, 177, 199, 200, 215
Ozone dataset, 79

Parity problem, 45
Partially Observable Markov Decision Process, 304
Particle filter, 324, 334, 356, 360–362
Pattern recognition, 17, 91, 361, 362
PBIL, *see* Population-Based Incremental Learning
PCA, *see* Principal Components Analysis
Perceptron, 18–20, 22, 23, 25, 26, 31–42, 44, 47–49, 52, 54, 70, 103, 105
algorithm, 23
example, 24
implementation, 26, 28
Pima Indian dataset, 37, 40, 92, 117, 193
Polak-Ribiere formula, 259
Policy, 296, 302, 305–307, 309
ϵ-greedy, 301, 302, 308–310, 314
off-policy, 307
on-policy, 307
Polytree, 337
POMDP, *see* Partially Observable Markov Decision Process
Population-Based Incremental Learning, 288
example, 289
implementation, 288
Positive definite, 127
Posterior, 169, 170, 315
Prediction, 1
Premature convergence, 278
Preprocessing, 192
Principal Components Analysis, 81, 213, 226, 227
algorithm, 229
example, 231

Copyrighted material

implementation, 230
Prior, 167, 169, 170, 340, 344
Proposal distribution, 320, 321, 323, 324, 326–329
Prostate dataset, 46
Pruning, 142, 143, 222
Pseudo-inverse, 105, 380
Pseudo-random numbers, 316
Punctuated equilibrium, 281, 282
Pythagorus' theorem, 42

Q-learning, 307, 309–312, 314
  algorithm, 307
Quadratic form, 240, 249
Quadratic programming, 124, 130

Radial Basis Function, 95, 99–101, 103, 106, 108, 116, 117, 127, 164
  algorithm, 103
  implementation, 104
Random numbers, 276, 317
  creating, 316
  Gaussian, 317
  testing, 317
Random walk, 325, 327
Randomised algorithm, 174
RBF, *see* Radial Basis Function
Receptive field, 95, 97–100, 108
Reconstruction error, 239
Recurrent network, 47, 93
Recursion, *see* Recursion
Regression, 8, 63, 70, 129, 147, 160
Regularisation, 113
Reinforcement learning, 7, 293, 294, 296, 301–304, 306, 309, 311–313
Rejection sampling, 321, 322, 340
  algorithm, 321
Relevance Vector Machine, 130
Reward function, 293, 294, 299, 301, 311, 313
Risk, 142, 146, 157, 171, 193
Robust statistic, 200
ROC curve, 70
Rosenbrock's function, 255, 268, 291
Royal Road fitness function, 292

Sampling, 270, 288, 316, 319–326, 328, 330, 341–343
Sampling-importance-resampling, 360
  algorithm, 323
  implementation, 323
Sarsa, 308, 310, 312
  algorithm, 308
Scatter, 223–225
Scrabble, 5
Self-organisation, 214
Self-Organising Map, 207–210, 213–215, 221
  algorithm, 210
  boundary conditions, 214, 215
  example, 216, 218
  implementation, 212
Sensitivity, 70
Sequential Minimal Optimisation, 130
Sequential update, 60
Sigmoid, 52, 54, 58, 64, 87–89, 127
Simulated annealing, 265, 301, 327, 328
  implementation, 266
Singular value decomposition, 253
SIR, *see* Sampling-importance-resampling
Slack variable, 124, 130
Slice operator, 79, 368, 377
Smoothing, 114, 115
Smoothing spline, 113–115
Soft-Margin Classifier, 124
Soft-max, 58, 59, 74, 105, 274, 301, 302
SOM, *see* Self-Organising Map
Specificity, 70
Spectral decomposition, 229
Spike train, 15
Spline, 108, 111, 112, 114–116, 178
Standardisation, 40, 63
State space, 293, 298, 313–315
Statistics, 161
Steepest descent, 250, 252, 259

Copyrighted material

example, 251
Hessian, 252
Strong AI, 12
Stumping, 160, 161
Supervised learning, 1, 6, 7, 17, 19, 20, 223, 294
Support vector, 121, 122, 131
Support Vector Machine, 37, 47, 119, 120, 125, 232
example, 128
extensions, 128
regression, 129
SVM, *see* Support Vector Machine
Swissroll dataset, 242, 244, 245
Symbolic processing, 5
Synapse, 12–14, 18

Tangent distance, 192–194
Target vector, 19
Taylor series, 251, 253
TD(λ), 308
TD-Gammon, 312
TD-learning, 307
Test set, 31, 67, 71, 76, 80
Thin-plate spline, 115
Thought experiment, 98
Time-series prediction, 63, 77, 79
Topology preservation, 208
Tournaments, 277, 278
Tracking, 356, 360, 361
Training data, 7
Training set, 31, 67, 71, 76, 80
Transition probability, 348, 351
Travelling Salesman Problem, 247, 261–263, 268
example, 266
Trellis, 351, 352
Tricube kernel, 185
Truncation selection, 273
Trust region, 249, 251, 254
TSP, *see* Travelling Salesman Problem
Two-norm, 98

UCI repository, x, 37, 75, 152, 165, 218
Uniform distribution, 321, 322
Uniform learning, 58
Unsupervised learning, 7, 103, 167, 178, 195, 205, 209

Validation set, 67, 69, 71–73, 76, 80, 84, 105, 127, 142, 200
Value, 305
Vapnik-Chernik dimension, 127
Variable elimination algorithm, 339
algorithm, 338
example, 337
Variance, 174
Vector quantisation, 206, 207
Viterbi algorithm, 352, 363
algorithm, 352
Voronoi tesselation, 207
Voting, *see* Majority voting

Weak learners, 155
Weight decay, 62
Weight space, 95–97, 100, 196, 200
Wine dataset, 131, 220, 246
Winner-takes-all, 201

XOR logic function, 34, 36, 47, 49, 117

Yeast dataset, 131, 220, 246

Copyrighted material

Computer Science/Computer Engineering

Chapman & Hall/CRC
Machine Learning & Pattern Recognition Series

Drawing from computer science, statistics, mathematics, and engineering, the multidisciplinary nature of machine learning is underscored by its applicability to areas ranging from finance to biology and medicine to physics and chemistry. **Machine Learning: An Algorithmic Perspective** provides an introduction to the field of machine learning covering all of the major areas, including neural networks, graphical models, reinforcement learning, evolutionary algorithms, dimensionality reduction methods, and the important area of optimization.

The book treads the fine line between adequate academic rigor and overwhelming readers with equations and mathematical concepts. The author addresses the topics in a practical way while providing complete information and references where other expositions can be found. He describes algorithms with code examples backed up by a website that provides working implementations in Python. The book also contains data from a variety of applications to demonstrate the methods.

Features

- Provides a clear introduction to the basic concepts of machine learning
- Focuses on algorithms and applications and uses explanation rather than equations and mathematical concepts
- Presents real-world problems through structured exercises and programming examples in Python
- Contains a chapter that introduces the use of Python

Written in an easily accessible style, this book bridges the gaps between disciplines, providing the ideal blend of theory and practical, applicable knowledge.

CRC Press
Taylor & Francis Group
an **informa** business
www.crcpress.com

6000 Broken Sound Parkway, NW
Suite 300, Boca Raton, FL 33487
270 Madison Avenue
New York, NY 10016
2 Park Square, Milton Park
Abingdon, Oxon OX14 4RN, UK

C6718

ISBN: 978-1-4200-6718-7
90000
9 781420 067187

Copyrighted material